A Practical Guide to G-Confid 1.07

Laurie-Anne D'Amours and Peter Wright

Statistics Canada

Version 16/12/2020

Table of contents

Overview

G-Confid is a Statistics Canada generalized system that prevents the release of confidential information in tabular data using the method of cell suppression. Preventing the release of this information is done by identifying the sensitive cells to be protected and finding appropriate complementary cells to suppress in order to protect the sensitive cells. In addition G-Confid may be used to audit cell suppression patterns, find sensitive aggregates and round tabular data.

In G-Confid the cells of a table are determined by specifying the dimensions of the table and for each dimension the hierarchy of levels to be included in the table. Hierarchies may be specified using intervals of values and may also have overlapping levels.

To determine the level of protection needed by a given cell, the cell's sensitivity is calculated. G-Confid may be used to calculate a cell's sensitivity according to commonly used sensitivity rules including the *p*-percent rule and the (*n,k*) rule, as well as the sets of rules used within Statistics Canada. Arbitrary linear sensitivity measures may also be calculated. In addition G-Confid may be used to determine sensitive aggregates of cells that are not necessarily the margins of any table.

To protect the sensitive cells, additional cells may need to be suppressed. G-Confid uses a linear programming algorithm to find complementary cells to suppress that protect the sensitive cells while attempting to minimize a cost function. G-Confid will also take into account cells which are forced to be published or suppressed. The weights used in the cost function may also be specified.

G-Confid consists of seven components: one SAS procedure and six SAS macros. All components are built in-house and work the same way as other SAS procedures and macros.

1. SENSITIVITY procedure
    - Calculates the sensitivities of the cells of the specified table.
    - Written in the C language and interfaces with SAS through the SAS/Toolkit C language functions.
2. %SUPPRESS macro
    - Finds a suppression pattern to protect sensitive cells.
    - Uses the SAS OPTMODEL procedure to solve the linear programming problems.
3. %AUDIT macro
    - Verifies that a suppression pattern protects the sensitive cells.
    - Recommended if manual adjustments are made to the pattern
4. %AGGREGATE macro
    - Determines sensitive aggregates of cells.
5. %REPORTCELLS macro
    - Creates a report which identifies primary and secondary cell suppressions.
6. %GCONFIDROUND macro
    - Creates a rounded table which is close to the original unrounded table.
7. %GCONFIDOPTROUND macro
    - Create an optimally rounded table which is close to the original unrounded table.

The components are easy to use and are well documented. They offer a methodology that is known and approved at Statistics Canada. The software is fully supported by a team of programmers and methodologists who continuously work at improving the software and are ready to offer technical and methodological support.

Statistics Canada also offers a version of G-Confid for use with SAS Enterprise Guide.

Objectives and structure of the document

The purpose of this document is to provide users with basic knowledge to use G-Confid, as well as examples and theory to which they can refer for more complex problems. There are six main sections in this document:

1. Getting started:	Very simple example for new users to demonstrate the main steps involved in using G-Confid.
2. Examples:	Provides several examples on different topics. This section focuses on practical application and can be a good starting point for learning how to use G-Confid in different contexts.
3. Details:	Reference section with details on using G-Confid and some of its options.
4. Strategies:	This section also focuses on the practical application to common problems that may require a particular strategy when using G-Confid.
5. Syntax:	Reference section with a list of the G-Confid components, along with a description of the options and statements for each component.
6. FAQ:	Answers to the most frequently asked questions.

For new users, we recommend beginning with the 1. Getting Started section, then reading the examples on 2.2 Sensitivity rules and 2.3 Hierarchies. Then, depending on the user's needs, they should consult other parts of the 2. Examples section. The 3. Details, 4. Strategies and 6. FAQ sections should be consulted for specific questions. The 5. Syntax section is provided for reference purposes, and it is not necessary to read it from start to finish.

We hope that this document will help new G-Confid users get started and find solutions to their most common problems. The G-Confid team is always available to answer your questions, and we welcome any suggestions you may have to improve our document. If you wish to submit a question, comment or suggestion, please contact us by email at G-Confid / G-Confid (STATCAN).

We hope you find this manual useful.

1. Getting started

Suppose we have data for 13 enterprises that are distributed across 2 industries and 3 regions. We would like to publish a two-way table that describes the revenue by type of industry and region.

Data file

EntID	Industry	Region	Value
1	1	A	28
1	1	C	1
2	2	C	3
2	2	A	2
3	1	B	1
3	2	B	1
3	1	C	3
4	1	A	5
4	2	A	2
5	1	A	2
6	1	C	1
7	2	B	1
8	2	C	3
9	2	C	3
10	1	B	1
11	2	B	1
12	1	B	1
13	2	A	2

Two-way table

	Industry		Total
Region	1	2	
A	35	6	41
B	3	3	6
C	5	9	14
Total	43	18	61

Publishing the two-way table as-is could result in disclosing confidential data of one or more responding enterprises. In this example, we will demonstrate the main use of G-Confid to guarantee the confidentiality of these data. The main G-Confid components are PROC SENSITIVITY and the %SUPPRESS macro. Additionally, the %AUDIT macro is important when making alterations to results of the %SUPPRESS macro. This example highlights these three processes, as well as the %REPORTCELLS macro. We should note that the example is somewhat simple and aims to demonstrate the general steps of using G-Confid.

Step 1: Creating the data file in SAS

The following code is used to create the SAS file *Ex_data* that contains the data previously described. It is important that the unique identifier (*Entid*) and the dimension variables (*Industry* and *Region*) be in character format and that the variable of interest (*Value*) be in numeric format so that the file may be used in the next step within PROC SENSITIVITY. Also, note that the data needs to be in skinny format (see Section 2.1 for more information).

```
data Ex_data;
length Entid $2.;
Entid='1';  Industry="1"; Region="A"; Value=28; output;
Entid='1';  Industry="1"; Region="C"; Value=1;  output;
Entid='2';  Industry="2"; Region="C"; Value=3;  output;
Entid='2';  Industry="2"; Region="A"; Value=2;  output;
Entid='3';  Industry="1"; Region="B"; Value=1;  output;
Entid='3';  Industry="2"; Region="B"; Value=1;  output;
Entid='3';  Industry="1"; Region="C"; Value=3;  output;
Entid='4';  Industry="1"; Region="A"; Value=5;  output;
Entid='4';  Industry="2"; Region="A"; Value=2;  output;
Entid='5';  Industry="1"; Region="A"; Value=2;  output;
Entid='6';  Industry="1"; Region="C"; Value=1;  output;
Entid='7';  Industry="2"; Region="B"; Value=1;  output;
Entid='8';  Industry="2"; Region="C"; Value=3;  output;
Entid='9';  Industry="2"; Region="C"; Value=3;  output;
Entid='10'; Industry="1"; Region="B"; Value=1;  output;
Entid='11'; Industry="2"; Region="B"; Value=1;  output;
Entid='12'; Industry="1"; Region="B"; Value=1;  output;
Entid='13'; Industry="2"; Region="A"; Value=2;  output;
run;
```

Ex_data

	Entid	Industry	Region	Value
1	1	1	A	28
2	1	1	C	1
3	2	2	C	3
4	2	2	A	2
5	3	1	B	1
6	3	2	B	1
7	3	1	C	3
8	4	1	A	5
9	4	2	A	2
10	5	1	A	2
11	6	1	C	1
12	7	2	B	1
13	8	2	C	3
14	9	2	C	3
15	10	1	B	1
16	11	2	B	1
17	12	1	B	1
18	13	2	A	2

Step 2: Calling PROC SENSITIVITY

Once the data have been entered into a SAS file, we can call PROC SENSITIVITY to determine which cells in our two-way table are sensitive. The following code is used and generates two output data files: *Outcell* and *Outconstraint*.[1]

```
proc sensitivity
   data=Ex_data
   outcell=outcell
   outconstraint=outconstraint
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	0.8	S	C	35	1	A
2	2	0	0	3	5	-0.7	V	C	5	1	C
3	3	0	0	3	9	-2.7	V	C	9	2	C
4	4	0	0	3	6	-1.8	V	C	6	2	A
5	5	0	0	3	3	-0.9	V	C	3	1	B
6	6	0	0	3	3	-0.9	V	C	3	2	B
7	7	0	0	13	61	-22.1	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-6.1	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-9.5	V	C	18	2	TOT_REGION
10	10	0	0	5	41	-3.2	V	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.8	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-7.7	V	C	14	TOT_INDUSTRY	C

Outconstraint

	ConstraintId	CellId	Coefficient
1	1	8	1
2	1	9	1
3	1	7	-1
4	2	10	1
5	2	11	1
6	2	12	1
7	2	7	-1
8	3	1	1
9	3	4	1
10	3	10	-1
11	4	5	1
12	4	6	1
13	4	11	-1
14	5	2	1
15	5	3	1
16	5	12	-1
17	6	1	1
18	6	5	1
19	6	2	1
20	6	8	-1
21	7	4	1
22	7	6	1
23	7	3	1
24	7	9	-1

Some explanation of the code that is used:

- In DATA, we enter the name of our data file.
- OUTCELL and OUTCONSTRAINT are used to name the files containing the results of the procedure. These files will be used in the next step with the macro %SUPPRESS.
- In SRULE, we enter the rule to use that calculates the sensitivity of the cells of the table. Different rules exist to calculate the sensitivity of the cells of a table, so here we use a “PQ 0.1” rule. For more information on the different rules that are available, see the 2.2 Sensitivity rules section.
- Two dimensions appear in our two-way table: *Industry* and *Region*. To specify them in PROC SENSITIVITY, we must both write the names of these variables in DIMENSION, and also specify the categories pertaining to each dimension in the HIERARCHY, including the totals. See 2.3 Hierarchies for more information on how to define correctly a hierarchy in PROC SENSITIVITY.
- In ID, the unique identifier is to be specified, which in this case is *Entid*.
- In VAR, we specify the name of the numeric variable of interest, which here is called *Value*.
- To conclude, note that many other options exist in PROC SENSITIVITY, but they are not described here to keep it simple.

[1] Note: As of G-Confid 1.07.002 the cell total is called *TotalVar* instead of *Total* in the *Outcell* file.

Key things to know about the *Outcell* file:

- In the *Outcell* file, each row represents a cell of the two-way table. Moreover, each cell receives an assigned serial number identified in the variable *CellId*.
- For each cell, the number of respondents is calculated, the cell total and the sensitivity.
- A cell with sensitivity greater than zero is considered sensitive, regardless of the sensitivity rule that is used. If a cell is sensitive, it receives the value "S" in the variable *Status*. We therefore observe that the cell referring to industry 1 and region A is sensitive. Non-sensitive cells take the value "V".

Key things to know about the *Outconstraint* file:

- This file describes the linear constraints of the two-way table that is specified.
- The *CellId* variable corresponds to the variable with the same name in the *Outcell* file. It follows that we observe in the *Outcell* file that cell 7 corresponds to the grand total, where (*Industry*="TOT_INDUSTRY" and *Region*="TOT_REGION").
- Each constraint *ConstraintId* represents a linear equation. There are seven equations in use here.
- For each constraint, the sum of the cells each with a coefficient of 1 must equal the value of the cell with coefficient -1.
- For example, constraint 1 indicates that the sum of cells 8 and 9 should equal the value of cell 7. Put another way, the sum of the two subtotals of industries 1 and 2 should equal the value of the grand total.
- This file is especially useful for the next steps of the example.

Step 3: The %SUPPRESS macro

To begin with, we identified one sensitive cell in our two-way table. However, suppressing the value of this cell is not enough to ensure confidentiality. If we suppressed this value and did nothing with the rest of the two-way table, it would be easy to deduce the value of the cell from the other information available in the table.

Two-way table

	Industry		Total
Region	1	2	
A	**X**	6	41
B	3	3	6
C	5	9	14
Total	43	18	61

To ensure confidentiality of the cell, we need to suppress the values of other cells so that it is not possible to estimate too precisely the value of the sensitive cell. There are different ways to choose the other cells to suppress; these are called "suppression patterns". The purpose of the %SUPPRESS macro is to choose a suppression pattern that minimizes the loss of information, but is the most efficient in terms of time and computer resources. The suppression pattern chosen is not necessarily optimal in terms of the loss of information, since it would be too costly in terms of time and computer resources to test all possible patterns, especially for large or complex files.

Coming back to our example, the simplest possible code for using the %SUPPRESS macro would be the one presented below. We simply enter the filenames *Outcell* and *Outconstraint* in the previous step and specify the desired output filename, i.e., *Outpattern*. Of course, many other options could be specified in this macro, but they are not presented here.

```
%suppress(
   InCell       = outcell,
   Constraint   = outconstraint,
   OutCell      = outpattern
);
```

Outpattern

	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity	Status	Type	total	Industry	Region
1	X	0.4	1	0	0	3	35	0.8	S	C	35	1	A
2	P	0	2	0	0	3	5	-0.7	V	C	5	1	C
3	P	0	3	0	0	3	9	-2.7	V	C	9	2	C
4	X	0.4	4	0	0	3	6	-1.8	V	C	6	2	A
5	X	0.4	5	0	0	3	3	-0.9	V	C	3	1	B
6	X	0.4	6	0	0	3	3	-0.9	V	C	3	2	B
7	P	0	7	0	0	13	61	-22.1	V	C	61	TOT_INDUSTRY	TOT_REGION
8	P	0	8	0	0	7	43	-6.1	V	C	43	1	TOT_REGION
9	P	0	9	0	0	8	18	-9.5	V	C	18	2	TOT_REGION
10	P	0	10	0	0	5	41	-3.2	V	C	41	TOT_INDUSTRY	A
11	P	0	11	0	0	5	6	-2.8	V	C	6	TOT_INDUSTRY	B
12	P	0	12	0	0	6	14	-7.7	V	C	14	TOT_INDUSTRY	C

The *Outpattern* file produced here closely resembles the *Outcell* file in the previous step, but it contains the new variable *Outstatus*. This variable has a value of "X" if the cell is suppressed and a value of "P" if the cell is published. In our example, we see that a total of four cells are suppressed, including the one that was identified as sensitive.

At the same time, the %SUPPRESS macro produces a table of results:

	Number	Value	Percent of total number of cells
All suppressed cells	4	47	33.33
Suppressed sensitive cells	1	35	8.33
Suppressed complements	3	12	25.00
Cells suppressed by user	0	0	0.00
Suppressed aggregates	0	0	N/A
Published cells	8	197	66.67

This table confirms what we observed in the *Outpattern* file: one sensitive cell was suppressed, and three other cells were suppressed to protect it.

To view the suppression pattern chosen, we can use the %REPORTCELLS macro. Simply enter the name of the cross-tabulated variables and the variable from the *Outpattern* file.

```
%reportCells(
        colname=Industry,
        rowname=Region,
        incell=outpattern
);
```

Suppression pattern

	Industry		
	1	2	TOT_INDUSTRY
Region			
A	35	6	41
B	3	3	6
C	5	9	14
TOT_REGION	43	18	61

The sensitive cell is highlighted in red, and the other cells selected for suppression are highlighted in yellow.

The resulting file would be as follows:

Two-way table

	Industry		Total
Region	1	2	
A	**X**	**X**	41
B	**X**	**X**	6
C	5	9	14
Total	43	18	61

Step 4: The %AUDIT macro

In practice, the %AUDIT macro should be used when we decide to change something in the suppression pattern provided by %SUPPRESS. Since the %SUPPRESS macro does not necessarily identify the best suppression pattern in terms of loss of information, it may be useful to make changes to the chosen suppression pattern in order to reduce the loss of information when the additional cells are suppressed. However, by making these changes, there is a risk of choosing a suppression pattern that no longer guarantees the confidentiality of the data. To check the validity of a modified suppression pattern, we can use the %AUDIT macro. As was the case for PROC SENSITIVITY and %SUPPRESS, many other options are available in the %AUDIT macro, but they are not discussed here for the sake of conciseness.

Initial suppression pattern

Going back to our example, let us see what the %AUDIT macro does with the suppression pattern provided by the %SUPPRESS macro. We enter into the macro the filenames *Outpattern* and *Outconstraint*, then the name we want to assign to the file with the results of the %AUDIT macro, i.e., *Outaudit*.

```
%Audit (
   InCell           = outpattern,
   Constraint       = outconstraint,
   OutCell          = outaudit
);
```

*Outaudit**

	IntervalWidth	MaxMinusMin	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity	Status	Type	total	Industry	Region	ProblemIndicator
1	8.5714285714	3	X	0.4	1	0	0	3	35	0.8	S	C	35	1	A	0
2	50	3	X	0.4	4	0	0	3	6	-1.8	V	C	6	2	A	0
3	100	3	X	0.4	5	0	0	3	3	-0.9	V	C	3	1	B	0
4	100	3	X	0.4	6	0	0	3	3	-0.9	V	C	3	2	B	0

*The *Outaudit* file contains more variables than what is displayed here.

At the same time, the %AUDIT macro also generates a table indicating whether the protection of the suppressed cells is good (based on the variable *ProblemIndicator* in the previous file). We can see here that the cells have good protection.

A. Summary of suppressed cells and sensitive aggregates by problem indicator

Problem indicator	Number of sensitive cells	Number of complements	Number of user suppressed cells (used as complements)	Number of user suppressed cells (not used as complements)	Number of sensitive aggregates	Total
Good protection	1	3	0	0	0	4
Protection not achieved	0	N/A	N/A	N/A	0	0
Exact disclosure	0	0	0	0	0	0
	1	**3**	**0**	**0**	**0**	**4**

Modified suppression pattern

Let us suppose that a user has incorrectly modified the suppression pattern and applies the %AUDIT macro to verify the new pattern. An example of an incorrect modification would be keeping the same suppression pattern, but publishing the result for the cell representing region B and industry 2. Let's take a look at what would happen in this situation.

Two-way table

	Industry		Total
Region	1	2	
A	**X**	**X**	41
B	**X**	3	6
C	5	9	14
Total	43	18	61

```
data Outpattern2;
set Outpattern;
if Industry="2" and Region="B" then OutStatus="P";
run;

%Audit (
   InCell           = outpattern2,
   Constraint       = outconstraint,
   OutCell          = outaudit2
);
```

*Outaudit2**

	IntervalWidth	MaxMinusMin	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity	Status	Type	total	Industry	Region	ProblemIndicator
1	0	0	X	0.4	1	0	0	3	35	0.8	S	C	35	1	A	2
2	0	0	X	0.4	4	0	0	3	6	-1.8	V	C	6	2	A	2
3	0	0	X	0.4	5	0	0	3	3	-0.9	V	C	3	1	B	2

* The *Outaudit2* file contains more variables than what is displayed here.

We notice that there are only three lines in the data file since only three cells are now suppressed. This time, in the table generated by the %AUDIT macro, we see that the three cells are in an exact disclosure situation, which is exactly what we would expect, given that the disclosure of one of the four suppressed cells in our example would make it possible to deduce the values of all the other suppressed cells.

A. Summary of suppressed cells and sensitive aggregates by problem indicator

Problem indicator	Number of sensitive cells	Number of complements	Number of user suppressed cells (used as complements)	Number of user suppressed cells (not used as complements)	Number of sensitive aggregates	Total
Good protection	0	0	0	0	0	0
Protection not achieved	0	N/A	N/A	N/A	0	0
Exact disclosure	1	2	0	0	0	3
	1	2	0	0	0	3

2. Examples

2.1 Transposing a file into skinny format

This first example is for users less familiar with SAS who want to create a file in skinny format so that it can be used by G-Confid. Although this process is not specific to G-Confid, we thought it would be useful to include an example in our documentation, since transposing a file is one of the first obstacles our users may encounter.

What is meant by 'skinny format'?

To perform its analyses, G-Confid requires that files be in skinny format. This means that the variable of interest, such as revenue, must appear in a single column. Below is an example of a file transposition to skinny format:

Original file

ENTID	GEO	A	B	C
S1	ON	31	25	32
S1	QC	33	32	34
S1	SK	29	23	30
S2	ON	32	35	32
S2	QC	30	20	36
S2	SK	33	30	33

*where *A*, *B* and *C* are revenues for various domains of interest

File in skinny format

ENTID	GEO	DOMAIN	REVENUE
S1	ON	A	31
S1	ON	B	25
S1	ON	C	32
S1	QC	A	33
S1	QC	B	32
S1	QC	C	34
S1	SK	A	29
S1	SK	B	23
S1	SK	C	30
S2	ON	A	32
S2	ON	B	35
S2	ON	C	32
S2	QC	A	30
S2	QC	B	20
S2	QC	C	36
S2	SK	A	33
S2	SK	B	30
S2	SK	C	33

Once the file is in skinny format, we can then perform analyses in G-Confid for revenue by province and domain. There are different ways to transpose a file into skinny format. Here we will only be showing how to use PROC TRANSPOSE, since we feel it is the easiest and most effective method:

```
/*Creation of the base file*/
data base_datafile;
ENTID="S1"; GEO="ON"; A=31; B=25; C=32; output;
ENTID="S1"; GEO="QC"; A=33; B=32; C=34; output;
ENTID="S1"; GEO="SK"; A=29; B=23; C=30; output;
ENTID="S2"; GEO="ON"; A=32; B=35; C=32; output;
ENTID="S2"; GEO="QC"; A=30; B=20; C=36; output;
ENTID="S2"; GEO="SK"; A=33; B=30; C=33; output;
run;

/*Transposing the file from base format to skinny format*/
proc transpose data=base_datafile
   out=out_datafile(rename=(_NAME_=DOMAIN col1=REVENUE));
   var A B C;
   by ENTID GEO;
run;
```

Simply put, to use PROC TRANSPOSE, we needed to enter variables *A*, *B* and *C* in the VAR statement and specify in BY the other two columns we wanted to keep (*ENTID* and *GEO*). Note that the RENAME= option enables us to assign the desired names to the *FIELD* and *REVENUE* variables; otherwise, they would have been named '*_NAME_*' and '*col1*' respectively by default.

2.2 Sensitivity rules

In the 1. Getting Started section, we presented a basic example that included 13 enterprises. In that example, we indicated that, for each cell in the two-way table, we were calculating the sensitivity (denoted here as *S*) based on the rule specified in the SRULE option of PROC SENSITIVITY. Keep in mind that a cell is considered to be sensitive when $S > 0$; otherwise, it is protected.

Using the same example, we will show the different rules available in G-Confid to calculate cell sensitivity. The data in that example may be generated as follows:

```
data Ex_data;
length Entid $2.;
Entid='1';  Industry="1"; Region="A"; Value=28; output;
Entid='1';  Industry="1"; Region="C"; Value=1;  output;
Entid='2';  Industry="2"; Region="C"; Value=3;  output;
Entid='2';  Industry="2"; Region="A"; Value=2;  output;
Entid='3';  Industry="1"; Region="B"; Value=1;  output;
Entid='3';  Industry="2"; Region="B"; Value=1;  output;
Entid='3';  Industry="1"; Region="C"; Value=3;  output;
Entid='4';  Industry="1"; Region="A"; Value=5;  output;
Entid='4';  Industry="2"; Region="A"; Value=2;  output;
Entid='5';  Industry="1"; Region="A"; Value=2;  output;
Entid='6';  Industry="1"; Region="C"; Value=1;  output;
Entid='7';  Industry="2"; Region="B"; Value=1;  output;
Entid='8';  Industry="2"; Region="C"; Value=3;  output;
Entid='9';  Industry="2"; Region="C"; Value=3;  output;
Entid='10'; Industry="1"; Region="B"; Value=1;  output;
Entid='11'; Industry="2"; Region="B"; Value=1;  output;
Entid='12'; Industry="1"; Region="B"; Value=1;  output;
Entid='13'; Industry="2"; Region="A"; Value=2;  output;
run;
```

Ex_data

	Entid	Industry	Region	Value
1	1	1	A	28
2	1	1	C	1
3	2	2	C	3
4	2	2	A	2
5	3	1	B	1
6	3	2	B	1
7	3	1	C	3
8	4	1	A	5
9	4	2	A	2
10	5	1	A	2
11	6	1	C	1
12	7	2	B	1
13	8	2	C	3
14	9	2	C	3
15	10	1	B	1
16	11	2	B	1
17	12	1	B	1
18	13	2	A	2

Keep in mind that this is the resulting two-way table, for which we wish to check confidentiality:

Two-way table

	Industry		Total
Region	1	2	
A	35	6	41
B	3	3	6
C	5	9	14
Total	43	18	61

Before we begin

As previously stated, a sensitivity rule determines whether a cell is sensitive. It also determines the level of protection the cell needs to be properly protected, which is very important in choosing a suppression pattern. There are five rules that can be specified in the SRULE option of PROC SENSITIVITY: PQ, NK, DUFFETT, C2 and ARB.

These rules can be expressed as follows:

$$S = \sum_{i=1}^{N} \alpha_i x_i,$$

Where

- S is a cell sensitivity
- If $S > 0$ then the cell is sensitive and must be protected
- N is the number of contributors in a cell
- $x_i, i = 1,2, \dots, N$ is the value of the i^{th} contributor to a cell, with contributions given in decreasing order ($x_1 \geq x_2 \geq \cdots \geq x_N \geq 0$)
- $\alpha_i, i = 1,2, \dots, N$ are the standard coefficients for the sensitivity rule chosen.

The PQ rule

The PQ rule is often used in practice. Its simplified form is named 'p-percent rule', where the *q* has the value 100%. The PQ rule and p-percent rule are frequently used interchangeably since we generally use a *q* value of 100%.[2] A cell is considered sensitive if the second largest contributor can estimate within ±(*pq*)% the value of the largest contributor. In PROC SENSITIVITY, we only need to specify the value of *pq*. The PQ rule can be expressed as follows:

$$S = (pq) \cdot x_1 - \sum_{i \geq 3} x_i \quad \text{where } 0 \leq pq \leq 1$$

This rule may be generalized using the following values for α_i:

$$\alpha_i = \begin{cases} pq & \text{if } i = 1 \\ 0 & \text{if } i = 2 \\ -1 & \text{if } i \geq 3 \end{cases}$$

As explained above, the idea behind this rule is to verify whether the value of the largest enterprise could easily be estimated by the second largest. If the values of x_i for $i \geq 3$ are relatively small in relation to x_1, then it would be easy for enterprise 2 to have a good estimate of x_1 by subtracting its value from the cell total.

For information purposes, in this type of scenario, the largest contributor is named the "target", while the second is named the "intruder" or "suspect" and the others are considered to provide "noise".

[2] For a more complete definition of the PQ rule, refer to Willenborg and De Waal (2001, p. 141).

Now, let's looks at our example. There are three contributors for industry 1 and region A:

Entid	Value	Role
1	28	Target
4	5	Intruder / Suspect
5	2	Noise

With a p-percent rule, where *p*=0.1, we obtain $S = 0.1 \times 28 - 2 = 0.8$, which is considered to be sensitive. It is also the only sensitive cell identified in the table. We can verify this with the following code in G-Confid:

```
proc sensitivity
   data=Ex_data
   outcell=outcell_PQ1
   outconstraint=outconstraint_PQ1
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_pq1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	0.8	S	C	35	1	A
2	2	0	0	3	5	-0.7	V	C	5	1	C
3	3	0	0	3	9	-2.7	V	C	9	2	C
4	4	0	0	3	6	-1.8	V	C	6	2	A
5	5	0	0	3	3	-0.9	V	C	3	1	B
6	6	0	0	3	3	-0.9	V	C	3	2	B
7	7	0	0	13	61	-22.1	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-6.1	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-9.5	V	C	18	2	TOT_REGION
10	10	0	0	5	41	-3.2	V	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.8	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-7.7	V	C	14	TOT_INDUSTRY	C

If a larger value is used for *p*, the rule is stricter. For example, when *p*=0.3, $S = 0.3 \times 28 - 2 = 6.4$. If we examine the SAS results presented below, we see that with this new value of *p*, the total for region A becomes sensitive. Note that the procedure also identifies a sensitive aggregate, which can be recognized by the additional line displayed at the bottom of the *Outcell* file where *Type*="A". See the examples on 2.4 Aggregates for more information on aggregates.

```
proc sensitivity
   data=Ex_data
   outcell=outcell_PQ2
   outconstraint=outconstraint_PQ2
   srule="pq 0.3"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_pq2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	6.4	S	C	35	1	A
2	2	0	0	3	5	-0.1	V	C	5	1	C
3	3	0	0	3	9	-2.1	V	C	9	2	C
4	4	0	0	3	6	-1.4	V	C	6	2	A
5	5	0	0	3	3	-0.7	V	C	3	1	B
6	6	0	0	3	3	-0.7	V	C	3	2	B
7	7	0	0	13	61	-16.3	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-0.3	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-8.5	V	C	18	2	TOT_REGION
10	10	0	0	5	41	2.4	S	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.4	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-7.1	V	C	14	TOT_INDUSTRY	C
13	13	0	0	5	40	2.7	S	A	40		

The NK rule

Another rule often used in practice is the NK rule, also referred to as the NK dominance rule. A cell is considered to be sensitive if the sum of the largest *n* contributions accounts for more than *k* percent of the total cell value. The user must provide the two parameters: *n* and *k*.

For the specified parameters *n* and *k*, sensitivity is defined as follows:

$$S = \frac{100}{k}\sum_{i=1}^{n} x_i - \sum_{i=1}^{N} x_i = \left(\frac{100}{k} - 1\right)\sum_{i=1}^{n} x_i - \sum_{i=n+1}^{N} x_i$$

where $n \geq 1, N \geq 1$ and $50 < k \leq 100$

This rule can be generalized using the following values for α_i:

$$\alpha_i = \begin{cases} \frac{100}{k} - 1 & if \quad 1 \le i \le n \\ -1 & if \quad n+1 \le i \le N \end{cases}$$

Coming back to our example, if *n*=1 and *k*=70, then we get sensitivity $S = \left(\frac{100}{70} - 1\right) \cdot 28 - 5 - 2 = 5$ for the cell for industry 1 and region A. This is also what we obtain in SAS using the following code:

```
proc sensitivity
   data=Ex_data
   outcell=outcell_NK1
   outconstraint=outconstraint_NK1
   srule="NK 1 70"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_nk1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	5	S	C	35	1	A
2	2	0	0	3	5	-0.714285714	V	C	5	1	C
3	3	0	0	3	9	-4.714285714	V	C	9	2	C
4	4	0	0	3	6	-3.142857143	V	C	6	2	A
5	5	0	0	3	3	-1.571428571	V	C	3	1	B
6	6	0	0	3	3	-1.571428571	V	C	3	2	B
7	7	0	0	13	61	-19.57142857	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-1.571428571	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-10.85714286	V	C	18	2	TOT_REGION
10	10	0	0	5	41	-1	V	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-3.142857143	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-9.714285714	V	C	14	TOT_INDUSTRY	C
13	13	0	0	5	40	1.4285714286	S	A	40		

With a rule that *n*=2 and *k*=85, the sensitivity of this same cell would be $S = \left(\frac{100}{85} - 1\right) \cdot (28 + 5) - 2 = 3.82$.

```
proc sensitivity
   data=Ex_data
   outcell=outcell_NK2
   outconstraint=outconstraint_NK2
   srule="NK 2 85"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_nk2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	3.8235294118	S	C	35	1	A
2	2	0	0	3	5	-0.294117647	V	C	5	1	C
3	3	0	0	3	9	-1.941176471	V	C	9	2	C
4	4	0	0	3	6	-1.294117647	V	C	6	2	A
5	5	0	0	3	3	-0.647058824	V	C	3	1	B
6	6	0	0	3	3	-0.647058824	V	C	3	2	B
7	7	0	0	13	61	-18.64705882	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-3	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-8.588235294	V	C	18	2	TOT_REGION
10	10	0	0	5	41	0.1764705882	S	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.470588235	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-6.941176471	V	C	14	TOT_INDUSTRY	C

It is possible to use several NK rules at the same time. Up to three NK rules can be entered at the same time in PROC SENSITIVITY. In such a situation, for each cell, sensitivity is defined by the maximum of the sensitivities calculated for each NK rule. In our example, if we combine the rules (*n*=1, *k*=70) and (*n*=2, *k*=85), the cell for industry 1 and region A has a sensitivity $S = \max(5, 3.82) = 5$.

```
proc sensitivity
   data=Ex_data
   outcell=outcell_NK3
   outconstraint=outconstraint_NK3
   srule="NK 1 70 2 85"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_nk3

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	5	S	C	35	1	A
2	2	0	0	3	5	-0.294117647	V	C	5	1	C
3	3	0	0	3	9	-1.941176471	V	C	9	2	C
4	4	0	0	3	6	-1.294117647	V	C	6	2	A
5	5	0	0	3	3	-0.647058824	V	C	3	1	B
6	6	0	0	3	3	-0.647058824	V	C	3	2	B
7	7	0	0	13	61	-18.64705882	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-1.571428571	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-8.588235294	V	C	18	2	TOT_REGION
10	10	0	0	5	41	0.1764705882	S	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.470588235	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-6.941176471	V	C	14	TOT_INDUSTRY	C
13	13	0	0	5	40	1.4285714286	S	A	40		

Duffett rule

The Duffett rule was developed by and for Statistics Canada and can only be used internally. *This rule has hardly been used since 2009. In its place, we recommend using Statistics Canada's C2 rule. This section is therefore presented for information purposes only.* The Duffett rule is a combination of NK rules, for which the parameter details are confidential. To use the Duffett rule, it is not necessary to enter a parameter in the SRULE option of PROC SENSITIVITY. Sensitivity is defined as:

$S = \max(S_1, S_2)$, where

- S_1 is calculated using a rule (n=1, k=k_1)
- S_2 is calculated using a rule (n=2, k=k_2).

We show here the code to use the DUFFETT option (the results are not shown for confidentiality reasons):

```
proc sensitivity
   data=Ex_data
   outcell=outcell_Duffett
   outconstraint=outconstraint_Duffett
   srule="DUFFETT"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Given that the rule is still available in G-Confid, but not recommended, the following warning message appears when the rule is used:

```
WARNING: Sensitivity rule parser: Duffett is not recommended. You should use another sensitivity rule.
```

C2 rule

The C2 rule was also developed by and for Statistics Canada, and can only be used internally. It is a combination of the PQ and NK rules. The details of the parameters used in this rule are confidential. This rule does not require the input of a parameter in the SRULE option of PROC SENSITIVITY. This rule is recommended for economic data at Statistics Canada. Sensitivity is defined as follows:

$S = \max(S_D, S_P)$, where

- S_D is calculated using a rule similar to the Duffett rule
- S_P is calculated using a p-percent rule.

We show here the code to use the C2 option (the results are not shown for confidentiality reasons):

```
proc sensitivity
   data=Ex_data
   outcell=outcell_C2
   outconstraint=outconstraint_C2
   srule="C2"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

ARB (arbitrary) rule

With this rule, the user can manually specify the first four α_i values in the general form of the sensitivity equation. For $i \geq 5$, $\alpha_i = -1$. Sensitivity is then calculated as follows:

$$S = \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \alpha_4 x_4 - \sum_{i=5}^{N} x_i.$$

Although this rule offers a lot more flexibility in specifying parameters, it is important to pay attention when specifying them so that the results make sense. This option is mostly used for research purposes.

Below is an example of what would happen if $\alpha_1 = 0.3, \alpha_2 = 0$ and $\alpha_3 = -1$ (α_4 would be -1 by default since it is not specified):

```
proc sensitivity
   data=Ex_data
   outcell=outcell_ARB
   outconstraint=outconstraint_ARB
   srule="ARB 0.3 0 -1"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_arb

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	6.4	S	C	35	1	A
2	2	0	0	3	5	-0.1	V	C	5	1	C
3	3	0	0	3	9	-2.1	V	C	9	2	C
4	4	0	0	3	6	-1.4	V	C	6	2	A
5	5	0	0	3	3	-0.7	V	C	3	1	B
6	6	0	0	3	3	-0.7	V	C	3	2	B
7	7	0	0	13	61	-16.3	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-0.3	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-8.5	V	C	18	2	TOT_REGION
10	10	0	0	5	41	2.4	S	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-2.4	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-7.1	V	C	14	TOT_INDUSTRY	C
13	13	0	0	5	40	2.7	S	A	40		

As expected, the results are identical to those of the example of the PQ rule when *p*=0.3.

In practice, we might want to apply a PQ rule that accounts for the collusion of two contributors. To achieve this, we simply need to specify that $\alpha_1 = pq, \alpha_2 = \alpha_3 = 0, \alpha_4 = -1$ (not presented as it was not deemed relevant with the example on which this is based).

MINRESP rule (minimum number of respondents)

This final rule, a bit different from the others, determines whether or not a cell is sensitive based on the number of respondents. A cell containing a number of respondents below a certain threshold is considered to be sensitive, and this threshold is chosen by the user. Contrary to other sensitivity rules presented in this document, the MINRESP rule is not a SRULE option, but rather a different option that can be added to one of our other sensitivity rules.

We will now return to our first example of the NK rule. By adding a constraint of at least five respondents per cell, we see that several cells become sensitive:

```
proc sensitivity
   data=Ex_data
   outcell=outcell_NK1_MIN5
   outconstraint=outconstraint_NK1_MIN5
   srule="NK 1 70"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   minresp=5
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_nk1_min5

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	3	35	5	S	C	35	1	A
2	2	0	0	3	5	1	S	C	5	1	C
3	3	0	0	3	9	1	S	C	9	2	C
4	4	0	0	3	6	1	S	C	6	2	A
5	5	0	0	3	3	1	S	C	3	1	B
6	6	0	0	3	3	1	S	C	3	2	B
7	7	0	0	13	61	-19.57142857	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	7	43	-1.571428571	V	C	43	1	TOT_REGION
9	9	0	0	8	18	-10.85714286	V	C	18	2	TOT_REGION
10	10	0	0	5	41	-1	V	C	41	TOT_INDUSTRY	A
11	11	0	0	5	6	-3.142857143	V	C	6	TOT_INDUSTRY	B
12	12	0	0	6	14	-9.714285714	V	C	14	TOT_INDUSTRY	C

Notes:

- The MINRESP rule cannot be used on its own since the SRULE option in PROC SENSITIVITY is mandatory. To make sure that the sensitivity is based only on MINRESP, we can however use SRULE=”ARB -1”.
- When MINRESP determines that a cell does not contain enough respondents, the sensitivity of this cell is the maximum value between 1 and the sensitivity obtained using the rule specified in SRULE (G-Confid retains the higher of the two values, but if that value exceeds the cell total, then the sensitivity equals the total). That is why the cell for industry 1 and region A has a sensitivity of 5 rather than 1.

Which rule should be chosen?

For Statistics Canada users, we generally recommend using the C2 rule. However, this rule is not available outside Statistics Canada in order to protect its confidentiality.

G-Confid external users may refer to the standards of their own statistical organization to choose the best rule(s) for protecting the confidentiality of their data contributors. For those who have not yet developed such standards, it should be stated that normally, national statistical agencies, including Statistics Canada, never reveal the values of the parameters they use.

However, as a starting point for external users, INSEE (National Institute of Statistics and Economic Studies) identifies the following two rules to maintain the statistical confidentiality of business data:

1. No cell must pertain to less than three units.
2. No cell must contain data for which one enterprise represents more than 85% of the total.

To do this using G-Confid, we would specify MINRESP=3 and SRULE="NK 1 85".

We can, however, recommend a rule that is similar and modern: the p-percent rule with a value of 0.15, which guarantees that (1) at least three enterprises will contribute to the cell; and (2) the largest contribution cannot be guessed to within 15%. Using G-Confid, we would then specify that SRULE="PQ 0.15". In that case, it would not be necessary to specify MINRESP=3 since the PQ rule implicitly requires a minimum of three contributors.

Of course, that is merely a starting point. Any further strategy requires reflection based on the survey or data type, the need to protect respondents, and the expectations of the statistical organization involved.

2.3 Hierarchies

One of the most important, and at times most difficult, steps to perform when indicating instructions in PROC SENSITIVITY is specifying the data hierarchy. Using the following examples, we will try to show how it can be specified. All of the examples will be based on the same fictitious data.

Creation of a fictitious data file

The fictitious file on which we will be basing our examples can be created using the following code:

```
data Data_Hierarchy;
id=0;
do i=1 to 10;
  do j=1 to 2;
    do k=1 to 5;
      id=id+1;
      Entid=left(put(id,3.));
      if i=1 then Province="10";
      else if i=2 then Province="11";
      else if i=3 then Province="12";
      else if i=4 then Province="13";
      else if i=5 then Province="24";
      else if i=6 then Province="35";
      else if i=7 then Province="46";
      else if i=8 then Province="47";
      else if i=9 then Province="48";
      else Province="59";
      if j=1 then Industry="A";
      else Industry="B";
      Value=round(((ranuni(7005)+0.25)**7)*1000)+1;
      output;
    end;
  end;
end;
drop i j k id;
run;
```

Data_hierarchy

	Entid	Province	Industry	Value
1	1	10	A	529
2	2	10	A	103
3	3	10	A	1144
4	4	10	A	1
5	5	10	A	1672
6	6	10	B	1298
7	7	10	B	248
8	8	10	B	135
9	9	10	B	1477
10	10	10	B	16
11	11	11	A	22
12	12	11	A	769
13	13	11	A	1
14	14	11	A	403
15	15	11	A	3629
16	16	11	B	1873
17	17	11	B	463
18	18	11	B	3734
19	19	11	B	8
20	20	11	B	626
21	21	12	A	2

It is not important to understand this code, but rather to know the contents of the file named *Data_hierarchy* that has been generated. First of all, the file contains 100 enterprises, numbered from 1 to 100 in the variable *Entid* (the above image only shows the first 21). These enterprises are spread across 10 provinces (10, 11, 12, 13, 24, 35, 46, 47, 48, 59) and divided into two types of industries: A and B. The *Value* variable contains the revenue of each of the enterprises and this is our variable of interest.

Example 1: One single dimension variable

For this first example, we will assume that we only want to publish one table with the total revenues by province. To do this, we will use the following code in PROC SENSITIVITY:

```
proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex1
   outconstraint=outconstraint_ex1
   srule="pq 0.1"
   hierarchy="TOT_PROVINCE 10 11 12 13
              24 35 46 47 48 59;"
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

Outcell_ex1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province
1	1	0	0	10	6623	-3306.8	V	C	6623	10
2	2	0	0	10	11528	-3791.6	V	C	11528	11
3	3	0	0	10	13916	-4905.2	V	C	13916	12
4	4	0	0	10	1569	-155.4	V	C	1569	13
5	5	0	0	10	4507	-1607.9	V	C	4507	24
6	6	0	0	10	9450	-4580	V	C	9450	35
7	7	0	0	10	3358	86.6	S	C	3358	46
8	8	0	0	10	4413	-883.2	V	C	4413	47
9	9	0	0	10	4185	-583.5	V	C	4185	48
10	10	0	0	10	12351	-3814.3	V	C	12351	59
11	11	0	0	100	71900	-62363.3	V	C	71900	TOT_PROVINCE

A few notes regarding the HIERARCHY option:

- It is necessary to specify that we are interested in the *Province* variable in the DIMENSION statement of PROC SENSITIVITY. Without that, G-Confid would not know which variable to look at for our hierarchy.
- The total for each province is named "TOT_PROVINCE", but we could have given it any other name.
- It is important to first specify the name that we wish to give the total, before specifying the values of the categories (in this case the provinces). Otherwise, there is a risk of obtaining results that do not make sense in the *Outcell* file.
- If we had forgotten to name one of the provinces in the HIERARCHY, it would not have appeared in the *Outcell* file and the total would not have accounted for it. The total is always based on the values specified in the HIERARCHY option rather than the values in the input file.
- If we had added to the HIERARCHY a province that did not exist in the input file (e.g., province "70"), it would not have appeared in the *Outcell* file since it does not exist.
- Once the entire hierarchy has been specified, it is necessary to insert a ';' to indicate to PROC SENSITIVITY that everything has been specified.

We will now present two alternative ways that we could have written this code to obtain the same results.

Option B: Using a macro variable

It is frequent in practice to specify the hierarchy in a macro variable. This is particularly useful for writing complex hierarchies that extend over several lines. We will be using macro variables in our next examples to specify the hierarchy. Below is code that is equivalent to the preceding code:

```
%let hierarchy_ex1b="TOT_PROVINCE 10 11 12 13 24 35 46 47 48 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex1b
   outconstraint=outconstraint_ex1b
   srule="pq 0.1"
   hierarchy=&hierarchy_ex1b
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

As we can see, the macro variable named *hierarchy_ex1b* contains the text for specifying the hierarchy (%LET statement). The next step is to indicate the name of this macro variable preceded by '&' in the hierarchy of PROC SENSITIVITY.

Option C: Using the -1 shortcut

As previously mentioned, if we put non-existent values in the province hierarchy, these values will simply be ignored since they do not exist in the input file. So, if the hierarchy lists all of the numbers from 10 to 59, we should obtain the same results as when we specify only the provinces present in the input file. In the HIERARCHY option, there is a shortcut that can be used if the user wants to include all of the values from one number to another. The user simply needs to indicate the start number, then -1, then the end number. Let's see how the shortcut could have been used in this situation:

```
%let hierarchy_ex1c="TOT_PROVINCE 10 -1 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex1c
   outconstraint=outconstraint_ex1c
   srule="pq 0.1"
   hierarchy=&hierarchy_ex1c
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

Advantages of using the -1 shortcut:

- The code may be easier to write and read;
- Less risk of forgetting values that need to be included;
- May be easier to update the list of values to be included.

Disadvantages of using the -1 shortcut:

- Running time may be longer since PROC SENSITIVITY must test each of the values in the list;
- The user cannot exclude certain values from the list.

Note: The use of the -1 shortcut may be applied to a variable that takes only positive integers. The shortcut works even though the variable is in alphanumeric format, as required by G-Confid.

Example 2: Two dimension variables

Now let's assume that we want to publish a two-way table showing the revenues for our enterprises by province and by industry. To do this, we need to specify the name of the variables in DIMENSION and their respective hierarchies in HIERARCHY. Below is how the code could be written:

```
%let hierarchy ex2="TOT INDUSTRY A B;
TOT_PROVINCE 10 11 12 13 24 35 46 47 48 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex2
   outconstraint=outconstraint_ex2
   srule="pq 0.1"
   hierarchy=&hierarchy_ex2
   ;
   id Entid;
   var Value;
   dimension Industry Province;
run;
```

This code generates the *Outcell* file shown below. Each of the lines in the file corresponds to a cell in the two-way table that we wish to publish. In order, we have the different combinations for the *Industry* and *Province* variables, the grand total, the marginal total by industry, then the marginal total by province. It is important to indicate in the hierarchy that the industries are 'A' and 'B', and not 'a' and 'b' (since this variable is written in upper case in the input file). In the hierarchy, the categories for our dimension variables must be written exactly the same as they appear in the input file, otherwise they will not be recognized by G-Confid.

Outcell_ex2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Province
1	1	0	0	5	3449	-465.8	V	C	3449	A	10
2	2	0	0	5	3174	-251.3	V	C	3174	B	10
3	3	0	0	5	4824	-63.1	V	C	4824	A	11
4	4	0	0	5	6704	-723.6	V	C	6704	B	11
5	5	0	0	5	3352	213.8	S	C	3352	A	12
6	6	0	0	5	10564	-1553.2	V	C	10564	B	12
7	7	0	0	5	1432	-18.4	V	C	1432	A	13
8	8	0	0	5	137	-1.9	V	C	137	B	13
9	9	0	0	5	1552	133.1	S	C	1552	A	24
10	10	0	0	5	2955	-203.1	V	C	2955	B	24
11	11	0	0	5	2357	-639.5	V	C	2357	A	35
12	12	0	0	5	7093	-2223	V	C	7093	B	35
13	13	0	0	5	1257	106.7	S	C	1257	A	46
14	14	0	0	5	2101	181.6	S	C	2101	B	46
15	15	0	0	5	799	51.8	S	C	799	A	47
16	16	0	0	5	3614	-203.2	V	C	3614	B	47
17	17	0	0	5	1021	15	S	C	1021	A	48
18	18	0	0	5	3164	72.5	S	C	3164	B	48
19	19	0	0	5	5845	-464.2	V	C	5845	A	59
20	20	0	0	5	6506	-319.3	V	C	6506	B	59
21	21	0	0	100	71900	-62363.3	V	C	71900	TOT_INDUSTRY	TOT_PROVINCE
22	22	0	0	50	25888	-18548.1	V	C	25888	A	TOT_PROVINCE
23	23	0	0	50	46012	-36475.3	V	C	46012	B	TOT_PROVINCE
24	24	0	0	10	6623	-3306.8	V	C	6623	TOT_INDUSTRY	10
25	25	0	0	10	11528	-3791.6	V	C	11528	TOT_INDUSTRY	11
26	26	0	0	10	13916	-4905.2	V	C	13916	TOT_INDUSTRY	12
27	27	0	0	10	1569	-155.4	V	C	1569	TOT_INDUSTRY	13
28	28	0	0	10	4507	-1607.9	V	C	4507	TOT_INDUSTRY	24
29	29	0	0	10	9450	-4580	V	C	9450	TOT_INDUSTRY	35
30	30	0	0	10	3358	86.6	S	C	3358	TOT_INDUSTRY	46
31	31	0	0	10	4413	-883.2	V	C	4413	TOT_INDUSTRY	47
32	32	0	0	10	4185	-583.5	V	C	4185	TOT_INDUSTRY	48
33	33	0	0	10	12351	-3814.3	V	C	12351	TOT_INDUSTRY	59

Below are a few important notes regarding the code to enter in PROC SENSITIVITY when there are two or more dimensions:

- The variable names in DIMENSION must be specified in the same order that they are specified in HIERARCHY.
- Each dimension is placed on a different line in HIERARCHY, but the ';' symbol is what is important to separate the dimensions. Everything could have been written on the same line.
- When we are done writing the instructions in HIERARCHY for a dimension, we need to add a ';' symbol. This ensures that each dimension is separated by a ';' in HIERARCHY.
- There must be the same number of dimensions in DIMENSION and HIERARCHY. Otherwise, PROC SENSITIVITY generates an error message and does not run.

Example 3: Provinces and regions

We would now like to publish a table that contains the results by province and by region. For the purposes of this example, it will be assumed that the provinces are divided into five regions, and that the first number of the province identifier corresponds to the home region. To do this, we will use the following code:

```
%let hierarchy_ex3="
        TOT REGION 1 2 3 4 5:
        1 10 11 12 13:
        2 24:
        3 35:
        4 46 47 48:
        5 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex3
   outconstraint=outconstraint_ex3
   srule="pq 0.1"
   hierarchy=&hierarchy_ex3
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

Outcell_ex3

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province
1	1	0	0	10	6623	-3306.8	V	C	6623	10
2	2	0	0	10	11528	-3791.6	V	C	11528	11
3	3	0	0	10	13916	-4905.2	V	C	13916	12
4	4	0	0	10	1569	-155.4	V	C	1569	13
5	5	0	0	10	4507	-1607.9	V	C	4507	24
6	6	0	0	10	9450	-4580	V	C	9450	35
7	7	0	0	10	3358	86.6	S	C	3358	46
8	8	0	0	10	4413	-883.2	V	C	4413	47
9	9	0	0	10	4185	-583.5	V	C	4185	48
10	10	0	0	10	12351	-3814.3	V	C	12351	59
11	11	0	0	100	71900	-62363.3	V	C	71900	TOT_REGION
12	12	0	0	40	33636	-24625.2	V	C	33636	1
13	13	0	0	10	4507	-1607.9	V	C	4507	2
14	14	0	0	10	9450	-4580	V	C	9450	3
15	15	0	0	30	11956	-6276.5	V	C	11956	4
16	16	0	0	10	12351	-3814.3	V	C	12351	5

In this hierarchy, we see the use of the ':' symbol for the first time. These are used when a dimension has several levels. In this case, the most general level is the region, then each region is divided into provinces. Note:

- It is important to remember that the first term of each expression is supposed to be a total. So, when we write "4 46 47 48:", we are indicating that region 4 corresponds to the total of provinces 46, 47 and 48.
- Each expression has been placed on a different line in our example, but the ':' symbols are what is important to separate the expressions. Everything could have been written on the same line.
- We cannot forget the expression "TOT_REGION 1 2 3 4 5 :" in the hierarchy. PROC SENSITIVITY requires that all categories are connected and joined together to make a grand total. Otherwise, an error message appears explaining that there are multiple roots, and there can only be one root.

Example 4: Results by region only

In the previous example, we were interested in the results by region and by province, but we could easily be interested in the results by region only. A simple way to obtain results by region only is to create a new variable in our data file:

```
data Data_Hierarchy;
set Data_Hierarchy;
Region=substr(Province,1,1);
run;
```

Data_hierarchy

	Entid	Province	Industry	Value	Region
1	1	10	A	529	1
2	2	10	A	103	1
3	3	10	A	1144	1
4	4	10	A	1	1
5	5	10	A	1672	1
6	6	10	B	1298	1
7	7	10	B	248	1

```
proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex4
   outconstraint=outconstraint_ex4
   srule="pq 0.1"
   hierarchy="TOT_REGION 1 2 3 4 5;"
   ;
   id Entid;
   var Value;
   dimension Region;
run;
```

Outcell_ex4

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	40	33636	-24625.2	V	C	33636	1
2	2	0	0	10	4507	-1607.9	V	C	4507	2
3	3	0	0	10	9450	-4580	V	C	9450	3
4	4	0	0	30	11956	-6276.5	V	C	11956	4
5	5	0	0	10	12351	-3814.3	V	C	12351	5
6	6	0	0	100	71900	-62363.3	V	C	71900	TOT_REGION

We can see that this time, only the results by region are displayed. There are no results for provinces.

Option B: Using a range

There is another way to obtain the same results without having to modify the *Data_Hierarchy* file. Starting with the provinces only, we can use the RANGE option to combine provinces into regions without the provinces being displayed in the *Outcell* file:

```
proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex4b
   outconstraint=outconstraint_ex4b
   srule="pq 0.1"
   range="1 10 11 12 13:
          2 24:
          3 35:
          4 46 47 48:
          5 59;"
   hierarchy="TOT_REGION 1 2 3 4 5;"
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

Here, the RANGE option has the same form as the hierarchy presented in example 3 to explain which provinces are included in each region.

Option C: Using range and macro variables

Similar to specifying a hierarchy, we can create a macro variable for the RANGE option. By creating a macro variable for the RANGE and another macro for the HIERARCHY option, we can produce the same instructions as those presented in option B.

```
%let range_ex4c="1 10 11 12 13:
                 2 24:
                 3 35:
                 4 46 47 48:
                 5 59;";

%let hierarchy_ex4c="TOT_REGION 1 2 3 4 5;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex4c
   outconstraint=outconstraint_ex4c
   srule="pq 0.1"
   range=&range_ex4c
   hierarchy=&hierarchy_ex4c
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

Example 5: Range when there is more than one variable

Now let's assume that we want to determine the sensitivity of cells in a two-way table based on region and industry. We would like to know how to specify the range when there is more than one dimension.

First of all, it is important to note that the DIMENSION, HIERARCHY and RANGE options must have the same number of dimensions, and these must be specified in the same order. If there is nothing to be written in the range for one of the dimensions, a ';' symbol should be added to indicate that the range is blank for that dimension.

In the following code, we see that ';' is specified in the range before the instructions to classify provinces by region. The ';' symbol is placed here because the *Industry* variable is listed before the *Province* variable in the DIMENSION statement.

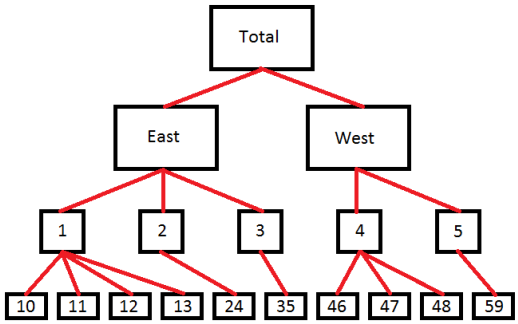
```
%let range_ex5="
         ;
         1 10 11 12 13:
         2 24:
         3 35:
         4 46 47 48:
         5 59;";

%let hierarchy_ex5="
         TOT_INDUSTRY A B;
         TOT_REGION 1 2 3 4 5;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex5
   outconstraint=outconstraint_ex5
   srule="pq 0.1"
   range=&range_ex5
   hierarchy=&hierarchy_ex5
   ;
   id Entid;
   var Value;
   dimension Industry Province;
run;
```

Outcell_ex5

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Province
1	1	0	0	20	13057	-6337.1	V	C	13057	A	1
2	2	0	0	20	20579	-11568.2	V	C	20579	B	1
3	3	0	0	5	1552	133.1	S	C	1552	A	2
4	4	0	0	5	2955	-203.1	V	C	2955	B	2
5	5	0	0	5	2357	-639.5	V	C	2357	A	3
6	6	0	0	5	7093	-2223	V	C	7093	B	3
7	7	0	0	15	3077	-1099.3	V	C	3077	A	4
8	8	0	0	15	8879	-3199.5	V	C	8879	B	4
9	9	0	0	5	5845	-464.2	V	C	5845	A	5
10	10	0	0	5	6506	-319.3	V	C	6506	B	5
11	11	0	0	100	71900	-62363.3	V	C	71900	TOT_INDUSTRY	TOT_REGION
12	12	0	0	50	25888	-18548.1	V	C	25888	A	TOT_REGION
13	13	0	0	50	46012	-36475.3	V	C	46012	B	TOT_REGION
14	14	0	0	40	33636	-24625.2	V	C	33636	TOT_INDUSTRY	1
15	15	0	0	10	4507	-1607.9	V	C	4507	TOT_INDUSTRY	2
16	16	0	0	10	9450	-4580	V	C	9450	TOT_INDUSTRY	3
17	17	0	0	30	11956	-6276.5	V	C	11956	TOT_INDUSTRY	4
18	18	0	0	10	12351	-3814.3	V	C	12351	TOT_INDUSTRY	5

Example 6: Multi-level hierarchy

Now, let's assume that we want to obtain the results by province, region and direction (East/West). We would like to create the following hierarchy:



Specifying this hierarchy is not really any more complex than when we had only the regions and provinces to indicate. To achieve this, we simply need to focus on the different red lines in the diagram. If we start at the top of the diagram, we first state that the total corresponds to the sum of East and West, then we indicate which regions make up East and West, and which provinces make up these regions. That is how we obtain the following code:

```
%let hierarchy_ex6="TOT_REGION EAST WEST:
                    EAST 1 2 3:
                    WEST 4 5:
                    1 10 11 12 13:
                    2 24:
                    3 35:
                    4 46 47 48:
                    5 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex6
   outconstraint=outconstraint_ex6
   srule="pq 0.1"
   hierarchy=&hierarchy_ex6
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

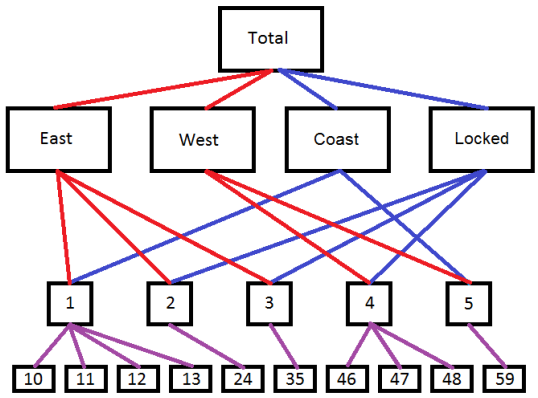
Outcell_ex6

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province
1	1	0	0	10	6623	-3306.8	V	C	6623	10
2	2	0	0	10	11528	-3791.6	V	C	11528	11
3	3	0	0	10	13916	-4905.2	V	C	13916	12
4	4	0	0	10	1569	-155.4	V	C	1569	13
5	5	0	0	10	4507	-1607.9	V	C	4507	24
6	6	0	0	10	9450	-4580	V	C	9450	35
7	7	0	0	10	3358	86.6	S	C	3358	46
8	8	0	0	10	4413	-883.2	V	C	4413	47
9	9	0	0	10	4185	-583.5	V	C	4185	48
10	10	0	0	10	12351	-3814.3	V	C	12351	59
11	11	0	0	100	71900	-62363.3	V	C	71900	TOT_REGION
12	12	0	0	60	47593	-38582.2	V	C	47593	EAST
13	13	0	0	40	24307	-15770.3	V	C	24307	WEST
14	14	0	0	40	33636	-24625.2	V	C	33636	1
15	15	0	0	10	4507	-1607.9	V	C	4507	2
16	16	0	0	10	9450	-4580	V	C	9450	3
17	17	0	0	30	11956	-6276.5	V	C	11956	4
18	18	0	0	10	12351	-3814.3	V	C	12351	5

Example 7: Multiple decomposition

We will now look at a more advanced example. Suppose we want to break down Canada geographically as we did in the previous example: by direction, then by region, followed by province at the lowest level of the hierarchy. However, our external client requests that we present separate results for the set of provinces with a coastline separate from the set of provinces that are landlocked. This decomposition represents an alternative to obtaining results by direction. G-Confid permits us to specify a hierarchy by regrouping cells at lower levels of the hierarchy in more than one way.

We can now reconstruct the hierarchy diagram as follows, where the red lines and blue lines represent two ways to calculate the total:



As we can see from the diagram, both the EAST + WEST set and the LOCKED + COAST set represent two different ways to regroup the five different regions (1=Atlantic, 2=Quebec, 3=Ontario, 4=Prairie, and 5=British Columbia). G-Confid only requires that the cells at the most detailed level of the hierarchy collectively represent the cell at the highest level of the hierarchy. We may regroup the intermediate levels in different ways, as we want. Moreover, if G-Confid detects that the cell at the highest level of the hierarchy is represented only by a subset of cells at the most detailed level of the hierarchy, G-Confid posts an error message in the SAS log.

Now let's write down the hierarchy. When describing the hierarchy in PROC SENSITIVITY, it is important to clarify each of the lines that appear in the above diagram. PROC SENSITIVITY needs to be able to link each of the categories to the grand total in order for everything to work properly. If a link is forgotten, SAS stops running the code and displays an error message that several roots have been detected, when only one is allowed. To get a good grasp of the categories and the links between them in the hierarchy, it may be quite useful to prepare a diagram.

If we take the time to specify each of the coloured lines in our diagram, we obtain the code and results below. In this case, we added three clarifications compared to example 6: COAST and LOCKED sum to the total, COAST is made up of regions 1 and 5, and LOCKED is made up of regions 2, 3 and 4.

```
%let hierarchy_ex7="TOT_REGION EAST WEST:
                    TOT_REGION COAST LOCKED:
                    EAST 1 2 3:
                    WEST 4 5:
                    COAST 1 5:
                    LOCKED 2 3 4:
                    1 10 11 12 13:
                    2 24:
                    3 35:
                    4 46 47 48:
                    5 59;";

proc sensitivity
   data=Data_Hierarchy
   outcell=outcell_ex7
   outconstraint=outconstraint_ex7
   srule="pq 0.1"
   hierarchy=&hierarchy_ex7
   ;
   id Entid;
   var Value;
   dimension Province;
run;
```

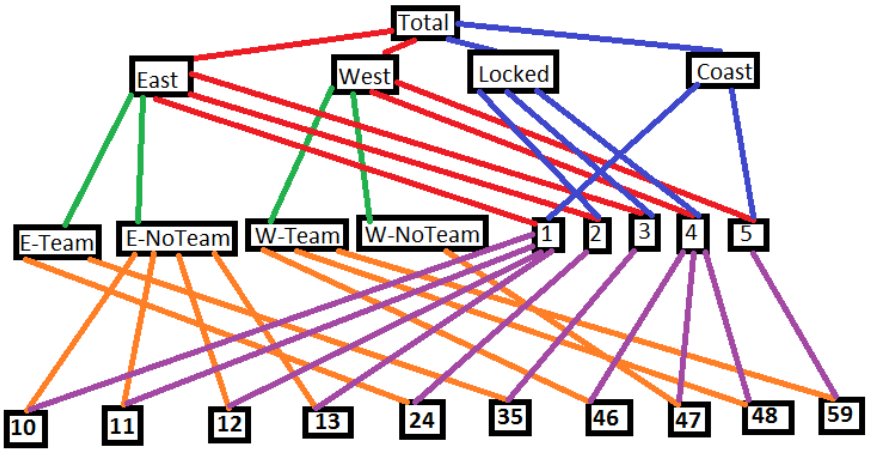
Outcell_ex7

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province
1	1	0	0	10	6623	-3306.8	V	C	6623	10
2	2	0	0	10	11528	-3791.6	V	C	11528	11
3	3	0	0	10	13916	-4905.2	V	C	13916	12
4	4	0	0	10	1569	-155.4	V	C	1569	13
5	5	0	0	10	4507	-1607.9	V	C	4507	24
6	6	0	0	10	9450	-4580	V	C	9450	35
7	7	0	0	10	3358	86.6	S	C	3358	46
8	8	0	0	10	4413	-883.2	V	C	4413	47
9	9	0	0	10	4185	-583.5	V	C	4185	48
10	10	0	0	10	12351	-3814.3	V	C	12351	59
11	11	0	0	100	71900	-62363.3	V	C	71900	TOT_REGION
12	12	0	0	60	47593	-38582.2	V	C	47593	EAST
13	13	0	0	40	24307	-15770.3	V	C	24307	WEST
14	14	0	0	50	45987	-36450.3	V	C	45987	COAST
15	15	0	0	50	25913	-20233.5	V	C	25913	LOCKED
16	16	0	0	40	33636	-24625.2	V	C	33636	1
17	17	0	0	10	4507	-1607.9	V	C	4507	2
18	18	0	0	10	9450	-4580	V	C	9450	3
19	19	0	0	30	11956	-6276.5	V	C	11956	4
20	20	0	0	10	12351	-3814.3	V	C	12351	5

Example 8: Summary example

This example combines the hierarchy concepts that previous examples already introduced. Suppose that, in addition to the request of our external client, the National Hockey League (NHL) has expressed interest in our project and has asked that at the direction level (EAST + WEST) we provide separate results for provinces with a NHL team separately from provinces that do not have an NHL team.

Each direction, EAST and WEST, is subdivided according to whether the provinces of each direction include NHL teams (Quebec, Ontario, Manitoba, Alberta, and British Columbia have NHL hockey teams). We therefore include four new subtotals in our hierarchy: E_TEAM, E_NOTEAM, W_TEAM, and W_NOTEAM. Now, the hierarchy for the province dimension will look like this:



If we update the hierarchy for our province dimension and add the industry dimension (with categories A and B), we obtain the following code and results (the resulting *Outcell* file is only displayed in part):

```
%let hierarchy ex8="
    TOTAL EAST WEST:
    TOTAL LOCKED COAST:
    EAST 1 2 3:
    EAST E_TEAM E_NOTEAM:
    WEST 4 5:
    WEST W_TEAM W_NOTEAM:
    LOCKED 2 3 4:
    COAST 1 5:
    E_TEAM 24 35:
    E_NOTEAM 10 11 12 13:
    W_TEAM 46 48 59:
    W_NOTEAM 47:
    1 10 11 12 13:
    2 24:
    3 35:
    4 46 47 48:
    5 59;
    IND_TOT A B;";

proc sensitivity
    data = data_hierarchy
    outcell = outcell_ex8
    outconstraint = outconstraint_ex8
    srule = "pq 0.2"
    hierarchy = &hierarchy_ex8
    ;
    id entid;
    var value;
    dimension province industry;
run;
```

Outcell_ex8

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province	Industry
1	1	0	0	5	3449	-298.6	V	C	3449	10	A
2	2	0	0	5	3174	-103.6	V	C	3174	10	B
3	3	0	0	5	4824	299.8	S	C	4824	11	A
4	4	0	0	5	6704	-350.2	V	C	6704	11	B
5	5	0	0	5	3352	486.6	S	C	3352	12	A
6	6	0	0	5	10564	-1118.4	V	C	10564	12	B
7	7	0	0	5	1432	87.2	S	C	1432	13	A
8	8	0	0	5	137	7.2	S	C	137	13	B
9	9	0	0	5	1552	272.2	S	C	1552	24	A
10	10	0	0	5	2955	-66.2	V	C	2955	24	B
11	11	0	0	5	2357	-536	V	C	2357	35	A
12	12	0	0	5	7093	-1981	V	C	7093	35	B
13	13	0	0	5	1257	229.4	S	C	1257	46	A
14	14	0	0	5	2101	383.2	S	C	2101	46	B
15	15	0	0	5	799	114.6	S	C	799	47	A
16	16	0	0	5	3614	60.6	S	C	3614	47	B
17	17	0	0	5	1021	71	S	C	1021	48	A
18	18	0	0	5	3164	349	S	C	3164	48	B
19	19	0	0	5	5845	-129.4	V	C	5845	59	A
20	20	0	0	5	6506	152.4	S	C	6506	59	B
21	21	0	0	100	71900	-61891.6	V	C	71900	TOTAL	IND_TOT
22	22	0	0	60	47593	-38147.4	V	C	47593	EAST	IND_TOT
23	23	0	0	40	24307	-15298.6	V	C	24307	WEST	IND_TOT
24	24	0	0	50	25913	-19957	V	C	25913	LOCKED	IND_TOT
25	25	0	0	50	45987	-35978.6	V	C	45987	COAST	IND_TOT
26	26	0	0	50	25888	-18185.2	V	C	25888	TOTAL	A
27	27	0	0	50	46012	-36003.6	V	C	46012	TOTAL	B
28	28	0	0	30	16966	-9883.2	V	C	16966	EAST	A
29	29	0	0	20	8922	-3206.4	V	C	8922	WEST	A
30	30	0	0	25	6986	-4089.8	V	C	6986	LOCKED	A
31	31	0	0	25	18902	-11199.2	V	C	18902	COAST	A
32	32	0	0	30	30627	-21181.4	V	C	30627	EAST	B
33	33	0	0	20	15385	-6959.6	V	C	15385	WEST	B
34	34	0	0	25	18927	-12971	V	C	18927	LOCKED	B
35	35	0	0	25	27085	-17076.6	V	C	27085	COAST	B
36	36	0	0	40	33636	-24190.4	V	C	33636	1	IND_TOT
37	37	0	0	10	4507	-1468.8	V	C	4507	2	IND_TOT
38	38	0	0	10	9450	-4338	V	C	9450	3	IND_TOT
39	39	0	0	20	13957	-8845	V	C	13957	E_TEAM	IND_TOT
40	40	0	0	40	33636	-24190.4	V	C	33636	E_NOTEAM	IND_TOT
41	41	0	0	20	13057	-5974.2	V	C	13057	1	A

2.4 Aggregates

In the series of examples of 2.2 Sensitivity rules, we saw that sensitive aggregates appear in *Outcell* output files. We will now show what these aggregates are and how to identify them.

An aggregate is the union of two or more cells sharing common values for all but one of the classification variables. In other words, an aggregate is a set of cells that appear in the same row or column of a table. In the table below, the cells highlighted in red are an example of an aggregate (of course, there are other aggregates possible in this table!):

	Industry		Total
Region	1	2	
A	35	6	41
B	3	3	6
C	5	9	14
Total	43	18	61

When we run PROC SENSITIVITY, G-Confid looks at the different possible combinations of aggregates and predicts in the *Outcell* file whether there are any sensitive aggregates. We can imagine different scenarios where sensitive aggregates may be found. We will look at three simple examples to show situations in which this may occur. Each example is based on a different data file.

Example 1

Our first example shows a case where enterprise S2 has contributed to two cells (A and B) on the same line of our table, and one of these two cells is sensitive. The first step is to create the data file:

```
data Data_Aggregate_ex1;
Entid='S1';   Region="A";   Value=100;   output;
Entid='S2';   Region="A";   Value=50;    output;
Entid='S3';   Region="A";   Value=4;     output;
Entid='S2';   Region="B";   Value=20;    output;
Entid='S4';   Region="B";   Value=5;     output;
Entid='S5';   Region="B";   Value=4;     output;
Entid='S6';   Region="C";   Value=20;    output;
Entid='S7';   Region="C";   Value=5;     output;
Entid='S8';   Region="C";   Value=4;     output;
run;
```

Data_aggregate_ex1

	Entid	Region	Value
1	S1	A	100
2	S2	A	50
3	S3	A	4
4	S2	B	20
5	S4	B	5
6	S5	B	4
7	S6	C	20
8	S7	C	5
9	S8	C	4

We see that S2 is the second largest contributor in region A and the largest contributor in region B. The other contributors are all different.

We will now show what happens when we run PROC SENSITIVITY with a p-percent rule where *p*=0.15. The next page shows the *Outcell* and *Outconstraint* output files since they are both important to fully understand the situation.

```
proc sensitivity
   data=Data_Aggregate_ex1
   outcell=outcell_ex1
   outconstraint=outconstraint_ex1
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Region;
run;
```

Outcell_ex1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	3	154	11	S	C	154	A
2	2	0	0	3	29	-1	V	C	29	B
3	3	0	0	3	29	-1	V	C	29	C
4	4	0	0	8	212	-27	V	C	212	TOT_REGION
5	5	0	0	5	183	2	S	A	183	

Outconstraint_ex1

	ConstraintId	CellId	Coefficient
1	1	1	1
2	1	2	1
3	1	3	1
4	1	4	-1
5	2	1	1
6	2	2	1
7	2	5	-1

In the *Outcell* file, there is a cell with *Type*="A". This is the sensitive aggregate. If we look at the *Outcell* file only, we cannot tell which cells make up the sensitive aggregate. We need to look at the *Outconstraint* file to get the answer. This latter file indicates the relationships between cells.

In *Outconstraint*, every *ConstraintId* represents a linear constraint and the *CellId* variable is a number assigned to each of the cells, including the aggregates. The numbering of the *CellId* variable in *Outconstraint* matches that of the *CellId* variable in the *Outcell* file. For each constraint, the sum of the cells with a coefficient of 1 must equal the cell with a coefficient of -1. So, we have two linear constraints described below:

1. The sum of cells 1, 2 and 3 is equal to cell 4. If we look at the *CellId* values in the *Outcell* file, we see that *CellId*=1, 2 and 3 correspond respectively to the values of regions A, B and C, while *CellId*=4 is the cell corresponding to the total (named "TOT_REGION"). This linear constraint states that if we add regions A, B and C, we should obtain our total.
2. The second linear constraint indicates that the sum of cells 1 and 2 is equal to cell 5. In other words, our aggregate (cell 5) consists of the sum of cells A and B.

We can then determine what the aggregates correspond to by examining both the *Outcell* and *Outconstraint* files. There is, however, a faster way to determine this: using the %AGGREGATE macro. Based on the following code, the %AGGREGATE macro produces a summary table of aggregates:

```
%Aggregate(
   Dimension=Region,
   Constraint=outconstraint_ex1,
   Incell=outcell_ex1,
   OutFile=Out_Aggregate_ex1
);
```

Out_aggregate_ex1

	Region	ConstraintId	CellId	Type	total	Sensitivity
1		2	5	A	183	2
2	B	2	2	C	29	-1
3	A	2	1	C	154	11

In order for the macro to work properly, we must enter the dimensions as specified in PROC SENSITIVITY, indicate the names of the *Outconstraint* and *Outcell* files produced by PROC SENSITIVITY, and indicate the name we want to assign to the output file. Here, we are naming the output file *Out_aggregate_ex1*. We quickly see in this file that our sensitive aggregate is indeed made up of cells A and B.

If S1 is the target and S2 is the intruder, we can verify the calculation of the aggregate sensitivity by:

$$S_{AB} = 0.15 \cdot 100 - 5 - 4 - 4 = 2$$

Example 2

Our second example deals with two sensitive cells (A and B), each of which has only one respondent. This is a typical example where sensitive cells cannot protect each other.

```
data Data_Aggregate_ex2;
Entid='S1';    Region="A";    Value=80;     output;
Entid='S2';    Region="B";    Value=40;     output;
Entid='S6';    Region="C";    Value=20;     output;
Entid='S7';    Region="C";    Value=5;      output;
Entid='S8';    Region="C";    Value=4;      output;
run;
```

Data_aggregate_ex2

	Entid	Region	Value
1	S1	A	80
2	S2	B	40
3	S6	C	20
4	S7	C	5
5	S8	C	4

Once again applying a p-percent rule where *p*=0.15, we obtain the following results:

```
proc sensitivity
   data=Data_Aggregate_ex2
   outcell=outcell_ex2
   outconstraint=outconstraint_ex2
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Region;
run;
```

Outcell_ex2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	1	80	12	S	C	80	A
2	2	0	0	1	40	6	S	C	40	B
3	3	0	0	3	29	-1	V	C	29	C
4	4	0	0	5	149	-17	V	C	149	TOT_REGION
5	5	0	0	2	120	12	S	A	120	

Outconstraint_ex2

	ConstraintId	CellId	Coefficient
1	1	1	1
2	1	2	1
3	1	3	1
4	1	4	-1
5	2	1	1
6	2	2	1
7	2	5	-1

We see in the *Outcell* file that cells A and B are sensitive and there is also a sensitive aggregate. According to the *Outconstraint* file, the sensitive aggregate consists of cells A and B. The %AGGREGATE macro also confirms this:

```
%Aggregate(
   Dimension=Region,
   Constraint=outconstraint_ex2,
   Incell=outcell_ex2,
   OutFile=Out_Aggregate_ex2
);
```

Out_aggregate_ex2

	Region	ConstraintId	CellId	Type	total	Sensitivity
1		2	5	A	120	12
2	B	2	2	C	40	6
3	A	2	1	C	80	12

As mentioned above, cells A and B cannot mutually protect each other. In other words, suppressing cells A and B is not enough to ensure confidentiality. If we know the total of (A+B), S1 can guess the revenue of enterprise S2 and vice versa. That is why we need to protect not only A and B, but also (A+B).

If S2 is the intruder, the sensitivity calculation is as follows for aggregate (A+B):

$$S_{AB} = 0.15 \cdot 80 = 12$$

Example 3

This final example shows a situation where the largest respondent for cell B is the second largest respondent for the aggregate (A+B), while cell A is sensitive.

```
data Data_Aggregate_ex3;
Entid='S1';   Region="A";   Value=100;   output;
Entid='S2';   Region="A";   Value=3;     output;
Entid='S3';   Region="A";   Value=1;     output;
Entid='S4';   Region="B";   Value=20;    output;
Entid='S5';   Region="B";   Value=5;     output;
Entid='S6';   Region="B";   Value=4;     output;
Entid='S7';   Region="C";   Value=20;    output;
Entid='S8';   Region="C";   Value=12;    output;
Entid='S9';   Region="C";   Value=4;     output;
run;
```

Data_aggregate_ex3

	Entid	Region	Value
1	S1	A	100
2	S2	A	3
3	S3	A	1
4	S4	B	20
5	S5	B	5
6	S6	B	4
7	S7	C	20
8	S8	C	12
9	S9	C	4

```
proc sensitivity
   data=Data_Aggregate_ex3
   outcell=outcell_ex3
   outconstraint=outconstraint_ex3
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C;"
   ;
   id Entid;
   var Value;
   dimension Region;
run;
```

Outcell_ex3

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	3	104	14	S	C	104	A
2	2	0	0	3	29	-1	V	C	29	B
3	3	0	0	3	36	-1	V	C	36	C
4	4	0	0	9	169	-34	V	C	169	TOT_REGION
5	5	0	0	6	133	2	S	A	133	

Outconstraint_ex3

	ConstraintId	CellId	Coefficient
1	1	1	1
2	1	2	1
3	1	3	1
4	1	4	-1
5	2	1	1
6	2	2	1
7	2	5	-1

```
%Aggregate(
   Dimension=Region,
   Constraint=outconstraint_ex3,
   Incell=outcell_ex3,
   OutFile=Out_Aggregate_ex3
);
```

Out_aggregate_ex3

	Region	ConstraintId	CellId	Type	total	Sensitivity
1		2	5	A	133	2
2	B	2	2	C	29	-1
3	A	2	1	C	104	14

In this example, enterprise S1 in cell A is at risk of being disclosed. S1 is also at risk of being disclosed if we know the total of (A+B). Therefore, we need to protect not only A, but also (A+B).

If S4 is the intruder, the sensitivity calculation is as follows for our aggregate:

$$S_{AB} = 0.15 \cdot 100 - 5 - 4 - 3 - 1 = 2$$

Supplementary note

This concludes our basic examples of aggregates. As we have seen, the %AGGREGATE macro is not essential to determine which are the aggregates, since this information can be determined manually. However, it does not require much code, and can be practical to use for more complex tables. Do not hesitate to use it!

2.5 Waivers

When an enterprise contributes significantly to the total of a cell that we want to publish, then the cell cannot be published without compromising the confidentiality of the data for this enterprise. To get around this problem, we can request a waiver from the enterprise, i.e., the enterprise agrees that all or part of the data it produces for this survey are no longer considered confidential. The purpose of such an exercise is to be able to publish the cell(s) of interest in the results table. G-Confid protects the confidentiality of all the other enterprises contributing to the cell.

Creation of a fictitious data file

The fictitious file on which our examples will be based can be created using the following code:

```
data data_waivers;
   length Entid$3 Industry $1 Region $1;
   input Entid$ Industry$ Region$ value
         wafl_1 wafl_2 wafl_3 wafl_4;
   cards;
111  A  1  720  1  1  0  1
2    A  1  700  0  1  1  1
3    A  1  40   0  0  0  0
4    A  1  20   0  0  0  0
111  A  2  290  1  1  0  0
111  A  2  200  1  1  0  0
7    A  2  25   0  0  0  0
8    A  2  15   0  0  0  0
9    A  2  10   0  0  0  0
10   A  3  60   0  0  0  0
11   A  3  50   0  0  0  0
12   A  3  40   0  0  0  0
111  B  1  450  1  1  0  1
111  B  1  270  1  1  0  1
15   B  1  60   0  0  0  0
16   B  1  40   0  0  0  0
17   B  1  30   0  0  0  0
18   B  2  6    0  0  0  0
19   B  2  4    0  0  0  0
20   B  2  3    0  0  0  0
21   B  2  2    0  0  0  0
22   B  3  90   0  0  0  0
22   B  3  80   0  0  0  0
24   B  3  75   0  0  0  0
25   B  3  70   0  0  0  0
run;
```

Data_waivers

	Entid	Industry	Region	value	wafl_1	wafl_2	wafl_3	wafl_4
1	111	A	1	720	1	1	0	1
2	2	A	1	700	0	1	1	1
3	3	A	1	40	0	0	0	0
4	4	A	1	20	0	0	0	0
5	111	A	2	290	1	1	0	0
6	111	A	2	200	1	1	0	0
7	7	A	2	25	0	0	0	0
8	8	A	2	15	0	0	0	0
9	9	A	2	10	0	0	0	0
10	10	A	3	60	0	0	0	0
11	11	A	3	50	0	0	0	0
12	12	A	3	40	0	0	0	0
13	111	B	1	450	1	1	0	1
14	111	B	1	270	1	1	0	1
15	15	B	1	60	0	0	0	0
16	16	B	1	40	0	0	0	0
17	17	B	1	30	0	0	0	0
18	18	B	2	6	0	0	0	0
19	19	B	2	4	0	0	0	0
20	20	B	2	3	0	0	0	0
21	21	B	2	2	0	0	0	0
22	22	B	3	90	0	0	0	0
23	22	B	3	80	0	0	0	0
24	24	B	3	75	0	0	0	0
25	25	B	3	70	0	0	0	0

Example 0 – Results without a waiver

To begin, let's calculate the sensitivity with the assumption that no waivers will be obtained for our data file. In the next few examples, we will look at different scenarios where there are waivers, while keeping the same parameters in PROC SENSITIVITY, so that we can make an effective comparison. If there are no waivers, we obtain the following results:

```
proc sensitivity
   data=data_waivers
   outcell=outcell_ex0
   outconstraint=outconstraint_ex0
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY A B;
              TOT_REGION 1 2 3;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
run;
```

Outcell_ex0

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	4	1480	12	S	C	1480	A	1
2	2	0	0	4	540	24	S	C	540	A	2
3	3	0	0	3	150	-34	V	C	150	A	3
4	4	0	0	4	850	2	S	C	850	B	1
5	5	0	0	4	15	-4.4	V	C	15	B	2
6	6	0	0	3	315	-53	V	C	315	B	3
7	7	0	0	20	3350	-527	V	C	3350	TOT_INDUSTRY	TOT_REGION
8	8	0	0	10	2170	-139	V	C	2170	A	TOT_REGION
9	9	0	0	11	1180	-218	V	C	1180	B	TOT_REGION
10	10	0	0	7	2330	-46	V	C	2330	TOT_INDUSTRY	1
11	11	0	0	8	555	9	S	C	555	TOT_INDUSTRY	2
12	12	0	0	6	465	-203	V	C	465	TOT_INDUSTRY	3
13	13	0	0	7	2020	11	S	A	2020		

We see that there are several sensitive cells due to enterprise 111, which is a significant contributor in some domains.

Example 1 – Waiver from one enterprise

Since enterprise 111 is a significant contributor to our problem cells, we will look at what happens when we obtain a waiver from this enterprise. This scenario is represented by the variable *wafl_1*, where there is a '1' for the contributions from enterprise 111 and a '0' otherwise. The following code is indicated in PROC SENSITIVITY:

```
proc sensitivity
   data=data_waivers
   outcell=outcell_ex1
   outconstraint=outconstraint_ex1
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY A B; TOT_REGION 1 2 3;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
   waiver wafl_1;
run;
```

In the *Outcell* file, we see that with the waiver from enterprise 111, there is only one sensitive cell left (*Industry*= "A", *Region*= "1").

Outcell_ex1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	SensitivityBeforeWaivers	FavCost	Industry	Region
1	1	0	0	4	1480	10	S	C	1480	12	1480	A	1
2	2	0	0	4	540	-22.5	V	C	540	24	2170	A	2
3	3	0	0	3	150	-34	V	C	150	-34	150	A	3
4	4	0	0	4	850	-64	V	C	850	2	2330	B	1
5	5	0	0	4	15	-4.4	V	C	15	-4.4	15	B	2
6	6	0	0	3	315	-53	V	C	315	-53	315	B	3
7	7	0	0	20	3350	-650	V	C	3350	-527	3350	TOT_INDUSTRY	TOT_REGION
8	8	0	0	10	2170	-190	V	C	2170	-139	2170	A	TOT_REGION
9	9	0	0	11	1180	-273	V	C	1180	-218	1180	B	TOT_REGION
10	10	0	0	7	2330	-120	V	C	2330	-46	2330	TOT_INDUSTRY	1
11	11	0	0	8	555	-37.5	V	C	555	9	555	TOT_INDUSTRY	2
12	12	0	0	6	465	-203	V	C	465	-203	465	TOT_INDUSTRY	3

Two new variables appear in the *Outcell* file: *SensitivityBeforeWaivers* and *FavCost*. As its name indicates, the *SensitivityBeforeWaivers* variable gives us results for sensitivity calculations without accounting for waivers. This corresponds to the sensitivity calculation results found in example 0. On the other hand, *FavCost* is an experimental variable for the %SUPPRESS macro, but it is still in development, so its use is not recommended at the moment.

We will now try to better understand the impact of the waiver on the sensitivity in cell "*Industry*=A, *Region*=1". Without the waiver, we end up with the following roles, which lead to the sensitivity calculation shown below:

Entid	Value	Role
111	720	Target
2	700	Intruder / Suspect
3	40	Noise
4	20	Noise

$$S = 0.1 \times 720 - 40 - 20 = 12$$

Once we obtain a waiver from enterprise 111, its information is no longer considered to be confidential and therefore no longer needs to be protected. When the cell total is published, enterprise 2 becomes the one with the highest risk of disclosure. That is why it now becomes the target in the sensitivity calculation. In this case, enterprise 111 becomes the intruder, since it is now the one that can most easily estimate the revenue of enterprise 2, by subtracting its own value from the cell total. Taking into account these role changes, sensitivity drops to a value of 10.

Entid	Value	Role
111	720	Intruder / Suspect
2	700	Target
3	40	Noise
4	20	Noise

$$S = 0.1 \times 700 - 40 - 20 = 10$$

Example 2 – Waiver from two enterprises

In the previous example, we saw that all of the cells in the table that we wanted to publish were protected except for one: the cell for industry A in region 1. In addition to enterprise 111, this cell also includes enterprise 2, which contributed significantly to the cell total. Obtaining a waiver from both enterprise 111 and enterprise 2 could enable us to publish the cell. This is what is presented in the scenario for variable *wafl_2*.

```
proc sensitivity
   data=data_waivers
   outcell=outcell_ex2
   outconstraint=outconstraint_ex2
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY A B; TOT_REGION 1 2 3;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
   waiver wafl_2;
run;
```

With waivers from the two enterprises, we see that the problem cell is no longer sensitive. The entire table is now publishable.

Outcell_ex2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	SensitivityBeforeWaivers	FavCost	Industry	Region
1	1	0	0	4	1480	-716	V	C	1480	12	2330	A	1
2	2	0	0	4	540	-22.5	V	C	540	24	2170	A	2
3	3	0	0	3	150	-34	V	C	150	-34	150	A	3
4	4	0	0	4	850	-64	V	C	850	2	2330	B	1
5	5	0	0	4	15	-4.4	V	C	15	-4.4	15	B	2
6	6	0	0	3	315	-53	V	C	315	-53	315	B	3
7	7	0	0	20	3350	-1233	V	C	3350	-527	3350	TOT_INDUSTRY	TOT_REGION
8	8	0	0	10	2170	-894	V	C	2170	-139	2170	A	TOT_REGION
9	9	0	0	11	1180	-273	V	C	1180	-218	1180	B	TOT_REGION
10	10	0	0	7	2330	-824	V	C	2330	-46	2330	TOT_INDUSTRY	1
11	11	0	0	8	555	-37.5	V	C	555	9	555	TOT_INDUSTRY	2
12	12	0	0	6	465	-203	V	C	465	-203	465	TOT_INDUSTRY	3

We will now show what happens in terms of calculating the sensitivity for cell "*Industry*=A, *Region*=1". Since the two largest enterprises (111 and 2) no longer need to be protected, enterprise 3 is now the most at risk of being disclosed and becomes the target. Enterprise 111 is now the intruder since it can most easily estimate the revenue of enterprise 3 by subtracting its own contribution from the cell total. In this situation, enterprise 2 produces a lot of noise (protection) for enterprise 3, which means that the sensitivity is now negative. Using the two waivers therefore enables us to publish the cell total.

Entid	Value	Role
111	720	Intruder / Suspect
2	700	Noise
3	40	Target
4	20	Noise

$$S = 0.1 \times 40 - 700 - 2 = -716$$

Example 3 – Waiver with no impact

Up to now, we have looked at situations where the enterprises contributing the most to a cell have produced a waiver. Now let's imagine a scenario where a waiver is obtained from enterprise 2, but not from enterprise 111. This scenario is presented in the variable *wafl_3*.

```
proc sensitivity
   data=data_waivers
   outcell=outcell_ex3
   outconstraint=outconstraint_ex3
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY A B; TOT_REGION 1 2 3;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
   waiver wafl_3;
run;
```

In this example, we can see that the waiver from enterprise 2 does not change the sensitivity calculations in comparison to the scenario where there is no waiver. The reason is simple: in such a situation, enterprise 111 remains the target and enterprise 2 remains the intruder. Obtaining a waiver from enterprise 2 does not change the roles of the enterprises, so the sensitivity calculation does not change.

Outcell_ex3

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	SensitivityBeforeWaivers	FavCost	Industry	Region
1	1	0	0	4	1480	12	S	C	1480	12	1480	A	1
2	2	0	0	4	540	24	S	C	540	24	540	A	2
3	3	0	0	3	150	-34	V	C	150	-34	150	A	3
4	4	0	0	4	850	2	S	C	850	2	850	B	1
5	5	0	0	4	15	-4.4	V	C	15	-4.4	15	B	2
6	6	0	0	3	315	-53	V	C	315	-53	315	B	3
7	7	0	0	20	3350	-527	V	C	3350	-527	3350	TOT_INDUSTRY	TOT_REGION
8	8	0	0	10	2170	-139	V	C	2170	-139	2170	A	TOT_REGION
9	9	0	0	11	1180	-218	V	C	1180	-218	1180	B	TOT_REGION
10	10	0	0	7	2330	-46	V	C	2330	-46	2330	TOT_INDUSTRY	1
11	11	0	0	8	555	9	S	C	555	9	555	TOT_INDUSTRY	2
12	12	0	0	6	465	-203	V	C	465	-203	465	TOT_INDUSTRY	3
13	13	0	0	7	2020	11	S	A	2020	11	2020		

Example 4 – Partial waiver

In all of the examples so far, the enterprises were providing a full waiver, i.e., the enterprise agreed to have its data published for all domains being studied. In practice, an enterprise may only agree to publish a portion of its data. Enterprise 111 might decide, for example, to keep its data confidential in industry A for region 2. The variable *wafl_4* presents the scenario where enterprise 111 decides to produce this partial waiver and enterprise 2 waives confidentiality for the only contribution it has made in the dataset. Due to the partial waiver from enterprise 111, we must use the PWAIVER statement rather than the WAIVER statement in PROC SENSITIVITY:

```
proc sensitivity
   data=data_waivers
   outcell=outcell_ex4
   outconstraint=outconstraint_ex4
   srule="pq 0.1"
   hierarchy="TOT_INDUSTRY A B; TOT_REGION 1 2 3;"
   ;
   id Entid;
   var Value;
   dimension Industry Region;
   pwaiver wafl_4;
run;
```

Outcell_ex4

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	SensitivityBeforeWaivers	FavCost	Industry	Region
1	1	0	0	4	1480	-716	V	C	1480	12	2330	A	1
2	2	0	0	4	540	24	S	C	540	24	540	A	2
3	3	0	0	3	150	-34	V	C	150	-34	150	A	3
4	4	0	0	4	850	-64	V	C	850	2	2330	B	1
5	5	0	0	4	15	-4.4	V	C	15	-4.4	15	B	2
6	6	0	0	3	315	-53	V	C	315	-53	315	B	3
7	7	0	0	20	3350	-527	V	C	3350	-527	3350	TOT_INDUSTRY	TOT_REGION
8	8	0	0	10	2170	-139	V	C	2170	-139	2170	A	TOT_REGION
9	9	0	0	11	1180	-273	V	C	1180	-218	1180	B	TOT_REGION
10	10	0	0	7	2330	-824	V	C	2330	-46	2330	TOT_INDUSTRY	1
11	11	0	0	8	555	9	S	C	555	9	555	TOT_INDUSTRY	2
12	12	0	0	6	465	-203	V	C	465	-203	465	TOT_INDUSTRY	3
13	13	0	0	7	2020	11	S	A	2020	11	2020		

As expected, the sensitivity of cell "*Industry*=A, *Region*=2" has not changed, while the domains for which a waiver has been received have decreased sensitivities.

Note:

- For partial waivers:
  - It is essential to use the PWAIVER statement.
  - The WAIVER statement causes an error message if an enterprise has partial waivers, so it does not work in this situation.
- For full waivers:
  - The PWAIVER and WAIVER statements work properly, as long as they are not specified in PROC SENSITIVITY at the same time.
  - The WAIVER statement should be more efficient than the PWAIVER statement in terms of running time.
  - Furthermore, the WAIVER statement can be used to check if any errors were made in the input data, once we know that only full waivers are being applied.

Supplementary note: NK rule with waivers

For more advanced users, below is a supplementary note that might prove helpful.

Applying an NK rule is less evident with the presence of waivers. G-Confid uses a p-percent rule. To do this, the following calculations are performed:

1.	Sensitivity without taking waivers into account	$S_0 = \frac{100-k}{k}\sum_{i=1}^{n} x_i - \sum_{i>n} x_i$
2.	Relative protection provided to largest contributor	$P = \frac{1}{x_1}\left(S_0 + \sum_{i\geq 3} x_i\right)$
3.	Sensitivity	$S = Px_t - \sum_{i\neq 1,t} x_i$

where x_i are the contributor values, *n* and *k* are parameters of the NK rule, and x_t is the value of the target contributor.

2.6 Survey weights

As of G-Confid version 1.07, survey weights can be included in sensitivity calculations. Accounting for weights may prove important in determining whether or not a cell total is sensitive. In practice, the first thing we notice when we account for weights in sensitivity calculations is that the cell total[3] for which we wish to calculate sensitivity is now weighted:

$$Total = \sum_i w_i x_i$$

where w_i are the weights and x_i are the values of each contributor i.

This has the following implications:

- **A contributor with a large weight needs less protection than a contributor with a weight of 1 since we assume that contributors are not aware of the survey weights.** For example, if an enterprise has a revenue of $250 with a weight of 3, its contribution to the cell total is $750. It is then difficult for an intruder to obtain a good estimate of the true revenue of $250, because if the intruder subtracts its own value from the cell total, the best guess is now $750 rather than $250. Therefore, a good sensitivity rule should take into account not only the protection offered by noise, but also the protection offered by weights.

- **There are different approaches for taking weights into account in the sensitivity calculation; here, they are referred to as "protection levels".** Before the introduction of G-Confid 1.07, the guideline was to consider that a contribution with a weight of 3 or more did not require protection. Based on this guideline, two approaches were created in G-Confid to calculate sensitivity with consideration for weights: LINEAR and STEP.[4] Using these approaches, we can now take into account the weights of all contributors in the sensitivity calculation, and also the protection offered by a weight between 1 and 3. More details on the LINEAR and STEP protection levels will be provided in the examples.

- **Both the *value* and *weight* of each contributor must be taken into account to determine its role (target, intruder, noise).** In general, the contributor with the highest value will be the target and the second largest contributor will be the intruder. However, with weights, the calculated sensitivity may be higher by choosing a target that is not the largest contributing enterprise. If we wish to cover the worst case scenario, we need to find the target/intruder pair with the highest sensitivity. G-Confid has an algorithm to identify this pair.

Bearing all of this in mind, let's now look at a few examples where weights are used in practice in G-Confid.

[3] Note: As of G-Confid 1.07.002 the cell total is called *TotalVar* in the *Outcell* and *Outpattern* files.

[4] The EXACT option is also available to take weights into account, but it is not based on the weight guideline of 3 or more. This option is not recommended (see Supplementary notes). It is preferable to use LINEAR or STEP.

Creating the fictitious data file

The fictitious file on which we will be basing our examples can be created using the following code:

```
data Data_weights;
   input ENTID$  VARIABLE$  DATAVALUE
         WEIGHT_A WEIGHT_B;
   cards;
S01 D1 200 1   1
S02 D1 180 1   1
S03 D1 30  1   1
S04 D1 5   1   1
S05 D2 200 1   3
S06 D2 180 1   1
S07 D2 30  1   1
S08 D2 5   1   1
S09 D3 200 1   2.9
S10 D3 180 1   1
S11 D3 30  1   1
S12 D3 5   1   1
S13 D4 200 1   2.5
S14 D4 180 1   1
S15 D4 30  1   1
S16 D4 5   1   1
S17 D5 200 1   2.5
S18 D5 180 1   1.5
S19 D5 30  1   1
S20 D5 5   1   1
S21 D6 200 1   2.5
S22 D6 180 1   1.5
S23 D6 30  1   2
S24 D6 5   1   1
S25 D7 200 1   1
S26 D7 180 1   1.5
S27 D7 30  1   2
S28 D7 5   1   1
run;
```

Data_weights

	ENTID	VARIABLE	DATAVALUE	WEIGHT_A	WEIGHT_B
1	S01	D1	200	1	1
2	S02	D1	180	1	1
3	S03	D1	30	1	1
4	S04	D1	5	1	1
5	S05	D2	200	1	3
6	S06	D2	180	1	1
7	S07	D2	30	1	1
8	S08	D2	5	1	1
9	S09	D3	200	1	2.9
10	S10	D3	180	1	1
11	S11	D3	30	1	1
12	S12	D3	5	1	1
13	S13	D4	200	1	2.5
14	S14	D4	180	1	1
15	S15	D4	30	1	1
16	S16	D4	5	1	1
17	S17	D5	200	1	2.5
18	S18	D5	180	1	1.5
19	S19	D5	30	1	1
20	S20	D5	5	1	1
21	S21	D6	200	1	2.5
22	S22	D6	180	1	1.5
23	S23	D6	30	1	2
24	S24	D6	5	1	1
25	S25	D7	200	1	1
26	S26	D7	180	1	1.5
27	S27	D7	30	1	2
28	S28	D7	5	1	1

Note that the file only contains seven domains, named D1 to D7, each containing 4 enterprises with the same values (200, 180, 30 and 5). These seven domains were created in order to compare different scenarios with varying survey weights.

Example 0 – No weight

To begin, let's try to perform sensitivity calculations with the assumption that no weights were obtained for our data file. In the next few examples, we will try different scenarios with weights, keeping the same parameters in PROC SENSITIVITY so that we can make reasonable comparisons. With no weights, we obtain the following results:

```
proc sensitivity
   data=Data_weights
   outcell=outcell_ex0
   outconstraint=outconstraint_ex0
   srule="pq 0.2"
   hierarchy="TOT D1 D2 D3
              D4 D5 D6 D7;"
   ;
   id entid;
   var datavalue;
   dimension variable;
run;
```

Outcell_ex0

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	VARIABLE
1	1	0	0	4	415	5	S	C	415	D1
2	2	0	0	4	415	5	S	C	415	D2
3	3	0	0	4	415	5	S	C	415	D3
4	4	0	0	4	415	5	S	C	415	D4
5	5	0	0	4	415	5	S	C	415	D5
6	6	0	0	4	415	5	S	C	415	D6
7	7	0	0	4	415	5	S	C	415	D7
8	8	0	0	28	2905	-2465	V	C	2905	TOT

Note that the same results are obtained in domains D1 to D7 since all of our domains have an identical profile. Here, the sensitivity calculation is based on a p-percent rule where *p*=0.2. We see four contributors in each domain with the following roles:

Value	Role
200	Target
180	Intruder / Suspect
30	Noise
5	Noise

For each of our domains, we obtain $S = 0.2 \times 200 - 30 - 5 = 5$, which is considered to be sensitive.

Example 1 – Weight of 1

Now let's show what happens when we include a weight variable that provides a weight of '1' for each enterprise in our example. This corresponds to the variable *WEIGHT_A* in our data file.

```
proc sensitivity
   data=Data_weights
   outcell=outcell_ex1
   outconstraint=outconstraint_ex1
   srule="pq 0.2"
   hierarchy="TOT D1 D2 D3
              D4 D5 D6 D7;"
   weightprotlevel="LINEAR"
   ;
   id entid;
   var datavalue;
   dimension variable;
   weight WEIGHT_A;
run;
```

Outcell_ex1

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	VARIABLE
1	1	0	0	4	415	5	S	C	415	D1
2	2	0	0	4	415	5	S	C	415	D2
3	3	0	0	4	415	5	S	C	415	D3
4	4	0	0	4	415	5	S	C	415	D4
5	5	0	0	4	415	5	S	C	415	D5
6	6	0	0	4	415	5	S	C	415	D6
7	7	0	0	4	415	5	S	C	415	D7
8	8	0	0	28	2905	-2465	V	C	2905	TOT

As expected, we obtained the same results using a weight of 1 as we did with no weight specified. In WEIGHTPROTLEVEL, we specify the *protection level* that we want to use to account for the weight during the sensitivity calculation. Here we have selected LINEAR, which is also the default option.

If we had chosen STEP instead of LINEAR, we would have obtained the same results due to the weight of 1. The next few examples will present varied weights so that we can better understand the differences between these options.

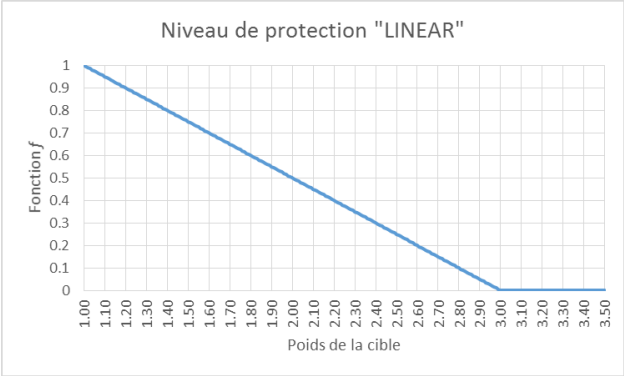
Example 2 – Varied weights with a LINEAR protection level

Let's begin by presenting the LINEAR protection level, which is, as previously stated, the default option. With this option, sensitivity is calculated as follows:

$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i \geq 3} w_i x_i$$

where w_i are the weights and x_i are the values of each contributor i, p is the value of p in the p-percent rule and f is defined as follows:

Weight of the target (w_1)	f function
[1 ; 3)	$\frac{3 - w_1}{2}$
> 3	0



It should be noted that if the target has a weight of 3, sensitivity should be zero or negative since the f function has a value of 0. The f function decreases linearly between weights 1 and 3, hence the name LINEAR to designate this level of protection. The greater the weight, the less protection is needed, until a weight of 3 is reached, at which point the target is no longer considered to require protection.

Now, let's look at the LINEAR protection level in practice. In the variable *WEIGHT_B*, we have different weights for each domain, which means that we obtain different sensitivity calculations from one domain to the next. We obtain the following results:

```
proc sensitivity
   data=Data_weights
   outcell=outcell_ex2
   outconstraint=outconstraint_ex2
   srule="pq 0.2"
   hierarchy="TOT D1 D2 D3
              D4 D5 D6 D7;"
   weightprotlevel="LINEAR"
   ;
   id entid;
   var datavalue;
   dimension variable;
   weight WEIGHT_B;
run;
```

Outcell_ex2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	VARIABLE
1	1	0	0	4	415	5	S	C	415	D1
2	2	0	0	4	415	-35	V	C	815	D2
3	3	0	0	4	415	-33	V	C	795	D3
4	4	0	0	4	415	-25	V	C	715	D4
5	5	0	0	4	415	-115	V	C	805	D5
6	6	0	0	4	415	-145	V	C	835	D6
7	7	0	0	4	415	-38	V	C	535	D7
8	8	0	0	28	2905	-4115	V	C	4915	TOT

We will now show how each of the domains obtained its sensitivity:

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D1</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S01</td><td>200</td><td>1</td></tr>
<tr><td>S02</td><td>180</td><td>1</td></tr>
<tr><td>S03</td><td>30</td><td>1</td></tr>
<tr><td>S04</td><td>5</td><td>1</td></tr>
</table>
</td><td>This is the simplest example, where all of the weights are 1. Let’s try applying the formula with a LINEAR level of protection. If we consider that S01 is the target and S02 is the intruder, we see that the value of $f = \frac{3-w_1}{2} = \frac{3-1}{2} = 1$. We obtain:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 40 - 0 - 30 - 5$$
$$= 5$$</td></tr>
<tr><td>D2</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S05</td><td>200</td><td>3</td></tr>
<tr><td>S06</td><td>180</td><td>1</td></tr>
<tr><td>S07</td><td>30</td><td>1</td></tr>
<tr><td>S08</td><td>5</td><td>1</td></tr>
</table>
</td><td>In the second domain, the weight of the target is 3. As we saw above, when the target has a weight of 3, function $f = 0$.
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 0 - 0 - 30 - 5$$
$$= -35$$</td></tr>
<tr><td>D3</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S09</td><td>200</td><td>2.9</td></tr>
<tr><td>S10</td><td>180</td><td>1</td></tr>
<tr><td>S11</td><td>30</td><td>1</td></tr>
<tr><td>S12</td><td>5</td><td>1</td></tr>
</table>
</td><td>If the target now has a weight of 2.9, $f = \frac{3-w_1}{2} = \frac{3-2.9}{2} = 0.05$. In this situation, we see that the sensitivity is slightly higher than what would have been obtained with a weight of 3:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.05 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 2 - 0 - 30 - 5$$
$$= -33$$</td></tr>
</table>

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D4</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S13</td><td>200</td><td>2.5</td></tr>
<tr><td>S14</td><td>180</td><td>1</td></tr>
<tr><td>S15</td><td>30</td><td>1</td></tr>
<tr><td>S16</td><td>5</td><td>1</td></tr>
</table>
</td><td>With a weight of 2.5, f is further increased: $f = \frac{3-w_1}{2} = \frac{3-2.5}{2} = 0.25$. The sensitivity is also increased:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.25 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 10 - 0 - 30 - 5$$
$$= -25$$</td></tr>
<tr><td>D5</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S17</td><td>200</td><td>2.5</td></tr>
<tr><td>S18</td><td>180</td><td>1.5</td></tr>
<tr><td>S19</td><td>30</td><td>1</td></tr>
<tr><td>S20</td><td>5</td><td>1</td></tr>
</table>
</td><td>In the fifth domain, we look at the impact of introducing a weight for the intruder. In this case, f remains the same as D4 since the weight of the target does not change. Sensitivity is a lot lower:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.25 \cdot 200 - (1.5-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 10 - 90 - 30 - 5$$
$$= -115$$
The formula reflects the fact that the intruder, in subtracting its value from the cell total to estimate x_1, does not account for its own weight w_2 (since it is assumed that the contributors are not aware of the survey weights). The further w_2 is from 1, the further the estimate by the intruder from the true value of x_1. The fact that $w_2 = 1.5$ in this case means that the target enterprise (S17) is protected.</td></tr>
<tr><td>D6</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S21</td><td>200</td><td>2.5</td></tr>
<tr><td>S22</td><td>180</td><td>1.5</td></tr>
<tr><td>S23</td><td>30</td><td>2</td></tr>
<tr><td>S24</td><td>5</td><td>1</td></tr>
</table>
</td><td>Here, we have introduced a weight for one of the enterprises contributing to the noise. Sensitivity is:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.25 \cdot 200 - (1.5-1) \cdot 180 - 2 \cdot 30 - 1 \cdot 5$$
$$= 10 - 90 - 60 - 5$$
$$= -145$$
As expected, a higher weight for an enterprise contributing to the noise produces even higher protection since it becomes difficult for the intruder to guess the true value of x_1.</td></tr>
</table>

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D7</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Weight</th></tr>
<tr><td>S25</td><td>200</td><td>1</td></tr>
<tr><td>S26</td><td>180</td><td>1.5</td></tr>
<tr><td>S27</td><td>30</td><td>2</td></tr>
<tr><td>S28</td><td>5</td><td>1</td></tr>
</table>
</td><td>To conclude, we will show that there are situations where the enterprise with the largest value is no longer the target due to weights. Domain D7 is like D6, but with a weight of 1 for the enterprise with the largest value. If we assume that S25 is the target enterprise and S26 is the intruder, then the sensitivity calculation is:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1.5 - 1) \cdot 180 - 2 \cdot 30 - 1 \cdot 5$$
$$= 40 - 90 - 60 - 5$$
$$= -115$$
However, if we take S26 as the target enterprise and S25 as the intruder, then we note that the sensitivity obtained is higher:
$$f = \frac{3 - w_1}{2} = \frac{3 - 1.5}{2} = 0.75$$
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.75 \cdot 180 - (1 - 1) \cdot 200 - 2 \cdot 30 - 1 \cdot 5$$
$$= 27 - 60 - 5$$
$$= -38$$
Since G-Confid takes the highest sensitivity among the various possible target-intruder pairs, a sensitivity of -38 is displayed in the Outcell file.</td></tr>
</table>

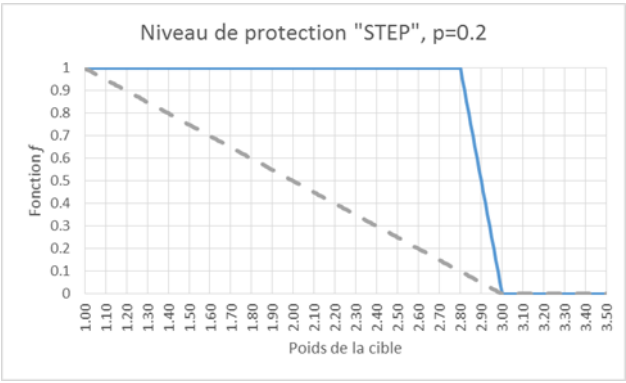
Example 3 – Varied weights with the STEP protection level

With the STEP protection level, sensitivity is calculated as follows:

$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$

where w_i are the weights and x_i are the values of each contributor i and p is the value of p in the p-percent rule. What changes in this case is the f function. It is defined as:

Weight of the target (w_1)	f function
[1 , 3-p)	1
[3-p , 3)	$\frac{3-w_1}{p}$
≥ 3	0



Similar to the *f* function in the LINEAR option (grey dotted line), the *f* function in the STEP option (blue line) has a value of 1 when the target weight is 1 and a value of 0 when the weight exceeds 3. The difference is between weights 1 to 3. Instead of decreasing linearly, the function remains at 1 for weights between 1 and 3-*p*, then decreases linearly between 3-*p* and 3. The *f* function in STEP is therefore more conservative than the *f* function in LINEAR.

Now let's look at the STEP protection level in practice. Continuing to use the variable *WEIGHT_B* for the weight, we obtain the following results:

```
proc sensitivity
   data=Data_weights
   outcell=outcell_ex3
   outconstraint=outconstraint_ex3
   srule="pq 0.2"
   hierarchy="TOT D1 D2 D3
              D4 D5 D6 D7;"
   weightprotlevel="STEP"
   ;
   id entid;
   var datavalue;
   dimension variable;
   weight WEIGHT_B;
run;
```

Outcell_ex3

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	VARIABLE
1	1	0	0	4	415	5	S	C	415	D1
2	2	0	0	4	415	-35	V	C	815	D2
3	3	0	0	4	415	-15	V	C	795	D3
4	4	0	0	4	415	5	S	C	715	D4
5	5	0	0	4	415	-85	V	C	805	D5
6	6	0	0	4	415	-115	V	C	835	D6
7	7	0	0	4	415	-29	V	C	535	D7
8	8	0	0	28	2905	-4115	V	C	4915	TOT

As we did in example 2, we will now show the calculation details for each of the seven domains D1 to D7.

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D1</td><td>
<table>
<tr><th>EntID</th><th>Value</th><th>Poids</th></tr>
<tr><td>S01</td><td>200</td><td>1</td></tr>
<tr><td>S02</td><td>180</td><td>1</td></tr>
<tr><td>S03</td><td>30</td><td>1</td></tr>
<tr><td>S04</td><td>5</td><td>1</td></tr>
</table>
</td><td>Once again, we begin with the simplest example, where all of the weights are 1. First of all, we note that the f function remains 1 when the weight of target $w_1$is between 1 and 3-p. The value of p in our p-percent rule is 0.2. So, the f function is 1 when w_1 is between 1 and 2.8.
If S01 is the target and S02 is the intruder, we obtain:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 40 - 0 - 30 - 5$$
$$= 5$$</td></tr>
<tr><td>D2</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S05</td><td>200</td><td>3</td></tr>
<tr><td>S06</td><td>180</td><td>1</td></tr>
<tr><td>S07</td><td>30</td><td>1</td></tr>
<tr><td>S08</td><td>5</td><td>1</td></tr>
</table>
</td><td>In the second domain, the weight of the target is 3. As we have seen, when the target has a weight of 3, function $f = 0$.
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 0 - 0 - 30 - 5$$
$$= -35$$</td></tr>
</table>

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D3</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S09</td><td>200</td><td>2.9</td></tr>
<tr><td>S10</td><td>180</td><td>1</td></tr>
<tr><td>S11</td><td>30</td><td>1</td></tr>
<tr><td>S12</td><td>5</td><td>1</td></tr>
</table>
</td><td>If the target now has a weight of 2.9, $f = \frac{3-w_1}{p} = \frac{3-2.9}{0.2} = 0.5$. In this situation, we see that sensitivity is higher than what we would have obtained with a weight of 3:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 0.5 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 20 - 0 - 30 - 5$$
$$= -15$$
Note that with the LINEAR approach, we obtained a sensitivity of -33 for this domain.</td></tr>
<tr><td>D4</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S13</td><td>200</td><td>2.5</td></tr>
<tr><td>S14</td><td>180</td><td>1</td></tr>
<tr><td>S15</td><td>30</td><td>1</td></tr>
<tr><td>S16</td><td>5</td><td>1</td></tr>
</table>
</td><td>For a weight of 2.5, f remains at 1. The cell therefore remains sensitive, as if the weight of the target was 1.
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 40 - 0 - 30 - 5$$
$$= 5$$
With the LINEAR approach, we obtained a sensitivity of -25 for this domain.</td></tr>
<tr><td>D5</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S17</td><td>200</td><td>2.5</td></tr>
<tr><td>S18</td><td>180</td><td>1.5</td></tr>
<tr><td>S19</td><td>30</td><td>1</td></tr>
<tr><td>S20</td><td>5</td><td>1</td></tr>
</table>
</td><td>In the fifth domain, we look at the impact of introducing a weight for the intruder. In this case, f remains at 1 since the weight of the target does not change. Sensitivity is much lower:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1.5-1) \cdot 180 - 1 \cdot 30 - 1 \cdot 5$$
$$= 40 - 90 - 30 - 5$$
$$= -85$$
In this case, having a weight $w_2 = 1.5$ therefore offers protection to the target enterprise (S17).</td></tr>
<tr><td>D6</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S21</td><td>200</td><td>2.5</td></tr>
<tr><td>S22</td><td>180</td><td>1.5</td></tr>
<tr><td>S23</td><td>30</td><td>2</td></tr>
<tr><td>S24</td><td>5</td><td>1</td></tr>
</table>
</td><td>Here, we have introduced a weight for one of the enterprises contributing to the noise. Sensitivity is:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1.5-1) \cdot 180 - 2 \cdot 30 - 1 \cdot 5$$
$$= 40 - 90 - 60 - 5$$
$$= -115$$
Once again, we see that it has the effect of reducing sensitivity.</td></tr>
</table>

<table>
<tr><th>Domain</th><th>Enterprises</th><th>Calculation Details</th></tr>
<tr><td>D7</td><td>
<table>
<tr><th>EntID</th><th>Valeur</th><th>Poids</th></tr>
<tr><td>S25</td><td>200</td><td>1</td></tr>
<tr><td>S26</td><td>180</td><td>1.5</td></tr>
<tr><td>S27</td><td>30</td><td>2</td></tr>
<tr><td>S28</td><td>5</td><td>1</td></tr>
</table>
</td><td>In this last domain, we will show once again that it is possible for the target to not be the enterprise with the highest value when weights are present. If we assume that S25 is the target enterprise and S26 is the intruder, then the sensitivity calculation is:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 200 - (1.5 - 1) \cdot 180 - 2 \cdot 30 - 1 \cdot 5$$
$$= 40 - 90 - 60 - 5$$
$$= -115$$

However, if we take S26 as the target enterprise and S25 as the intruder, then we note that the sensitivity obtained is higher:
$$S = pfx_1 - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$$
$$= 0.2 \cdot 1 \cdot 180 - (1 - 1) \cdot 200 - 2 \cdot 30 - 1 \cdot 5$$
$$= 36 - 60 - 5$$
$$= -29$$

Since G-Confid takes the highest sensitivity among the various possible pairs of target/intruder, sensitivity -29 is displayed in the Outcell file.</td></tr>
</table>

Supplementary notes

To conclude, below are a few notes about working with weights in G-Confid:

<u>EXACT protection level</u>

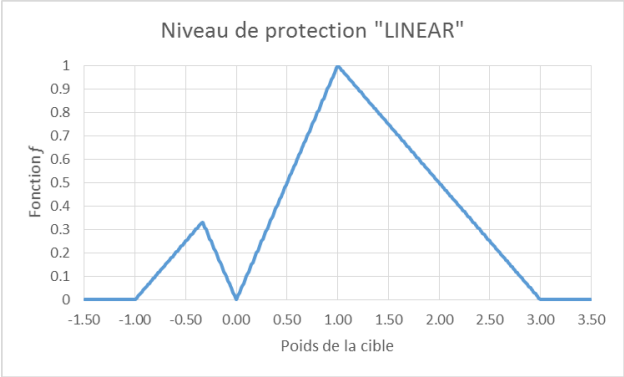
- For information purposes, here is how sensitivity is calculated with an EXACT protection level:
$$S = pw_1x_1 - \sum_{i\geq 3} w_i x_i$$
- This formula is for a case where the intruder knows its own survey weight and the target's weight. In such a case, we can simply adjust the x_i values by w_i weights in the basic sensitivity formula: $S = px_1 - \sum_{i\geq 3} x_i$
- Unlike other levels of protection, a high weight for the target increases the sensitivity rather than lowering it. The user must therefore have good reason to believe that the intruder knows the survey weights before using this formula; otherwise, there is a general risk of obtaining a sensitivity that is too high.
- However, in practice, it is highly unlikely that the intruder is aware of the survey weights. This level of protection is not recommended when the intruder is not aware of the survey weights. That is why we have not presented examples for this level of protection.

Handling weights smaller than 1 and negative weights

- The three levels of protection available (LINEAR, STEP and EXACT) can handle weights smaller than 1 and negative weights to calculate sensitivity. However, at the present time, we cannot claim that the way these weights are handled in G-Confid 1.07 is optimal. We therefore recommend that users exercise caution when using these types of weights.
- For the LINEAR and STEP protection levels, the sensitivity formula is:
  $S = pf|x_1| - (|w_2| - 1)|x_2| - \sum_{i\geq 3} |w_i| \cdot |x_i|$
- Where the *f* functions are:

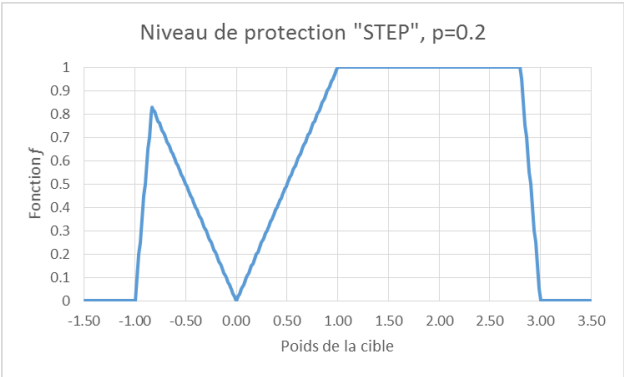
LINEAR Protection Level

Weight of the target (w_1)	*f* function
< -1	0
[-1 ; -0.33)	$\frac{w_1 + 1}{2}$
[-0.33 ; 0)	$-w_1$
[0 ; 1)	w_1
[1 ; 3)	$\frac{3 - w_1}{2}$
> 3	0



STEP protection level

Weight of the target (w_1)	*f* function
< -1	0
[-1 , $\frac{-1}{1+p}$)	$\frac{w_1 + 1}{p}$
[$\frac{-1}{1+p}$, 0)	$-w_1$
[0 , 1)	w_1
[1 , 3-p)	1
[3-p , 3)	$\frac{3 - w_1}{p}$
≥ 3	0



Multiple contributions with different weights for an enterprise

- We know that it is possible to have several contributions from the same enterprise included in a cell total. Having different weights for each contribution can cause a problem since it is uncertain which weight to use to determine the value of *f* if the enterprise is the target. The actual weight is then used.
- For example, if the target enterprise has two contributions, A and B, its actual weight is: $w_{1*} = \frac{w_{1A}x_{1A}+w_{1B}x_{1B}}{x_{1A}+x_{1B}}$. Based on this w_{1*}, we can determine the value of *f*, then calculate the sensitivity: $S = pf(x_{1A} + x_{1B}) - (w_2 - 1)x_2 - \sum_{i\geq 3} w_i x_i$.
- The same type of formula applies when there are more than two contributions for the same target enterprise.

NK rule and weights

- Without weights, sensitivity is calculated as follows with the NK rule: $S = \frac{100-k}{k}\sum_{i\leq n} x_i - \sum_{i>n} x_i$
- With weights, the formula is: $S = \frac{100-k}{k}\sum_{i\leq n} f_i x_i - \sum_{i>n} w_i x_i$.
- In other words, if *n*=2, we must calculate f_1 and f_2 for both dominant enterprises.
- Note that the *n* dominant enterprises are not necessarily the ones with the largest values, since we also need to account for the weights. We must therefore test different combinations of *n* dominant enterprises and choose those that give the greatest sensitivity.

2.7 Negative values

As mentioned in the examples of 2.2 Sensitivity rules, the normal method for calculating the sensitivity of various cells in a table requires that the data are not negative. Prior to version 1.07, G-Confid automatically excluded negative values from processing, displaying a warning message in the SAS log. One of the new features of G-Confid 1.07 is that it is now possible to work with negative values.

Obviously, when dealing with negative values, a slightly different strategy is needed to perform sensitivity calculations. The following examples will show different ways of handling negative values in G-Confid.

Creation of the fictitious data file

To begin, we will create a fictitious data file on which we will be basing this series of examples:

```
data Data_Neg;
Entid='S01';   Region="A";   Profit=180;   Size=9125;   output;
Entid='S02';   Region="A";   Profit=-35;   Size=10000;  output;
Entid='S03';   Region="A";   Profit=30;    Size=985;    output;
Entid='S04';   Region="A";   Profit=15;    Size=35;     output;
Entid='S01';   Region="B";   Profit=-90;   Size=-50000; output;
Entid='S05';   Region="B";   Profit=40;    Size=45;      output;
Entid='S02';   Region="B";   Profit=30;    Size=15000;  output;
Entid='S06';   Region="B";   Profit=20;    Size=760;    output;
Entid='S07';   Region="C";   Profit=-95;   Size=4570;   output;
Entid='S08';   Region="C";   Profit=40;    Size=3875;   output;
Entid='S02';   Region="C";   Profit=-40;   Size=10000;  output;
Entid='S09';   Region="D";   Profit=100;   Size=500;    output;
Entid='S10';   Region="D";   Profit=0;     Size=8000;   output;
Entid='S11';   Region="D";   Profit=40;    Size=7025;   output;
run;
```

Data_neg

	Entid	Region	Profit	Size
1	S01	A	180	9125
2	S02	A	-35	10000
3	S03	A	30	985
4	S04	A	15	35
5	S01	B	-90	-50000
6	S05	B	40	45
7	S02	B	30	15000
8	S06	B	20	760
9	S07	C	-95	4570
10	S08	C	40	3875
11	S02	C	-40	10000
12	S09	D	100	500
13	S10	D	0	8000
14	S11	D	40	7025

The file contains data on the profits of 11 enterprises in four different regions (A, B, C and D). Note that enterprise S01 is in regions A and B, and enterprise S02 is in regions A, B and C. All of the other enterprises are in one region only. The *Profit* variable has a few negative values and one zero value. Note that the *Size* variable will be used in example 3, which deals with proxy variables. We will come back to it at that time.

Example 0: Rejected negative values

As explained above, earlier versions of G-Confid did not allow us to work with negative values. To ensure consistency between version 1.07 and earlier versions, G-Confid rejects negative values by default, as it did in the past. We will begin by looking at what happens when G-Confid encounters negative values with no further instructions:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex0
   outconstraint=outconstraint_ex0
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C D;"
   ;
   id Entid;
   var Profit;
   dimension Region;
run;
```

Outcell_ex0

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	3	225	12	S	C	225	A
2	2	0	0	3	90	-14	V	C	90	B
3	3	0	0	1	40	6	S	C	40	C
4	4	0	0	3	140	15	S	C	140	D
5	5	0	0	10	495	-188	V	C	495	TOT_REGION

We see that in the *Outcell* file, some of the contributors have not been included (see the *NbRespondents* variable). Our four negative values were automatically excluded and are not considered at all in the *Outcell* file. The same is observed in the sensitivity calculations. If we attempt to reproduce the sensitivity calculation for region A, we obtain $S_A = 0.15 \cdot 180 - 15 = 12$. In other words, the value -35 is not considered at all in the calculation.

It may be interesting to examine the SAS log. One of the first things we note is the warning message due to the negative values:

```
WARNING: There were 4 observations dropped from the DATA data set because the variable Profit is
         negative.
WARNING: These observations will not be used for sensitivity calculation.
```

G-Confid is warning that the four negative values for the *Profit* variable have been excluded and will not be considered in the sensitivity calculation. The Microdata Statistics table is also interesting in the SAS log. We see that we have 10 valid observations, including a zero, but that four of the observations are considered invalid because they are negative.

```
Microdata Statistics

Total number of observations                                                    14
1. Number of valid observations                                                 10
   a. # of valid observs from anonymous respondents                              0
   b. # of valid observs from non-anonymous respondents                         10
   c. # of valid observs with negative data for respondents                      0
   d. # of valid observs with negative or missing data for proxy variable       0
   e. # of valid observs with zero or negative data for weight variable         0
   f. # of valid observs with invalid data for waiver variable                  0

2. Number of valid observations with zero value                                 1

3. Number of invalid observations                                               4
   a. # of observs invalid due to missing data for respondents                  0
   b. # of observs invalid due to negative data for respondents                 4
   c. # of observs invalid due to missing code for dimension                    0
   d. # of observs invalid due to invalid code for dimension                    0
   e. # of observs invalid due to missing data for shadow variable              0
   f. # of observs invalid due to missing data for weight variable              0

4. # of observs invalid because dimension code set not in hierarchy            0
```

Example 1: PQ rule

In order to work with negative values, G-Confid requires the ACCEPTNEGATIVE option. If we redo the above example including this option, we obtain the following results:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex1a
   outconstraint=outconstraint_ex1a
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   ;
   id Entid;
   var Profit;
   dimension Region;
run;
```

Outcell_ex1a

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	4	190	-18	V	C	260	A
2	2	0	0	4	0	-36.5	V	C	180	B
3	3	0	0	3	-95	-25.75	V	C	175	C
4	4	0	0	3	140	15	S	C	140	D
5	5	0	0	11	235	-339.5	V	C	755	TOT_REGION

This time, when we look at each of the regions, we find the expected number of contributors in the *NbRespondents* variable. In the SAS log, instead of a warning message, we see the following note:

```
NOTE: There were 4 observations read from the DATA data with negative values in variable Profit.
```

Also, in the Microdata Statistics table in the SAS log, we see that the four negative values are now considered to be valid:

```
Microdata Statistics

Total number of observations                                                14
1. Number of valid observations                                             14
   a. # of valid observs from anonymous respondents                          0
   b. # of valid observs from non-anonymous respondents                      14
   c. # of valid observs with negative data for respondents                   4
   d. # of valid observs with negative or missing data for proxy variable    0
   e. # of valid observs with zero or negative data for weight variable      0
   f. # of valid observs with invalid data for waiver variable               0

2. Number of valid observations with zero value                              1

3. Number of invalid observations                                           0
   a. # of observs invalid due to missing data for respondents              0
   b. # of observs invalid due to negative data for respondents             0
   c. # of observs invalid due to missing code for dimension                0
   d. # of observs invalid due to invalid code for dimension                0
   e. # of observs invalid due to missing data for shadow variable          0
   f. # of observs invalid due to missing data for weight variable          0

4. # of observs invalid because dimension code set not in hierarchy         0
```

The negative values now contribute to sensitivity calculations. The question becomes: how do we account for them? In reality, the answer is rather intuitive. We simply take the absolute value of our values to perform the calculations and to determine the roles of the contributors in each cell. For example, for region A, there are four contributors with the following values: 180, -35, 30 and 15. If we take the absolute value of each of these values, we get: 180, 35, 30 and 15. The value of 180 is our target, 35 is the intruder and the other two values contribute to the noise. We obtain the following sensitivity calculation: $S_A = 0.15 \cdot 180 - 30 - 15 = -18$.

To obtain the results for regions B, C and D, we apply the same process. However, things get a bit complicated for the marginal value representing the total for all the regions. As a reminder, we have two enterprises, S01 and S02, spread across different regions.

First approach: ADDITIVENOISE

The first way of accounting for the negative values is to simply add the absolute values from the *Profit* variable by enterprise and then determine the roles of the enterprises accordingly.

We see that enterprise S01 has made two contributions to the total: 180 and -90. If we add the absolute values of these contributions, we get: $|180| + |-90| = 270$. Using the same principle for S02, we get: $|-35| + |30| + |-40| = 105$. These enterprises are the two largest contributors to the total. The first is therefore considered to be the target, while the second is the intruder. We then obtain the following sensitivity calculation:

$$\begin{aligned} S_{TOT} &= 0.15 \cdot 270 - (100 + 95 + 40 + 40 + 40 + 30 + 20 + 15) \\ &= 40.5 - 380 \\ &= -339.5 \end{aligned}$$

We end up with the sensitivity as calculated in our *Outcell* file for the total. Note that this approach is referred to as ADDITIVENOISE in G-Confid and it is the default option used when ACCEPTNEGATIVE is indicated with no further details. The same results are obtained by specifying "ACCEPTNEGATIVE" alone or by specifying "ACCEPTNEGATIVE ADDITIVENOISE" in the PROC SENSITIVITY options.

Second approach: NOADDITIVENOISE

There is, however, another approach in G-Confid to resolve this problem. To use this approach, NOADDITIVENOISE must be specified in the options:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex1b
   outconstraint=outconstraint_ex1b
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   NoAdditiveNoise
   ;
   id Entid;
   var Profit;
   dimension Region;
run;
```

Outcell_ex1b

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	4	190	-18	V	C	260	A
2	2	0	0	4	0	-36.5	V	C	180	B
3	3	0	0	3	-95	-25.75	V	C	175	C
4	4	0	0	3	140	15	S	C	140	D
5	5	0	0	11	235	-305	V	C	515	TOT_REGION

We see that the sensitivity calculations give the same results, other than the cell that contains the total. This is normal, since the ADDITIVENOISE and NOADDITIVENOISE approaches may obtain different results for marginal cells, but not for internal cells.

When there are different contributions for the same enterprise, ADDITIVENOISE takes the absolute value for each of the contributions and adds them together. Conversely, NOADDITIVENOISE begins by adding the contributions, then takes the absolute value of the sum.

So, if we want to calculate the sensitivity of the total with NOADDITIVENOISE, we use the value $|180 - 90| = 90$ for S1 and the value $|-35 + 30 - 40| = 45$ for S2. Note that S01 and S02 will no longer be the target and intruder, since there are two enterprises (S09 and S07) with values greater than 100 and 95. S09 therefore plays the role of target and S07 is the intruder, while all of the other enterprises contribute to the noise. We obtain the following sensitivity calculation for the total cell:

$$\begin{aligned} S_{TOT} &= 0.15 \cdot 100 - (90 + 45 + 40 + 40 + 40 + 30 + 20 + 15) \\ &= 15 - 320 \\ &= -305 \end{aligned}$$

Example 2: NK rule

In this second example, we want to show what happens to the sensitivity calculation when a NK rule is used with negative values. Let's begin with the ADDITIVENOISE approach:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex2a
   outconstraint=outconstraint_ex2a
   srule="NK 2 80"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   AdditiveNoise
   ;
   id Entid;
   var Profit;
   dimension Region;
run;
```

Outcell_ex2a

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	4	190	8.75	S	C	260	A
2	2	0	0	4	0	-17.5	V	C	180	B
3	3	0	0	3	-95	-6.25	V	C	175	C
4	4	0	0	3	140	35	S	C	140	D
5	5	0	0	11	235	-286.25	V	C	755	TOT_REGION

Similar to the PQ rule, the method for calculating sensitivity with negative values is rather intuitive. As a reminder, we have four contributors to region A: 180, -35, 30 and 15. Once again, we take the absolute value of these values to perform our calculations. We then obtain: 180, 35, 30 and 15. Given that we have a NK rule where $n = 2$ and $k = 80$, the calculation is:

$$\begin{aligned} S_A &= \left(\frac{100 - 80}{80}\right) \cdot (180 + 35) - (30 + 15) \\ &= 53.75 - 45 \\ &= 8.75 \end{aligned}$$

The same calculation is performed to obtain the sensitivity for the other regions. As for the total, remember that we are using the ADDITIVENOISE approach in this example, which means that the absolute values of the contributions by enterprise will be added together before their roles are determined. Enterprise S01 has a total contribution of 270 and S02 has a total contribution of 105. They are therefore the two dominant enterprises, and we obtain:

$$\begin{aligned} S_{TOT} &= \left(\frac{100 - 80}{80}\right) \cdot (270 + 105) - (100 + 95 + 40 + 40 + 40 + 30 + 20 + 15) \\ &= 93.75 - 380 \\ &= -286.25 \end{aligned}$$

Second approach: NOADDITIVENOISE

Now let's see what happens when NOADDITIVENOISE is specified:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex2b
   outconstraint=outconstraint_ex2b
   srule="NK 2 80"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   NoAdditiveNoise
   ;
   id Entid;
   var Profit;
   dimension Region;
run;
```

Outcell_ex2b

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region
1	1	0	0	4	190	8.75	S	C	260	A
2	2	0	0	4	0	-17.5	V	C	180	B
3	3	0	0	3	-95	-6.25	V	C	175	C
4	4	0	0	3	140	35	S	C	140	D
5	5	0	0	11	235	-271.25	V	C	515	TOT_REGION

As explained in example 1, using this second approach does not change the sensitivity calculation for internal cells. The difference is seen in the marginal cells. In this example, we see that the sensitivity for the total is not the same as it was with the first method.

In this case, the contributions by enterprise are added together before obtaining the absolute value. S01 therefore has a value of 90 and S02 has a value of 45. Our two dominant enterprises are now S09 with a value of 100 and S07 with a value of 95. The sensitivity calculation for the total cell is:

$$\begin{aligned} S_{TOT} &= \left(\frac{100-80}{80}\right)\cdot(100+95)-(90+45+40+40+40+30+20+15) \\ &= 48.75-320 \\ &= -271.25 \end{aligned}$$

Example 3: Using a proxy

In this third example, we will show how to use a proxy. A proxy is used to get a second view of the importance of an enterprise in a cell. This may be useful in better protecting contributions that are negative or close to zero. In our fictitious data file *Data_Neg*, *Size* is the variable that will serve as an auxiliary variable to calculate the proxy variable *Z*. Let's begin by showing the code necessary to include a proxy in G-Confid. We can then explain how this variable is used.

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex3a
   outconstraint=outconstraint_ex3a
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   Proxypercentile=0.25
   AdditiveNoise
   ;
   id Entid;
   var Profit;
   dimension Region;
   proxysize Size;
run;
```

Outcell_ex3a

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	TotalProxy	Region
1	1	0	0	4	190	-18	V	C	260	20145	A
2	2	0	0	4	0	-33.75	V	C	287.5	-34195	B
3	3	0	0	3	-95	-25.75	V	C	175	18445	C
4	4	0	0	3	140	-13	V	C	168	15525	D
5	5	0	0	11	235	-354.75	V	C	890.5	19920	TOT_REGION

First of all, like the previous examples, we will begin by showing what happens with ADDITIVENOISE before showing NOADDITIVENOISE. What is new in this code is the PROXYPERCENTILE option and the PROXYSIZE statement. If PROXYPERCENTILE is not specified, G-Confid chooses a value of PROXYPERCENTILE=0.10 by default. Here we have chosen a PQ 0.15 rule, which is the same rule that was used in example 1. We see that the sensitivity calculation is the same as example 1 for regions A and C, but different for regions B and D and the total.

The following notes appear in the SAS log:

```
NOTE: There were 4 observations read from the DATA data with negative values in variable Profit.
NOTE: There were 1 observations read from the DATA data set with missing or negative values in
      variable Size
ProxyRatio (calculated from ProxyPercentile =    0.25000) =    0.00350
```

Our PROXYPERCENTILE=0.25 value was used to calculate a value of 0.0035 for the *ProxyRatio*. We will now look at what that means. Understanding this will help to understand how a sensitivity calculation works with a proxy.

For each contribution in our dataset, there is a value for our variable of interest (*Profit*) and a value for our auxiliary variable (*Size*). For each contribution *i*, we take the absolute value of the ratio between the two variables: $Ratio(i) = \left|\frac{Profit(i)}{Size(i)}\right|$. Next, since we have specified that PROXYPERCENTILE=0.25, we are seeking the 25th percentile of these ratios and we obtain *ProxyRatio*=0.0035. Once the *ProxyRatio* has been determined, we can perform the sensitivity calculations [5].

The potential impact of the proxy is best seen in the sensitivity calculation for region D. The following three enterprises were in region D:

Enterprise	Profit	Size
S09	100	500
S10	0	8000
S11	40	7025

With a PQ 0.15 rule, we obtained the following sensitivity: $S = 100 \cdot 0.15 - 0 = 15$. In other words, S09 is the target, S11 is the intruder and S10 is noise, but does not provide any real protection. However, when we look at the *Size* variable, we see that enterprise S10 actually has a large size. We would like to take this into account in our sensitivity calculation.

With a PROXYSIZE statement, G-Confid calculates sensitivity based on the proxy variable *Z*, which is calculated the following way:

$$Z = \max(|Profit|, ProxyRatio \cdot |Size|)$$

We obtain:

$$Z_{S09} = \max(|100|, 0.0035 \cdot |500|) = 100$$
$$Z_{S10} = \max(|0|, 0.0035 \cdot |8000|) = 28$$
$$Z_{S11} = \max(|40|, 0.0035 \cdot |7025|) = 40$$

The sensitivity calculation becomes: $S = 100 \cdot 0.15 - 28 = -13$. In other words, in this situation, we were able to account for the protection that would have normally been obtained in this cell due to enterprise S10.

We proceed the same way for the sensitivity calculation for the other regions. We will now look at how the sensitivity calculation is performed for the total cell. As a reminder, we have used the ADDITIVENOISE option, so we determine *Z* for each of the contributions, then add them together to obtain a total by enterprise. Enterprises S01 and S02 obtain the following values for *Z*:

$$\begin{aligned} Z_{S01} &= \max(|180|, 0.0035 \cdot |9125|) + \max(|-90|, 0.0035 \cdot |-50000|) \\ &= \max(180, 31.9375) + \max(90, 175) \\ &= 355 \\ Z_{S02} &= \max(|-35|, 0.0035 \cdot |10000|) + \max(|30|, 0.0035 \cdot |15000|) + \max(|-40|, 0.0035 \cdot |10000|) \\ &= \max(35, 35) + \max(30, 52.5) + \max(40,35) \\ &= 127.5 \end{aligned}$$

[5] Note that we could have specified "PROXYRATIO=0.0035" in PROC SENSITIVITY rather than "PROXYPERCENTILE=0.25" in this example and we would have obtained the exact same results.

If we take the time to calculate all of the *Z* values for the 11 enterprises, we obtain: 355, 127.5, 30, 15, 40, 20, 95, 40, 100, 28, 40, which gives the following sensitivity calculation:

$$\begin{aligned} S_{TOT} &= 0.15 \cdot 355 - (100 + 95 + 40 + 40 + 40 + 30 + 28 + 20 + 15) \\ &= 53.25 - 408 \\ &= -354.75 \end{aligned}$$

Second approach: NOADDITIVENOISE

If we now use the NOADDITIVENOISE option, we obtain the following results:

```
proc sensitivity
   data=Data_Neg
   outcell=outcell_ex3b
   outconstraint=outconstraint_ex3b
   srule="pq 0.15"
   hierarchy="TOT_REGION A B C D;"
   AcceptNegative
   Proxypercentile=0.25
   NoAdditiveNoise
   ;
   id Entid;
   var Profit;
   dimension Region;
   proxysize Size;
run;
```

Outcell_ex3b

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	TotalProxy	Region
1	1	0	0	4	190	-18	V	C	260	20145	A
2	2	0	0	4	0	-33.75	V	C	287.5	-34195	B
3	3	0	0	3	-95	-25.75	V	C	175	18445	C
4	4	0	0	3	140	-13	V	C	168	15525	D
5	5	0	0	11	235	-386.540625	V	C	673.5625	19920	TOT_REGION

Similar to examples 1 and 2, we see that the NOADDITIVENOISE option only affects the total cell. With this option, only one *Z* is calculated per enterprise rather than a *Z* for each contribution. We obtain the following results for enterprises S01 and S02:

$$\begin{aligned} Z_{S01} &= \max(|180 - 90|, 0.0035 \cdot |9125 - 50000|) \\ &= \max(90, 143.0625) \\ &= 143.0625 \\ Z_{S02} &= \max(|-35 + 30 - 40|, 0.0035 \cdot |10000 + 15000 + 10000|) \\ &= \max(45, 122.5\ ) \\ &= 122.5 \end{aligned}$$

These are the only two *Z* values that change. So, for the 11 enterprises, we end up with the following *Z* values: 143.0625, 122.5, 30, 15, 40, 20, 95, 40, 100, 28, 40. The sensitivity calculation is then:

$$\begin{aligned} S_{TOT} &= 0.15 \cdot 143.0625 - (100 + 95 + 40 + 40 + 40 + 30 + 28 + 20 + 15) \\ &= 21.459375 - 408 \\ &= -386.540625 \end{aligned}$$

PROXYDIAG option

Using the PROXYDIAG option in PROC SENSITIVITY enables us to show a few interesting results regarding the effect of a proxy on cell sensitivity. If we add the PROXYDIAG option to the code above, two new tables are displayed in the SAS log:

Table P1: Effect on sensitivity of using a proxy variable

	Sensitivity decreased	Sensitivity unchanged	Sensitivity increased
Number of internal cells	1	2	1
Contains positive and negative data	0	2	1
All contributions >=0 and some > 0	1	0	0
All contributions = 0	0	0	0
All contributions <=0 and some < 0	0	0	0
Number of marginal cells	1	0	0
Contains positive and negative data	1	0	0
All contributions >=0 and some > 0	0	0	0
All contributions = 0	0	0	0
All contributions <=0 and some < 0	0	0	0
Total number of published cells	2	2	1
Contains positive and negative data	1	2	1
All contributions >=0 and some > 0	1	0	0
All contributions = 0	0	0	0
All contributions <=0 and some < 0	0	0	0

Table P2: Effect on cell status counts of using a proxy variable

	Sensitivity without proxy		Sensitivity with proxy	
	Safe	Sensitive	Safe	Sensitive
Number of internal cells	3	1	4	0
Contains positive and negative data	3	0	3	0
All contributions >=0 and some > 0	0	1	1	0
All contributions = 0	0	0	0	0
All contributions <=0 and some < 0	0	0	0	0
Number of marginal cells	1	0	1	0
Contains positive and negative data	1	0	1	0
All contributions >=0 and some > 0	0	0	0	0
All contributions = 0	0	0	0	0
All contributions <=0 and some < 0	0	0	0	0
Total number of published cells	4	1	5	0
Contains positive and negative data	4	0	4	0
All contributions >=0 and some > 0	0	1	1	0
All contributions = 0	0	0	0	0
All contributions <=0 and some < 0	0	0	0	0
Number of aggregates	0	0	0	0

P1 indicates whether the proxy causes the sensitivity to increase or decrease, or whether it remains unchanged. Among the internal cells, including those for regions A, B, C and D, there are two cells whose sensitivity remains unchanged, one cell whose sensitivity increased and another cell whose sensitivity decreased. The marginal cell, in this case the total, had a decreased sensitivity when the proxy is used.

Table P2 shows the impact of the proxy on the status of the cells (sensitive or safe). We see that one of the internal cells was originally sensitive, but became safe through the use of the proxy. Our marginal cell for the total remained unchanged.

2.8 G-Confid and the SAS log

This example is meant to explain the content of the SAS log when G-Confid is used.

Consider the following dataset detailing business enterprises' money donations to sports programs in their province/territory of operation, as well as their method of donation (a data hierarchy has been included as well, which will be needed for the subsequent run of PROC SENSITIVITY):

```
data sports;
   input ENTID$ SPORT$ GEO$ VIA$ DONAMT;
   cards;
   DA1001 BASE NL CHEQUE 7463
   DA1301 BASE NB CHEQUE 4832
   DA1302 BASE NB CHEQUE 8473
   DA2401 BASE QC ETRANS 9204
   DA2402 FOOT QC ETRANS 3403
   DA2403 FOOT QC ETRANS 4934
   DA2403 FOOT QC ETRANS 2348
   DA3501 FOOT ON CHEQUE 3897
   DA3501 HOCKEY ON CHEQUE 8828
   DA3501 HOCKEY ON CHEQUE 4234
   DA3502 HOCKEY ON CHEQUE 7489
   DA4601 HOCKEY MB ETRANS 7273
   DA4601 VOLLEY MB ETRANS 2848
   DA4701 VOLLEY SK CHEQUE 3086
   DA4801 VOLLEY AB ETRANS 5482
   DA4802 VOLLEY AB ETRANS 4038
   DA5901 BASKET BC CHEQUE 8374
   DA5902 BASKET BC CHEQUE 5948
   DA5903 BASKET BC CHEQUE 5994
   DA6201 BASKET NU ETRANS 6473
   ;
run;

%let myhierarchy="
   /* sport dimension */
   SPORT_TOT ICE FIELD GYM:
   ICE HOCKEY:
   FIELD FOOT BASE:
   GYM BASKET VOLLEY;

   /* geo dimension */
   CAN ATL QC ON PRAIR BC TERR:
   ATL NL PE NS NB:
   PRAIR MB SK AB:
   TERR YT NT NU;

   /* via dimension */
   VIA CHEQUE ETRANS;";
```

Our dataset is for example use only, and thus is purposely small (20 entries). Each entry contains the enterprise's ID (notice that some enterprises made multiple donations and thus we see a repeated ID), the sport to which they donated money, their province/territory of operation (and thus the province/territory of the sport to which they donated), the method via which they donated money, and the dollar amount of the donation.

PROC SENSITIVITY

To obtain a suppression pattern for our tabular data, we must first run PROC SENSITIVITY to calculate the sensitivity values of our table cells. The code we input into SAS to do this is:

```
proc sensitivity
   data=sports
   outconstraint=outconstraint
   outcell=outcell
   outlargest=outlargest
   srule="pq 0.2"
   hierarchy=&myhierarchy
   ;
   id entid;
   var donamt;
   dimension sport geo via;
run;
```

Now we will look at what the Log window produces as a result of running PROC SENSITIVITY. In the first part of the Log window we see this:

```
131  proc sensitivity
132      data=sports
133      outconstraint=outconstraint
134      outcell=outcell
135      outlargest=outlargest
136      srule="pq 0.2"
137      hierarchy=&myhierarchy
138      ;
139      id entid;
140      var donamt;
141      dimension sport geo via;
142  run;
```

Here, as expected, the Log window just outputs the code we inputted.

Next we see the following:

```
NOTE: --- G-Confid System 1.07.001 developed by Statistics Canada ---
NOTE: PROCEDURE SENSITIVITY Version 1.07.001
NOTE: Created on Mar 2 2018 at 13:08:25
NOTE: Email: G-Confid@statcan.gc.ca
NOTE: This procedure/macro is for use at Statistics Canada only.

NOTE: DATA = WORK.SPORTS
NOTE: OUTCONSTRAINT = WORK.OUTCONSTRAINT
NOTE: OUTCELL = WORK.OUTCELL
NOTE: OUTLARGEST = WORK.OUTLARGEST
NOTE: OUTTARGETS = _NULL_
NOTE: OUTPAIRS = _NULL_
NOTE: SRULE = pq 0.2
NOTE: HIERARCHY = /* sport dimension */SPORT_TOT   ICE FIELD GYM:ICE   HOCKEY:FIELD   FOOT
      BASE:GYM   BASKET VOLLEY;/* geo dimension */CAN   ATL QC ON PRAIR BC TERR:ATL   NL
      PE NS NB:PRAIR   MB SK AB:TERR   YU NT NU;/* via dimension */VIA   CHEQUE ETRANS;
NOTE: RANGE not specified.
NOTE: M = 5 (default)
NOTE: X = 0.00000 (default)
NOTE: Y = 0.00000 (default)
NOTE: Z = 0.00000 (default)
NOTE: TOLERANCE = 0.01000 (default)
NOTE: MINRESP not specified.
NOTE: NOPROXYDIAG
NOTE: WEIGHTPROTLEVEL = LINEAR
NOTE: MINRESPW not specified.
NOTE: ADDITIVENOISE
NOTE: NOWEIGHTDIAG
NOTE: REJECTNEGATIVE
```

```
NOTE: ID = ENTID
NOTE: VAR = DONAMT
NOTE: SHADOW not specified.
NOTE: PROXYSIZE not specified.
NOTE: WEIGHT not specified.
NOTE: DIMENSION = SPORT GEO VIA
NOTE: WAIVER not specified.
NOTE: PWAIVER not specified.
NOTE: BY not specified.
NOTE: PRINTCODES
NOTE: LIMITWARNINGS

SRule Alpha[0][0] = 0.200000

SRule Alpha[0][1] = 0.000000

SRule Alpha[0][2] = -1.000000

SRule Alpha[0][3] = -1.000000

SRule Alpha[0][4] = -1.000000
```

In the first five lines, we just get logistical information regarding G-Confid and PROC SENSITIVITY.

Then we see output regarding the components of our specific PROC SENSITIVITY run. You'll notice that this output looks very similar to the code we ran; it tells us what PROC SENSITIVITY has identified as the specific inputs for its various components (this is a good place to check if PROC SENSITIVITY has correctly identified what we want it to identify). For example, for the DATA component (the tabular data we want to calculate sensitivity values for), we see the input of "WORK.SPORTS". "WORK" here refers to the Work subfolder of the Explorer folder, which contains outputs like tables that have been produced as part of the user's 'work' in SAS; it is the default location to which G-Confid saves output files (the user is permitted to define a LIBNAME and write output files to that directory instead if they wish). Continuing with our example, if you were to go to the Work subfolder, you would find a table called *Sports*, which is of course the name we gave to our original input data. You will see that the Log window has identified inputs for all of its components, even optional ones that our PROC SENSITIVITY run doesn't include (these are simply labeled as "not specified", which is what we expect to see). Some components of PROC SENSITIVITY are required to be specified, while others are not. If there is a required component that you have not specified, an error message will appear indicating such. As we are dealing with a basic example of PROC SENSITIVITY, there may be some components that are unfamiliar. This is nothing to be concerned about, we can trust that PROC SENSITIVITY is doing its job correctly for those components. For more information on the purpose of each component, see the 5. Syntax section.

The last five lines of the output for this section relate to the mathematics behind the sensitivity calculations, specifically the values that the alphas take (which determine the value of the cell's sensitivity). The alphas are described in the examples about 2.2 Sensitivity rules.

After this section, we have:

```
Dimension SPORT

    Code SPORT_TOT - Decomposition [1/1]

        Code ICE - Decomposition [1/1]

            Code HOCKEY - Decomposition [0/0]

        Code FIELD - Decomposition [1/1]

            Code FOOT - Decomposition [0/0]

            Code BASE - Decomposition [0/0]

        Code GYM - Decomposition [1/1]

            Code BASKET - Decomposition [0/0]

            Code VOLLEY - Decomposition [0/0]
Dimension GEO

    Code CAN - Decomposition [1/1]

        Code ATL - Decomposition [1/1]

            Code NL - Decomposition [0/0]

            Code PE - Decomposition [0/0]

            Code NS - Decomposition [0/0]

            Code NB - Decomposition [0/0]

        Code QC - Decomposition [0/0]

        Code ON - Decomposition [0/0]

        Code PRAIR - Decomposition [1/1]

            Code MB - Decomposition [0/0]

            Code SK - Decomposition [0/0]

            Code AB - Decomposition [0/0]

        Code BC - Decomposition [0/0]

        Code TERR - Decomposition [1/1]

            Code YU - Decomposition [0/0]

            Code NT - Decomposition [0/0]

            Code NU - Decomposition [0/0]
```

```
Dimension VIA

    Code VIA - Decomposition [1/1]

        Code CHEQUE - Decomposition [0/0]

        Code ETRANS - Decomposition [0/0]

Range for dimension SPORT

Range is empty

Range for dimension GEO

Range is empty

Range for dimension VIA

Range is empty
```

In this section, we essentially have a representation of the hierarchy we specified. For each of our three dimensions (*SPORT*, *GEO*, and *VIA*), we get a decomposition of the components indicated in our hierarchy. Where we see a "1/1" in square brackets for a given code, it means that component decomposes into other components. For example, the SPORT_TOT code has a "1/1" because it decomposes into the codes ICE, FIELD, and GYM. Where we see a "0/0" in square brackets for a given code, it means that component is at the lowest level of the hierarchy (thus it cannot be decomposed further). For example, the FOOT code has a "0/0" because it is a lowest level member of the FIELD code. In the next seven lines of this section, we see a series of statements regarding the range. In our example, we didn't specify a range, so the Log window indicates that our range is empty for all three of our dimensions. If we did have a range, we would see an output similar to what the Log window produced for the hierarchy.

Nearing the end of our PROC SENSITIVITY Log window documentation, we have:

```
Microdata Statistics

Total number of observations                                                    20
1. Number of valid observations                                                 20
   a. # of valid observs from anonymous respondents                              0
   b. # of valid observs from non-anonymous respondents                         20
   c. # of valid observs with negative data for respondents                      0
   d. # of valid observs with negative or missing data for proxy variable       0
   e. # of valid observs with zero or negative data for weight variable         0
   f. # of valid observs with invalid data for waiver variable                  0

2. Number of valid observations with zero value                                 0

3. Number of invalid observations                                               0
   a. # of observs invalid due to missing data for respondents                  0
   b. # of observs invalid due to negative data for respondents                 0
   c. # of observs invalid due to missing code for dimension                    0
   d. # of observs invalid due to invalid code for dimension                    0
   e. # of observs invalid due to missing data for shadow variable              0
   f. # of observs invalid due to missing data for weight variable              0

4. # of observs invalid because dimension code set not in hierarchy            0
```

```
Cell Statistics

                                Number calculated     Number sensitive    Percent sensitive

All cells with zero value                       0                    0                 0.00
All cells with non-zero value
  Internal cells                               12                   11                91.67
  Marginal cells                              104                   65                62.50

All cells                                     116                   76                65.52
Aggregates                                     19                    1                 5.26

Total                                         135                   77                57.04


Cell Statistics

                                Number calculated     Number sensitive    Percent sensitive

All cells with zero value                       0                    0                 0.00
All cells with non-zero value
  Internal cells                               12                   11                91.67
  Marginal cells                              104                   65                62.50

All cells                                     116                   76                65.52
Aggregates                                     19                    1                 5.26

Total                                         135                   77                57.04
```

In the Microdata Statistics section, the Log window outputs statistics regarding the microdata (in our case, the *Sports* dataset we created). Here we get information on the types of observations PROC SENSITIVITY recognized from our input data, and how many observations of each type. As indicated in the statistics, a valid observation is one without missing data (or negative data if the ACCEPTNEGATIVE parameter isn't specified) for respondents, without missing or invalid codes for the dimension, and without missing data for the shadow or weight variables. As of the most recent version of G-Confid (version 1.07), negative data values for respondents can be accepted provided that the ACCEPTNEGATIVE parameter is specified (previous versions did not have this functionality). In the Cell Statistics section, the Log window gives us statistics about the sensitivity of the cells in our input data, including internal and marginal cells, and aggregates (multiple contributions by the one *Entid* in the same row or column of the table). More detailed sensitivity information (such as the exact sensitivity values for each of the cells) can be found in the *Outcell* table in the Work subfolder.

The final section to our PROC SENSITIVITY Log window output is:

```
NOTE: There were 20 observations read from the data set WORK.SPORTS.
NOTE: The data set WORK.OUTCONSTRAINT has 416 observations and 3 variables.
NOTE: The data set WORK.OUTCELL has 117 observations and 11 variables.
NOTE: The data set WORK.OUTLARGEST has 475 observations and 5 variables.
NOTE: PROCEDURE SENSITIVITY used (Total process time):
      real time           0.24 seconds
      cpu time            0.18 seconds
```

Here we get another confirmation on the number of entries PROC SENSITIVITY recognized in our data, as well as the individual sizes of the *Outconstraint*, *Outcell*, and *Outlargest* files we asked PROC SENSITIVITY to produce (all can be conveniently found in the Work subfolder). We also are given the time SAS needed to process our PROC SENSITIVITY run (generally PROC SENSITIVITY should take little time to run).

The %SUPPRESS macro

We now move on to the Log window documentation for %SUPPRESS (a macro), which will generate the suppression pattern for our data.

The code we run in our SAS program is:

```
%SUPPRESS(INCELL=outcell,
        CONSTRAINT=outconstraint,
        CFUNCTION1=size,
        CFUNCTION2=information,
        OUTCOMPLEMENT=outcomplement,
        OUTCELL=outpattern);
```

The first thing the Log window produces is familiar:

```
143  %SUPPRESS(INCELL=outcell,
144      CONSTRAINT=outconstraint,
145      CFUNCTION1=size,
146      CFUNCTION2=information,
147      OUTCOMPLEMENT=outcomplement,
148      OUTCELL=outpattern);

--- G-CONFID System 1.07.01 developed by Statistics Canada ---
MACRO SUPPRESS Version 1.07.001
Created on 13FEB2018:16:37:59
Email: G-Confid@statcan.gc.ca
```

The first part of this output is again just a replica of the code we ran for %SUPPRESS. The second part is again just logistical information, this time regarding %SUPPRESS.

Next we see:

```
This procedure/macro is for use at Statistics Canada only.
Input Cells data set ...................: WORK.OUTCELL
Constraint data set ....................: WORK.OUTCONSTRAINT
Output Cells data set ..................: WORK.OUTPATTERN
Complements data set ...................: WORK.OUTCOMPLEMENT
Cost function for phase 1 ..............: SIZE
Cost variable for phase 1 ..............: TOTAL (default value)
Cost function for phase 2 ..............: INFORMATION
Cost variable for phase 2 ..............: TOTAL (default value)
Scale cost .............................: NONE (default value)
Debug information ......................: NO (default value)
BY Variable(s) .........................: Not specified
```

This section is equivalent to the one in the PROC SENSITIVITY run. It breaks down what %SUPPRESS identifies as the input for all of its components (whether one has been specified or not). Again, some components are mandatory while others are not (you will receive an error message if you are missing a required component).

The last section of the %SUPPRESS Log window output is the most informative:

```
Started at 16MAR2018:10:42:41

Phase 1
                                Cells         Cells                            This
Phase    Counter     CellId  Remaining      To treat              Elapse       Cell
                                    77 of         77 (  100%) 0:00:00.83
    1          1         19         44 of         77 (57.14%) 0:00:00.85 0:00:00.01
    1          2         56         36 of         77 (46.75%) 0:00:00.85 0:00:00.00
    1          3          3         19 of         77 (24.68%) 0:00:00.86 0:00:00.00
    1          4          1         13 of         77 (16.88%) 0:00:00.86 0:00:00.00
    1          5         12          0 of         77 (    0%) 0:00:00.86 0:00:00.00

Total processing time for phase 1 .....................................: 0:00:00.05

Phase 2
                                Cells         Cells                            This
Phase    Counter     CellId  Remaining      To treat              Elapse       Cell
                                    77 of         77 (  100%) 0:00:00.88
    2          1         19         57 of         77 (74.03%) 0:00:00.89 0:00:00.00
    2          2          6         27 of         77 (35.06%) 0:00:00.89 0:00:00.00
    2          3         56         19 of         77 (24.68%) 0:00:00.90 0:00:00.00
    2          4          1         13 of         77 (16.88%) 0:00:00.90 0:00:00.00
    2          5         12          0 of         77 (    0%) 0:00:00.90 0:00:00.00

Total processing time for phase 2 .....................................: 0:00:00.03

Total processing time for phases 1 and 2 ..............................: 0:00:00.11

Process completed for WORK.OUTCELL
Total processing time 0:00:02.13
Completed at 16MAR2018:10:42:43
149  run;
```

In this section, we get information concerning the suppression pattern that %SUPPRESS creates. In Phase 1, %SUPPRESS will build the best pattern given the limitations of its implementation (that is, the pattern it will build will likely not be the optimal one, as this would take an enormous amount of processing power even for relatively small tables). That being said, the pattern it builds is generally pretty good. In Phase 2, it will attempt to improve on the pattern it built by freeing up previously suppressed cells that it finds aren't needed for complementary suppression. In our case, Phase 2 didn't free up any extra cells, but in many other cases it does. In both phases, we are given the number of cells treated at each Counter measure and the time elapsed in the treatment of those cells. At the end of Phase 2, we get a processing time for both phases followed by a total processing time for the entirety of %SUPPRESS (it is important to note that while in our example %SUPPRESS ran relatively quickly, it can take much longer with large datasets, sometimes on the order of hours). It is important to know that the full suppression pattern can be found in the *Outpattern* table in the Work subfolder, as can the *Outcomplement* table. As a final note, don't be too concerned if you notice that %SUPPRESS is spending a lot of time processing at each counter; this is often the case with large microdata files dealt with in practice (patience is a virtue).

Although no warning messages appear for the previous example, warning messages are an important topic to address in the context of Log window output. In practice, and rather non-specifically, one will encounter warning messages that both require and don't require attention. For this reason, the user is invited to consult the section containing the list of 3.1 Common error and warning messages.

2.9 Influencing a suppression pattern in %SUPPRESS

To protect the confidentiality of our data, it is not enough to simply suppress sensitive cells; we must also ensure that the values of these cells cannot be mathematically deduced from other published cells. This is done by selecting a set of complementary cells to suppress at the same time as the sensitive cells. The set of cells chosen for suppression is called a "suppression pattern".

There are many possible suppression patterns for a given set of sensitive cells. The %SUPPRESS macro is used to choose a suppression pattern. There are all sorts of reasons why users may want to influence the suppression pattern. Of course, sensitive cells will always be suppressed, but the cells selected for complementary suppression can be influenced to a certain extent. We will see in the following examples how the %SUPPRESS macro can be used to "favour" the suppression or publication of certain cells.

Create a fictitious data file

Let's start by creating a fictitious data file on which to base this series of examples:

```
data Data_sup;
id=0;
do i=1 to 10;
 do j=1 to 10;
  do k=1 to 5;

   id=id+1;
   Entid=left(put(id,3.));

   if i=1 then Province="10";
   else if i=2 then Province="11";
   else if i=3 then Province="12";
   else if i=4 then Province="13";
   else if i=5 then Province="24";
   else if i=6 then Province="35";
   else if i=7 then Province="46";
   else if i=8 then Province="47";
   else if i=9 then Province="48";
   else Province="59";

   if j=1 then Industry="A";
   else if j=2 then Industry="B";
   else if j=3 then Industry="C";
   else if j=4 then Industry="D";
   else if j=5 then Industry="E";
   else if j=6 then Industry="F";
   else if j=7 then Industry="G";
   else if j=8 then Industry="H";
   else if j=9 then Industry="I";
   else Industry="J";

   Value=round(((ranuni(128009)+0.25)**3.5)*1000)+1;
   if Industry="C" and Province="59" then Value=Value+200;
   if Industry="H" and Province="24" then Value=Value+100;

      output;
    end;
  end;
end;
drop i j k id;
run;
```

*Data_sup**

	Entid	Province	Industry	Value
1	1	10	A	1505
2	2	10	A	23
3	3	10	A	29
4	4	10	A	179
5	5	10	A	253
6	6	10	B	74
7	7	10	B	73
8	8	10	B	146
9	9	10	B	22
10	10	10	B	31
11	11	10	C	42
12	12	10	C	39
13	13	10	C	42
14	14	10	C	78
15	15	10	C	739
16	16	10	D	370
17	17	10	D	1045
18	18	10	D	63
19	19	10	D	816
20	20	10	D	9
21	21	10	E	322
22	22	10	E	349

The *Data_sup* file contains more observations than what is displayed above.

It is not necessary to understand the code used to generate the data. The file contains a total of 500 observations and two dimensions: *Province* and *Industry*. *Value* is our variable of interest.

Identifying sensitive cells using PROC SENSITIVITY

Before presenting different suppression patterns using the %SUPPRESS macro, we will first identify the sensitive cells using PROC SENSITIVITY.

```
proc sensitivity
   data=Data_sup
   outcell=outcell
   outconstraint=outconstraint
   srule="pq 0.15"
   hierarchy="TOT_PROVINCE 10 11 12 13
              24 35 46 47 48 59;
              TOT_INDUSTRY A B C D E F
              G H I J;"
   ;
   id Entid;
   var Value;
   dimension Province Industry;
run;
```

*Outcell**

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Province	Industry
1	1	0	0	5	1989	-5.25	V	C	1989	10	A
2	2	0	0	5	346	-104.1	V	C	346	10	B
3	3	0	0	5	940	-12.15	V	C	940	10	C
4	4	0	0	5	2303	-285.25	V	C	2303	10	D
5	5	0	0	5	3026	-477.05	V	C	3026	10	E
6	6	0	0	5	5937	-1659.5	V	C	5937	10	F
7	7	0	0	5	5705	-2863.35	V	C	5705	10	G
8	8	0	0	5	3528	-69.15	V	C	3528	10	H
9	9	0	0	5	3114	-559.05	V	C	3114	10	I
10	10	0	0	5	880	-196.3	V	C	880	10	J
11	11	0	0	5	1233	-338.65	V	C	1233	11	A
12	12	0	0	5	2841	-281.6	V	C	2841	11	B
13	13	0	0	5	3073	-621.45	V	C	3073	11	C
14	14	0	0	5	2584	-460.75	V	C	2584	11	D
15	15	0	0	5	692	-192.5	V	C	692	11	E
16	16	0	0	5	563	14	S	C	563	11	F
17	17	0	0	5	6067	-1946.85	V	C	6067	11	G
18	18	0	0	5	2365	-428.95	V	C	2365	11	H
19	19	0	0	5	2444	22.45	S	C	2444	11	I
20	20	0	0	5	2507	-509.9	V	C	2507	11	J
21	21	0	0	5	3424	-957.5	V	C	3424	12	A
22	22	0	0	5	4382	-1347.1	V	C	4382	12	B
23	23	0	0	5	4336	-1222.55	V	C	4336	12	C

*The *Outcell* file contains more observations than what is displayed above.

Below is the cross-tabulation we want to publish with the sensitive cells shown in red. We see that there are nine sensitive cells based on the criteria we used.

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

Example 1: Without constraints

As we will see later, there are different constraints that can be placed on %SUPPRESS to modify the suppression pattern. But let's start by showing what happens when we use the macro without any constraints. In its simplest form, we simply need to specify in the macro the names of the *Outcell* and *Outconstraint* files generated by PROC SENSITIVITY and the desired name of the file that will contain the suppression pattern results (in this case, *Outpattern_1*).

```
%suppress(
   InCell       = outcell,
   Constraint   = outconstraint,
   OutCell      = outpattern_1
);
```

*Outpattern_1**

	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity
1	P	0	1	0	0	5	1989	-5.25
2	P	0	2	0	0	5	346	-104.1
3	P	0	3	0	0	5	940	-12.15
4	P	0	4	0	0	5	2303	-285.25
5	P	0	5	0	0	5	3026	-477.05
6	P	0	6	0	0	5	5937	-1659.5
7	P	0	7	0	0	5	5705	-2863.35
8	P	0	8	0	0	5	3528	-69.15
9	P	0	9	0	0	5	3114	-559.05
10	P	0	10	0	0	5	880	-196.3
11	X	11.225	11	0	0	5	1233	-338.65
12	P	0	12	0	0	5	2841	-281.6
13	P	0	13	0	0	5	3073	-621.45
14	P	0	14	0	0	5	2584	-460.75
15	P	0	15	0	0	5	692	-192.5
16	X	7	16	0	0	5	563	14
17	P	0	17	0	0	5	6067	-1946.85
18	P	0	18	0	0	5	2365	-428.95
19	X	11.225	19	0	0	5	2444	22.45
20	P	0	20	0	0	5	2507	-509.9
21	P	0	21	0	0	5	3424	-957.5
22	P	0	22	0	0	5	4382	-1347.1

*The *Outpattern_1* file contains more variables and observations than what is displayed above.

The resulting *Outpattern_1* file contains the same variables as *Outcell*, but also contains two new variables:[6] *OutStatus* and *NetVariation*.

- The *OutStatus* variable indicates whether a cell is suppressed ("X") or published ("P").
- The *NetVariation* variable represents the net variation in absolute value of the cell—required to protect sensitive cells—as calculated by the macro. All published cells have a net variation of zero, while suppressed cells (because they are sensitive or because they are chosen for complementary suppression) have a net variation different from zero.

Each time, the %SUPPRESS macro also generates summary tables such as these:

Summary of the suppression process phase 1

	Number	Value	Percent of total number of cells
All suppressed cells	15	31906	12.40
Suppressed sensitive cells	9	19690	7.44
Suppressed complements	6	12216	4.96
Cells suppressed by user	0	0	0.00
Suppressed aggregates	0	0	N/A
Published cells	106	1123886	87.60

Summary of the entire suppression process

	Number	Value	Percent of total number of cells
All suppressed cells	15	31906	12.40
Suppressed sensitive cells	9	19690	7.44
Suppressed complements	6	12216	4.96
Cells suppressed by user	0	0	0.00
Suppressed aggregates	0	0	N/A
Published cells	106	1123886	87.60

[6] The *total_old* variable also appears, but corresponds to the *total* variable of the *Outcell* file. The *total* variable of the *Outpattern* file corresponds to *TotalNoise* in *Outcell.*
This was changed in Version 1.07.002 of G-Confid to avoid confusion. Consequently, the names of the *TotalVar* and *TotalNoise* variables are maintained in both the *Outcell* and *Outpattern* files.

In this case, since we do not have a second phase, the tables for phase 1 and the entire process are identical. If there is a second phase, the second table may be different. We see that a total of 15 cells are suppressed. Nine cells are suppressed because they are sensitive and six cells are selected for complementary suppression.

The %REPORTCELLS macro can be used to produce a more visual representation of the suppression pattern. It is very easy to use: just specify the name of the dimensions and the name of the *Outpattern* file, which is the file generated by the %SUPPRESS macro:

```
%reportCells(
        colname=Industry,
        rowname=Province,
        incell=outpattern_1
);
```

Default suppression pattern

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

Our nine sensitive cells are in red, while the six complementary cells selected for suppression are in yellow. We will see in the following examples that different yellow cells can be selected to protect our red cells.

OUTCOMPLEMENT option

An interesting option with the %SUPPRESS macro is to request the *Outcomplement* output file. This file indicates the order in which cells are selected for complementary suppression and which cells are used to protect a sensitive cell. Adding this option to the %SUPPRESS macro give us the following file:

```
%suppress(
   InCell         = outcell,
   Constraint     = outconstraint,
   OutCell        = outpattern_1,
   OutComplement  = outcomp_1
);
```

Outcomp_1

	Phase	ProtectionNumber	CellId	ComplementId
1	1	1	58	42
2	1	1	58	48
3	1	1	58	52
4	1	2	71	52
5	1	2	71	53
6	1	2	71	73
7	1	2	71	91
8	1	2	71	92
9	1	3	19	11
10	1	3	19	28
11	1	3	19	29
12	1	3	19	53
13	1	3	19	58
14	1	3	19	71
15	1	3	19	73
16	1	4	16	11
17	1	4	16	91
18	1	4	16	96

To understand this file, the corresponding *CellId* numbers must be retrieved from the *Outcell* file. For example, cell 58 corresponds to province 35 and industry H. Cell 35H is therefore protected by cells 24B, 24H and 35B. In short, this table is very useful for understanding how the suppression pattern is selected.

Example 2A: Favouring suppression of certain cells

Using the %SUPPRESS macro, it is possible to influence the suppression pattern, and even to force certain cells to be suppressed or published. There are pros and cons of "favouring" versus "forcing" the suppression or publication of certain cells. We will start by focusing on how to use each method in practice. The pros and cons of each method will be discussed at the end of this series of examples (Additional notes section).

Let's begin by seeing how to favour suppression of certain cells. There may be times when you want to favour the suppression of certain cells without forcing it, for example, if the information in some cells is of lower quality. Let's assume for the purposes of our example that we would like to favour suppression of cells for industry J, except for the marginal cell that contains the subtotal for industry J.

By default, the suppression pattern is based on the *total*[7] variable. The larger the value in a cell, the more it will "cost" to suppress it. In the %SUPPRESS macro, a "cost variable" that is different from the total can be specified. So, if we want to favour suppression of the cells for industry J, we can assign these cells a very low cost and leave the other cells with their default cost (i.e., the cell total). Let's start by creating our cost variable in the *Outcell* file:

```
data outcell;
set outcell;
if Industry="J" and Province ne "TOT_PROVINCE" then cvar_J=1;
else cvar_J=Total;  /*Replace Total with TotalNoise as of version 1.07.002.*/
run;
```

Our less important cells were assigned a cost of 1.

Now that our cost variable was created in *Outcell*, it can be used in the %SUPPRESS macro:

```
%suppress(
   InCell       = outcell,
   Constraint   = outconstraint,
   OutCell      = outpattern_2A,
   cvar1        = cvar_J
);


%reportCells(
        colname=Industry,
        rowname=Province,
        incell=outpattern_2A
);
```

Favouring suppression of Ind=J

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

[7] Or *TotalNoise* in version 1.07.002 of G-Confid.

We see that 11 cells were selected for complementary suppression. Seven cells are now suppressed in column J, while no cells were suppressed in this column in our first suppression pattern. Note as well that %SUPPRESS did not select all cells in column J for suppression, only the ones that could be used to protect sensitive cells.

Example 2B: Favouring publication of certain cells

Let's now assume that we want to favour the publication of certain cells in our table (without forcing it). The idea here is to assign a higher cost to the cells we want to publish. For this example, we'll assume that we want to favour both suppression of cells for industry J and publication of cells for province 12. There are different ways to give some cells a higher cost, but it is generally recommended that the cost variable not exceed the value of a cell's smallest margin. For simplicity, we decided to assign a value of 22,000 for the cost variable of cells for province 12 (including J12). The following suppression pattern is generated:

```
data outcell;
set outcell;
if Industry="J" and
   Province not in ("TOT_PROVINCE","12")
   then cvar_J12=1;
else if Province="12" then cvar_J12=22000;
else cvar_J12=Total; /*Replace Total with
TotalNoise as of version 1.07.002.*/
run;

%suppress(
   InCell       = outcell,
   Constraint   = outconstraint,
   OutCell      = outpattern_2B,
   cvar1        = cvar_J12
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_2B
);
```

Favouring suppression of Ind=J and publication of Prov=12

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

If we look at row 12, we see that the only change is that cell I12 is now published. Cell H12 remains suppressed since it is a sensitive cell, and cell J12, although it has a high cost (22,000), was still selected for complementary suppression.

This shows that changing our cost variable this way does not guarantee that the cells we want to favour for publication will actually be published. We will see later how to force G-Confid to publish certain cells.

Example 3A: Forcing suppression of certain cells

In practice, we may absolutely want to suppress certain cells, whether or not they are sensitive. This may be the case, for example, if we know that certain cells will be suppressed anyway due to quality.

We can tell G-Confid that certain cells must be suppressed, even if they are not sensitive. To do this, we simply set the status of the cells that must be suppressed to "X" in our *Outcell* file. Let's suppose in this example that the cells of industry A and provinces 46, 47 and 48 must be suppressed. Keeping the same cost variable as in Example 2B generates the following suppression pattern:

```
data outcell_3A;
set outcell;
if Industry="A" and
   Province in ("46","47","48")
   then status="X";
run;

%suppress(
   InCell        = outcell_3A,
   Constraint    = outconstraint,
   OutCell       = outpattern_3A,
   cvar1         = cvar_J12
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_3A
);
```

Forcing the suppression of A46, A47 and A48*

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

*While keeping the same cost variable that favours the suppression of J and the publication of 12.

We see that cells A46, A47 and A48 are indeed suppressed. Cell A47 was already sensitive, so it continues to be displayed in red. The rest of the suppression pattern is still trying to favour suppression of cells in column J and publication of in row 12.

Example 3B: Forcing publication of certain cells

We may also want to force G-Confid to publish certain cells if, for example, they are considered especially important to publish. Let's suppose that in addition to forcing suppression of cells A46, A47 and A48, we also want to force publication of the cells in row 13.

As in the previous example, constraints can be added to the *Outcell* file. To force publication of a cell, it is given a status of "P". This generates the following suppression pattern:

```
data outcell_3B;
set outcell_3A;
if Province ="13" then status="P";
run;

%suppress(
   InCell        = outcell_3B,
   Constraint    = outconstraint,
   OutCell       = outpattern_3B,
   cvar1         = cvar_J12
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_3B
);
```

Forcing the suppression of A46, A47 and A48 and publication of row 13*

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

* While keeping the same cost variable that favours the suppression of J and the publication of 12.

As might be expected, no cell was selected for suppression in row 13. So here we have a suppression pattern that reflects all the following constraints:

- Favour suppression of cells in column J.
- Favour publication of cells in row 12.
- Force suppression of cells A46, A47 and A48.
- Force publication of cells in row 13.

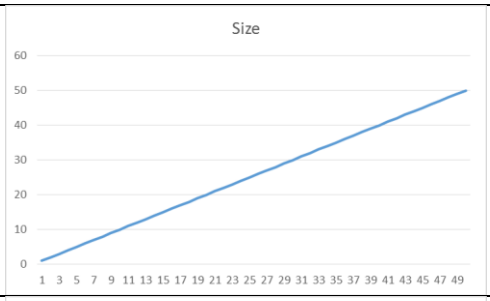
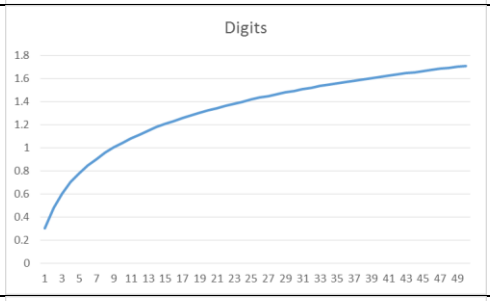
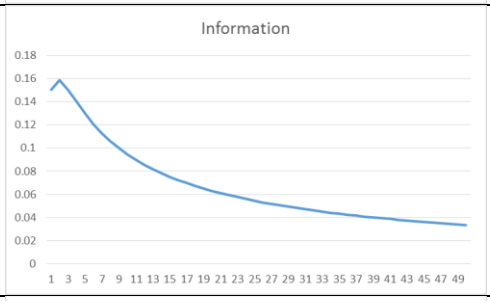
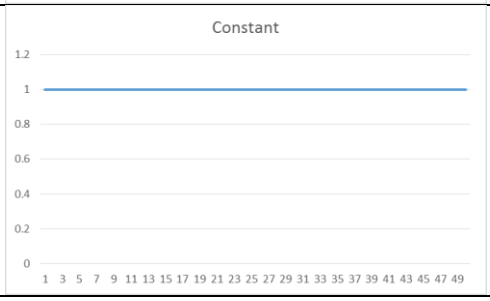
Of course, we did not need to go through examples 2A, 2B and 3A to get to this result. We simply did this to show users how a suppression pattern can be influenced by adding each of our different constraints. In practice, we generally put all our constraints together in the %SUPPRESS macro at the outset.

Finally, it should be noted that it is impossible to force the publication of a sensitive cell. If this is requested in the %SUPPRESS macro, an error message similar to the one below is displayed:

```
ERROR: SUPPRESS: Status is 'P' for a sensitive cell
```

Example 4: Cost functions

In the previous examples, we saw that the cost variable can be modified to favour the suppression or publication of certain cells. In %SUPPRESS, we also have the option of choosing a cost function. There are currently four cost functions in %SUPPRESS: SIZE, DIGITS, INFORMATION, and CONSTANT. DIGITS is the default function. Below is a summary table of our four cost functions:

Name	Definition*	Effect	Function chart
SIZE	$CVar$	A cell with a high total has a higher cost. This results in further suppressing small cells.	
DIGITS	$\log_{10}(CVar+1)$	A variant of SIZE. It also favours suppression of small cells, but tends to produce better suppression patterns than SIZE for the number of suppressed cells.	
INFORMATION	$\frac{\log_{10}(CVar+1)}{(Cvar+1)}$	A cell with a small total has a higher cost. This results in favouring publication of small cells.	
CONSTANT	1	All cells cost the same.	

**CVar* is our cost variable.

We generally use SIZE or DIGITS for phase 1 in %SUPPRESS because we want to favour suppression of small cells, and therefore give them a lower cost. In general, DIGITS should perform better than SIZE in terms of the number of suppressed cells, while SIZE should perform better than DIGITS in terms of information loss. Of course, the performance of each depends on data and constraints. This means that there is no guarantee that SIZE will always lose less information than DIGITS or that DIGITS will always suppress fewer cells than SIZE.

The INFORMATION cost function is normally used in phase 2 of the %SUPPRESS macro (we will see how it works soon). Finally, the CONSTANT cost function is not particularly useful in the context of magnitude data. It is used more with counts. Therefore, we will not discuss it in this set of examples.

Now let's compare the suppression patterns generated using the DIGITS versus SIZE cost function. We are still using the same four constraints as in Example 3B.

```
%suppress(
   InCell       = outcell_3B,
   Constraint   = outconstraint,
   OutCell      = outpattern_4A,
   cvar1        = cvar_J12,
   cfunction1   = digits
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_4A
);
```

DIGITS*

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

*Generates the same output as Example 3B because it used DIGITS implicitly

```
%suppress(
   InCell       = outcell_3B,
   Constraint   = outconstraint,
   OutCell      = outpattern_4B,
   cvar1        = cvar_J12,
   cfunction1   = size
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_4B
);
```

SIZE

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

We see that the suppression patterns are quite similar for our two cost functions. However, we observe that there are 22 cells suppressed with SIZE, while there were 21 cells suppressed with DIGITS. Adding all suppressed cells in our tables yields 56526 with the DIGITS function and 54679 with the SIZE function. As a result, one less cell was suppressed with DIGITS, but less information was suppressed with SIZE.

Example 5: Using a second phase

In practice, a second phase is generally used in %SUPPRESS to take another look at the cells selected for secondary suppression and see if it would be possible to avoid suppressing some of them. Typically, it is possible to avoid suppressing small cells. For this reason, the INFORMATION cost function, which favours small cell publication, is normally used in phase 2. Thus, small cells unnecessarily suppressed in phase 1 can be recovered in phase 2. It is also possible to specify a cost variable in phase 2, but it is recommended to keep the default cost variable, i.e. the cell total. We will now show what happens to the suppression pattern when phase 2 is added. We are going to try it with the DIGITS and SIZE cost functions so we can compare them with Example 4:

```
%suppress(
   InCell        = outcell_3B,
   Constraint    = outconstraint,
   OutCell       = outpattern_5A,
   cvar1         = cvar_J12,
   cfunction1    = digits,
   cfunction2    = Information
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_5A
);
```

DIGITS

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

```
%suppress(
   InCell        = outcell_3B,
   Constraint    = outconstraint,
   OutCell       = outpattern_5B,
   cvar1         = cvar_J12,
   cfunction1    = size,
   cfunction2    = Information
);

%reportCells(
         colname=Industry,
         rowname=Province,
         incell=outpattern_5B
);
```

SIZE

	Industry										
	A	B	C	D	E	F	G	H	I	J	TOT_INDUSTRY
Province											
10	1989	346	940	2303	3026	5937	5705	3528	3114	880	27768
11	1233	2841	3073	2584	692	563	6067	2365	2444	2507	24369
12	3424	4382	4336	1458	3302	4764	3499	1209	960	5143	32477
13	2182	1222	2527	1872	2302	926	5712	2692	2271	1672	23378
24	2837	2427	2312	1610	2649	5070	1247	3443	1987	3092	26674
35	3444	3685	2529	1046	1468	5063	3015	2361	3974	2915	29500
46	3792	2405	1749	5859	2376	4809	5567	3322	2821	1068	33768
47	1575	4770	2939	1767	1524	3460	440	3319	2281	4627	26702
48	1313	2676	3237	5689	5483	4794	2503	1348	3724	1434	32201
59	661	2897	2113	3292	1556	2980	5991	4023	3652	4946	32111
TOT_PROVINCE	22450	27651	25755	27480	24378	38366	39746	27610	27228	28284	288948

We see that using phase 2 resulted in the release of cells in both tables. In the DIGITS table, cells J11 and J35 were released. In the SIZE table, J11 and H48 were released. This example shows how it may be beneficial to use the INFORMATION cost function in phase 2.

Additional notes

Favouring or forcing suppression/publication?

One might ask whether it is preferable to force suppression/publication of a cell by modifying the *Outcell* file or whether it would be possible to simply use the cost variable to favour the suppression/publication of our cells. This decision is at the discretion of the user, but it is important to know the advantages and disadvantages of each method.

Forcing suppression/publication using an "X/P" status in *Outcell*:

- Advantages:
  - There is the certainty that the cells selected for suppression will actually be suppressed.
  - There is the certainty that the cells selected for publication will actually be published.
- Disadvantages:
  - If there are too many constraints, the %SUPPRESS macro may not find a solution for our suppression pattern.
  - You might end up with a suppression pattern that suppresses more information than necessary if cell suppression is exaggerated.

Using the cost variable:

- Advantages:
  - The %SUPPRESS macro should be able to find a solution for our suppression pattern because it is free to choose any cell for suppression.
  - Cells that cannot be used to protect others will not be unnecessarily suppressed, even if they have been given a very low cost.
- Disadvantages:
  - Cells that we may prefer to have published will not necessarily be published.
  - Same principle for suppressed cells.

Of course, regardless of the situation, it is entirely possible to run the %SUPPRESS macro several times by changing certain parameters to find different suppression patterns and to choose the one that best suits our needs.

Clarification of forced cell suppression

It is possible to force suppression of a cell using an "X" in the *Outcell* file, but this does not mean that it will be impossible to deduce this cell from the rest of the table. Indeed, if the suppression of a non-sensitive cell is forced, there is no obligation on G-Confid to protect that cell. For a cell to be protected, it must have positive sensitivity.

Examples 4 and 5 for the SIZE function show that cell A46 is suppressed but can be deduced from the rest of row 46. This means that this cell does not protect any sensitive cells and thus, theoretically, could be published without risk to the confidentiality of other cells in the table. Thus, unless there are other reasons to suppress this cell, it has been suppressed "for nothing" because its suppression does not protect any sensitive cells.

Of course, in a case where this cell would be suppressed anyway for quality reasons, for example, it would be entirely appropriate to force its suppression in the suppression pattern. At best, it may be useful at the complementary suppression stage. And, at worst, it will simply be suppressed, which was already its fate. Thus, there would be nothing to lose by forcing its suppression in this situation.

Should we use DIGITS or SIZE in phase 1 of secondary suppression?

As we saw earlier, DIGITS minimizes the number of cells chosen for suppression, while SIZE focuses on the total amount of information lost. In practice, however, this is not necessarily what is observed. Depending on the data and the constraints specified, one of the two cost functions may be more advantageous for both the number of cells suppressed and the information lost. By default, we recommend using DIGITS, but the two functions can always be tested separately and then the one that gives the best suppression pattern can be used.

Using phase 2 of the %SUPPRESS macro

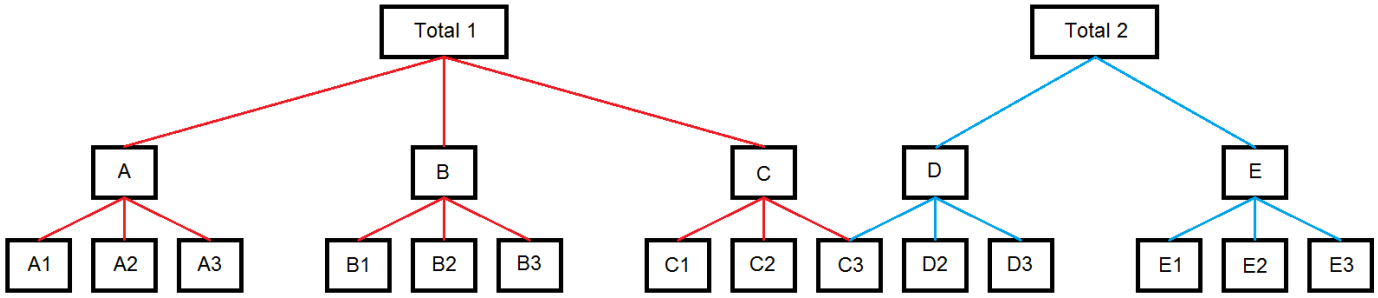
Phase 2 of the %SUPPRESS macro is used to release cells suppressed unnecessarily in phase 1. At best, it will find several cells that can be released and, at worst, it will not be able to change anything in the phase 1 suppression pattern. Therefore, its use is highly recommended regardless of the type of problem encountered.

2.10 Processing linked tables

One or more cells may be common to two tables to be published. In such a situation, it is important to know what to do in G-Confid to ensure the confidentiality of the sensitive cells in both tables. We will therefore present two different examples and explain how to approach them.

Example 1

The first problem we will present has two dimensions: region and domain. The domain has a particular hierarchy in which any cell in the category C3 is common to both of our tables. Everything is shown in the diagram below. The hierarchy of Table 1 is shown in red, while the hierarchy of Table 2 is shown in blue. If we try to enter the hierarchy as is in G-Confid, we may get an error message because there are two totals and G-Confid accepts only one total.



Before starting: The importance of accounting for the link between the tables

Before showing specifically *how* to account for the links between our two tables, let's show *why* it is important to take them into account. Let's assume an inexperienced user of G-Confid decides to run G-Confid separately for each table without taking into account the link between the two. For each table, the user runs PROC SENSITIVITY, then %SUPPRESS, and finally %REPORTCELLS to display the results. The following tables are generated:

	Domain												
	A	A1	A2	A3	B	B1	B2	B3	C	C1	C2	C3	TOT1
Region													
1	8099	3722	1233	3144	11586	5372	4091	2123	9200	1322	4723	3155	28885
2	9907	2463	4561	2883	10855	5804	2842	2209	4469	1335	1563	1571	25231
3	7719	1989	3047	2683	10766	3053	4093	3620	14152	5734	2461	5957	32637
4	11711	5651	2686	3374	8598	3067	2637	2894	5595	827	1002	3766	25904
5	8153	4777	1862	1514	6840	2896	2084	1860	4648	1468	1281	1899	19641
REGION	45589	18602	13389	13598	48645	20192	15747	12706	38064	10686	11030	16348	132298

	Domain								
	C3	D	D2	D3	E	E1	E2	E3	TOT2
Region									
1	3155	7937	2445	2337	6943	1635	1255	4053	14880
2	1571	6724	3188	1965	12037	4173	4079	3785	18761
3	5957	13316	5045	2314	8159	3498	2063	2598	21475
4	3766	9460	3223	2471	6919	300	1387	5232	16379
5	1899	10910	3366	5645	8178	4820	1892	1466	19088
REGION	16348	48347	17267	14732	42236	14426	10676	17134	90583

We see the sensitive cells in red and the cells selected by the %SUPPRESS macro for secondary suppression in yellow. We see that column C3 exists in both tables. In particular, in this column, the cell for region 5 is identified as sensitive. Since the tables were processed separately, the %SUPPRESS macro did not select the same cells in column C3 for secondary suppression. In the first table, the cell for region 4 was selected, while the cell for region 2 was selected in the second table. Thus, even though the value for region 4 in C3 is suppressed in Table 1, we can deduce from Table 2 that its value is 3766. This means that it is possible to accurately guess the cell value for region 5 in C3. The result is an exact disclosure!

This example shows that you need to be very careful when dealing with linked tables. In this case, it was possible to deduce the exact value of a sensitive cell but, sometimes, it is also possible to unintentionally create partial disclosures if the link between tables is not taken into account.

Create the fictitious data file

Let's start by creating the fictitious data file. As with the other problems presented in this document, it is not necessary to understand the code used to generate our fictitious data.

```
data Data_ex1;
id=0;
do i=1 to 5;
  do j=1 to 14;
    do k=1 to 5;
      id=id+1;
      Entid=left(put(id,3.));
      if i=1 then Region="1";
      else if i=2 then Region="2";
      else if i=3 then Region="3";
      else if i=4 then Region="4";
      else Region="5";

      if j=1 then Domain="A1";
      else if j=2 then Domain="A2";
      else if j=3 then Domain="A3";
      else if j=4 then Domain="B1";
      else if j=5 then Domain="B2";
      else if j=6 then Domain="B3";
      else if j=7 then Domain="C1";
      else if j=8 then Domain="C2";
      else if j=9 then Domain="C3";
      else if j=10 then Domain="D2";
      else if j=11 then Domain="D3";
      else if j=12 then Domain="E1";
      else if j=13 then Domain="E2";
      else Domain="E3";

      Value=round(((ranuni(7018)+0.25)**4)*1000)+1;

      output;
    end;
  end;
end;
drop i j k id;
run;
```

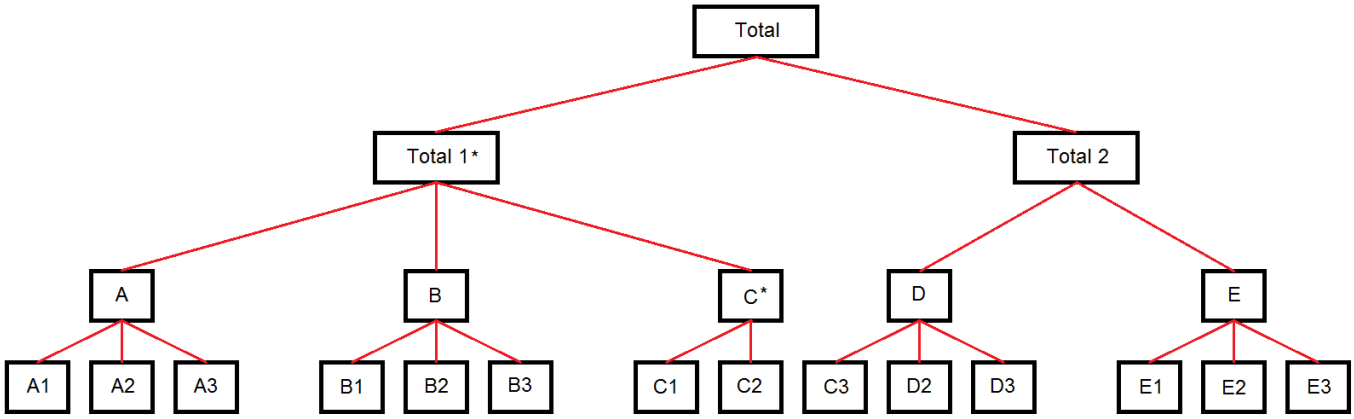
*Data_ex1**

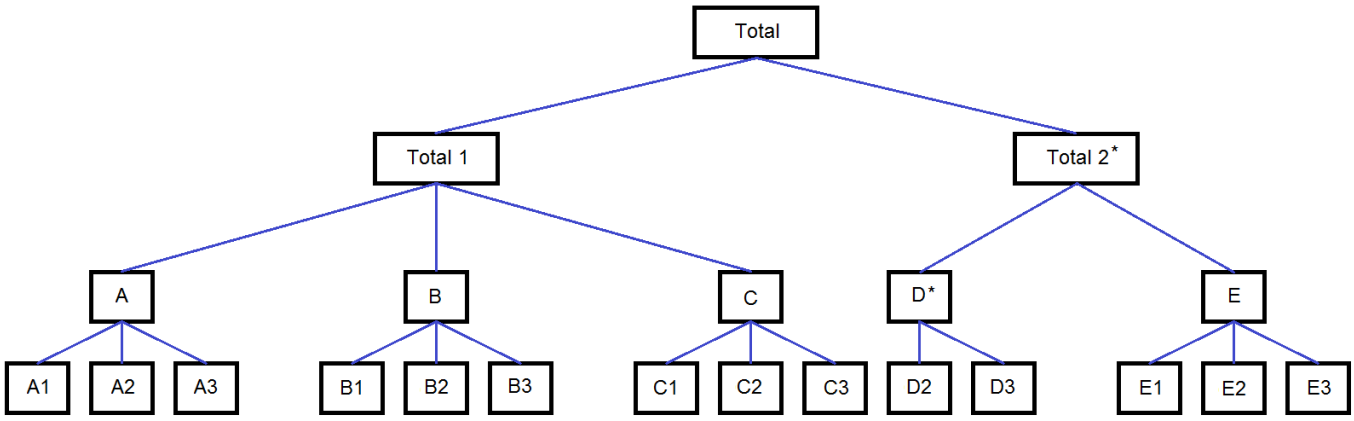
	Entid	Region	Domain	Value
1	1	1	A1	11
2	2	1	A1	13
3	3	1	A1	2194
4	4	1	A1	255
5	5	1	A1	1249
6	6	1	A2	167
7	7	1	A2	595
8	8	1	A2	194
9	9	1	A2	247
10	10	1	A2	30
11	11	1	A3	2273
12	12	1	A3	359
13	13	1	A3	20
14	14	1	A3	320
15	15	1	A3	172
16	16	1	B1	841
17	17	1	B1	38
18	18	1	B1	1915
19	19	1	B1	1978
20	20	1	B1	600
21	21	1	B2	8
22	22	1	B2	1113
23	23	1	B2	1958

*The *Data_ex1* file contains more observations than what is displayed above.

Strategy

Now that we have our data file, let's discuss the strategy needed to address this issue. As mentioned earlier, G-Confid does not accept two totals in a hierarchy. For the hierarchy to be accepted, there must be only one total that represents the sum of all children at the lowest hierarchical level. The following two hierarchies, which have been slightly modified from the initial diagram, are therefore acceptable in G-Confid:





We see that, in each case, a fictitious grand total was created along with fictitious subtotals (Total 1*, C*, Total 2*, D*). This allows us to obtain a grand total that is the sum of all children at the lowest hierarchical level.

Knowing that both hierarchies are acceptable, which one should be included in PROC SENSITIVITY? Both! Remember that G-Confid accepts multiple decompositions. If we choose to input only the first hierarchy, we will not have calculated the sensitivity for "Total 1" or "C". And if we choose only the second hierarchy, we will not have the sensitivity for "Total 2" and "D". Thus, to ensure the confidentiality of each cell we intend to publish, we must write a hierarchy with multiple decompositions.

Writing the hierarchy in PROC SENSITIVITY

Now that we have a strategy, the hard part is done. The next step is to translate the previous diagrams into an understandable hierarchy for G-Confid. This involves translating each line in the diagrams into a code understandable by G-Confid. If we take the time to describe each of the links between our cells, we get the following hierarchy:

```
%let myhierarchy=
 "
 REGION 1 2 3 4 5;

 TOTAL TOT1s TOT2:
 TOTAL TOT1 TOT2s:

 TOT1 A B C:
 TOT2 D E:
 TOT1s A B Cs:
 TOT2s Ds E:

 A A1 A2 A3:
 B B1 B2 B3:
 C C1 C2 C3:
 D C3 D2 D3:
 E E1 E2 E3:

 Cs C1 C2:
 Ds D2 D3;
"
;
```

For example, the line “TOTAL TOT1s TOT2” line indicates that Total = Total1*+Total2. Right below, we have “TOTAL TOT1 TOT2s”, which indicates that Total= Total1+Total2*. The same principle applies to all links between cells in our two diagrams. Once the hierarchy is entered in the macro variable *myhierarchy*, it is simply a case of referencing this macro variable in PROC SENSITIVITY:

```
proc sensitivity
   data=Data_ex1
   outcell=outcell_ex1
   outconstraint=outconstraint_ex1
   srule="pq 0.2"
   hierarchy=&myhierarchy
   ;
   id Entid;
   var Value;
   dimension Region Domain;
run;
```

*Outcell_ex1**

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Region	Domain
1	1	0	0	5	3722	159.8	S	C	3722	1	A1
2	2	0	0	5	1233	-272	V	C	1233	1	A2
3	3	0	0	5	3144	-57.4	V	C	3144	1	A3
4	4	0	0	5	5372	-1083.4	V	C	5372	1	B1
5	5	0	0	5	4091	-628.4	V	C	4091	1	B2
6	6	0	0	5	2123	258	S	C	2123	1	B3
7	7	0	0	5	1322	-122.6	V	C	1322	1	C1
8	8	0	0	5	4723	-725.6	V	C	4723	1	C2
9	9	0	0	5	3155	-269	V	C	3155	1	C3
10	10	0	0	5	2445	-516	V	C	2445	1	D2
11	11	0	0	5	2337	292.8	S	C	2337	1	D3
12	12	0	0	5	1635	-448.2	V	C	1635	1	E1
13	13	0	0	5	1255	62.6	S	C	1255	1	E2
14	14	0	0	5	4053	-236	V	C	4053	1	E3
15	15	0	0	5	2463	72	S	C	2463	2	A1
16	16	0	0	5	4561	-507.6	V	C	4561	2	A2
17	17	0	0	5	2883	-142	V	C	2883	2	A3

*The O*utcell_ex1* file contains more observations than what is displayed above.

Now that we have identified our sensitive cells, we can run the %SUPPRESS macro to select a suppression pattern. This gives the following suppression pattern:

```
%suppress(
   InCell        = outcell_ex1,
   Constraint    = outconstraint_ex1,
   OutCell       = outpattern_ex1,
   Cfunction1    = digits,
   Cfunction2    = information
);

%reportCells(
         colname=Domain,
         rowname=Region,
         incell=outpattern_ex1
);
```

	Domain																								
	A	A1	A2	A3	B	B1	B2	B3	C	C1	C2	C3	Cs	D	D2	D3	Ds	E	E1	E2	E3	TOT1	TOT1s	TOT2	TOT2s	TOTAL
Region																										
1	8099	3722	1233	3144	11586	5372	4091	2123	9200	1322	4723	3155	6045	7937	2445	2337	4782	6943	1635	1255	4053	28885	25730	14880	11725	40610
2	9907	2463	4561	2883	10855	5804	2842	2209	4469	1335	1563	1571	2898	6724	3188	1965	5153	12037	4173	4079	3785	25231	23660	18761	17190	42421
3	7719	1989	3047	2683	10766	3053	4093	3620	14152	5734	2461	5957	8195	13316	5045	2314	7359	8159	3498	2063	2598	32637	26680	21475	15518	48155
4	11711	5651	2686	3374	8598	3067	2637	2894	5595	827	1002	3766	1829	9460	3223	2471	5694	6919	300	1387	5232	25904	22138	16379	12613	38517
5	8153	4777	1862	1514	6840	2896	2084	1860	4648	1468	1281	1899	2749	10910	3366	5645	9011	8178	4820	1892	1466	19641	17742	19088	17189	36830
REGION	45589	18602	13389	13598	48645	20192	15747	12706	38064	10686	11030	16348	21716	48347	17267	14732	31999	42236	14426	10676	17134	132298	115950	90583	74235	206533

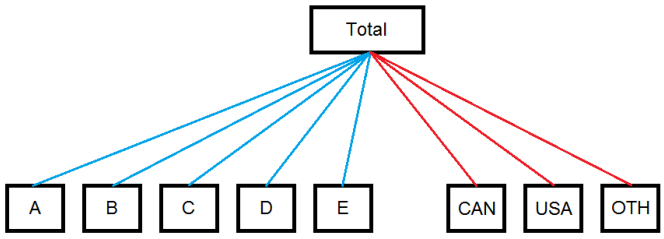
We see that the cells in column C3 have a suppression pattern that is consistent in both tables we are publishing.

Example 2

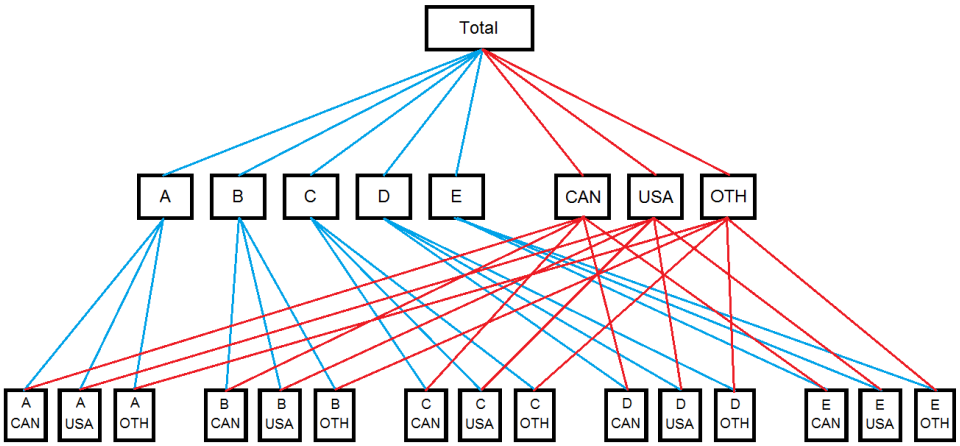
Let's move on to our second example. Let's assume that a questionnaire collects the total sales revenue of a business by type of sale (A, B, C, D, E) and by destination (CAN, USA, OTH). The data are presented as follows:

	Entid	Type_A	Type_B	Type_C	Type_D	Type_E	Pays_CAN	Pays_USA	Pays_OTH	Total
1	10	10060	0	0	37520	7	32907	0	14680	47587
2	11	15	0	0	1908	23	157	1789	0	1946
3	12	5764	9	0	14003	0	19041	713	22	19776
4	13	13450	587	0	2	5387	19415	0	11	19426
5	14	2333	0	1	0	94	2426	0	2	2428
6	15	412	0	0	10	458	869	11	0	880
7	16	203360	-4787380	5060	2365660	0	-2215831	2360	171	-2213300
8	17	1	195	850	2844	4	3559	332	3	3894
9	18	5010	14444	0	0	1	19265	0	190	19455
10	19	3661	0	4238	0	519	8261	154	3	8418
11	20	0	76	183	5912	0	6156	15	0	6171
12	21	0	0	0	262017	0	262017	0	0	262017
13	22	38759	313	0	0	2558	41525	0	105	41630
14	23	177706	200005	0	0	6	377154	534	29	377717
15	24	2	48	6437	876	2838	9420	763	18	10201
16	25	34923	134616	0	8308	6	177851	0	2	177853
17	26	8411	0	0	41	1	8453	0	0	8453
18	27	0	3	62398	36	7145	69512	70	0	69582
19	28	19800	13	0	0	14	18285	1537	5	19827
20	29	111	247	0	0	32	370	0	20	390

We see that there are two ways to obtain a company's total sales: by adding the amounts by type or by destination. The hierarchy would look something like this:



Remember that G-Confid accepts that there are different ways to calculate the total provided the total can be defined as a sum of all children at the bottom of the hierarchy. To make this possible in this example, we would need to know the details of the amounts when cross-tabulating the type of sale with the destination, as shown in the following diagram:



However, based on our data file, we do not have the values needed to cross-tabulate between the two variables. This leaves us with two options to solve this problem in G-Confid:

1. Run two jobs in G-Confid and take into account the link between them
2. Impute the values of the cross-tabulation between the two variables, which is called “data completion”

In this example, we will show how to solve the problem using two jobs. For users who would like to try using data completion, we recommend that you contact the G-Confid team.

Generating the data

Let's start by generating the fictitious data file we presented above.

```
data Data_ex2;
id=9;
do i=1 to 20;
  id=id+1;
  Entid=left(put(id,3.));

  Type_A=round(((ranuni(123)+0.25)**15)*10000);
  Type_B=round(((ranuni(676)+0.25)**25)*2000);
  Type_C=round(((ranuni(797)+0.25)**30)*200);
  Type_D=round(((ranuni(980)+0.25)**25)*1000);
  Type_E=round(((ranuni(286)+0.25)**10)*1000);

  Pays_CAN=.;
  Pays_USA=round(((ranuni(145)+0.25)**10)*250);
  Pays_OTH=round(((ranuni(107)+0.25)**15)*100);
  Pays_CAN=(Type_A+Type_B+Type_C+Type_D+Type_E)-(Pays_OTH+Pays_USA);

  Total=Pays_USA+Pays_OTH+Pays_CAN;

  if Entid="16" then do;
    Type_A=Type_A*20;
    Type_B=Type_B*(-20);
    Type_C=Type_C*20;
    Type_D=Type_D*20;
    Type_E=Type_E*20;
    Pays_USA=Pays_USA*20;
    Pays_OTH=171;
    Pays_CAN=(Type_A+Type_B+Type_C+Type_D+Type_E)-(Pays_OTH+Pays_USA);
    Total=Pays_USA+Pays_OTH+Pays_CAN;
  end;

  if Entid="10" then do;
    Pays_OTH=14680;
    Pays_CAN=32907;
  end;

  output;
end;
drop i id;
run;
```

Transforming the data into skinny format

The idea is to run one job for types of sales and one for destinations. To do this, we need to start by creating skinny data files that we will need to use later in PROC SENSITIVITY. To create the skinny files, we follow the same process shown earlier in the document: by using PROC TRANSPOSE. This gives the following files:

```
proc transpose data=Data_ex2
   out=Data_ex2_type
        (rename=(_NAME_=Var_Type col1=Value));
   var Type_A Type_B Type_C Type_D Type_E;
   by ENTID;
run;

proc transpose data=Data_ex2
   out=Data_ex2_pays
        (rename=(_NAME_=Var_Pays col1=Value));
   var Pays_USA Pays_OTH Pays_CAN;
   by ENTID;
run;
```

Data_ex2_type

	Entid	NAME OF FORMER VARIABLE	Value
1	10	Type_A	10060
2	10	Type_B	0
3	10	Type_C	0
4	10	Type_D	37520
5	10	Type_E	7
6	11	Type_A	15
7	11	Type_B	0
8	11	Type_C	0
9	11	Type_D	1908
10	11	Type_E	23
11	12	Type_A	5764
12	12	Type_B	9
13	12	Type_C	0
14	12	Type_D	14003
15	12	Type_E	0
16	13	Type_A	13450
17	13	Type_B	587

Data_ex2_pays

	Entid	NAME OF FORMER VARIABLE	Value
1	10	Pays_USA	0
2	10	Pays_OTH	14680
3	10	Pays_CAN	32907
4	11	Pays_USA	1789
5	11	Pays_OTH	0
6	11	Pays_CAN	157
7	12	Pays_USA	713
8	12	Pays_OTH	22
9	12	Pays_CAN	19041
10	13	Pays_USA	0
11	13	Pays_OTH	11
12	13	Pays_CAN	19415
13	14	Pays_USA	0
14	14	Pays_OTH	2
15	14	Pays_CAN	2426
16	15	Pays_USA	11
17	15	Pays_OTH	0

Job 1: Type variable

We will start by identifying the sensitive cells for the type variable. We do this by running PROC SENSITIVITY and %SUPPRESS as we would normally. Note that the ACCEPTNEGATIVE and ADDITIVENOISE options were used to account for negative values in the data.

```
proc sensitivity
   data=Data_ex2_type
   outcell=outcell_ex2_type
   outconstraint=outconstraint_ex2_type
   srule="pq 0.2"
   hierarchy="Total Type_A Type_B
        Type_C Type_D Type_E;"
   acceptnegative
   additivenoise
   ;
   id Entid;
   var Value;
   dimension Var_Type;
run;

%suppress(
   InCell         = outcell_ex2_type,
   Constraint     = outconstraint_ex2_type,
   OutCell        = outpattern_ex2_type,
   OutComplement  = outcomp_ex2_type,
   Cfunction1     = digits,
   Cfunction2     = information
);
```

The end result is the *Outpattern* file below. We see that several cells were sensitive, including the total. We also have two sensitive aggregates. Only Type_A and Type_E cells are published.

Outpattern_ex2_type

	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity	Status	Type	total	Var_Type
1	P	0	1	0	0	20	523778	-102040	V	C	523778	Type_A
2	X	403462.5	2	0	0	20	-4436824	806925	S	C	5137936	Type_B
3	X	1073.8	3	0	0	20	79167	2147.6	S	C	79167	Type_C
4	X	467748.5	4	0	0	20	2699137	401672	S	C	2699137	Type_D
5	P	0	5	0	0	20	19093	-5132	V	C	19093	Type_E
6	X	467748.5	6	0	0	20	-1115649	752358	S	C	8459111	Total
7	X	467748.5	7	0	0	20	-1658520	935497	S	A	7916240	
8	X	467748.5	8	0	0	20	-1134742	771445	S	A	8440018	

The other interesting file that is generated at the same time by the %SUPPRESS macro is *Outcomplement*. This file tells us the order in which the cells are selected for suppression and which cells protect each other. The table shows us the suppressed cell circuits step by step.

Outcomp_ex2_type

	Phase	ProtectionNumber	CellId	ComplementId
1	1	1	7	3
2	1	1	7	4
3	1	1	7	6
4	1	1	7	8
5	1	2	2	6
6	1	2	2	7
7	1	2	2	8
8	2	1	7	4
9	2	1	7	6
10	2	1	7	8
11	2	2	2	4
12	2	3	3	4

Phase 1: Protection of aggregate 7

Phase 1: Protection of *CellId*=2 (Type_B)

Phase 2: We attempt to release cells suppressed in phase 1, without success.

The first phase in this example selects two cell circuits for suppression, represented by the *ProtectionNumber* variable. The first circuit is to protect aggregate 7 because it is the most sensitive cell in the *Outcell* file. The second circuit is to protect *CellId*=2 (the Type_B cell) because, at this stage, all other sensitive cells have already been suppressed via circuit 1.

Phase 2 attempts to see if it is possible to release cells suppressed in phase 1. Sometimes, we realize that it is possible to rearrange circuits to suppress fewer cells while protecting sensitive cells, but in this case, rearranging the circuits did not release cells. We are left with three circuits of suppressed cells:

- Aggregate 7, Type_D, Total, Aggregate 8 (*CellId*=7,4,6,8)
- Type_B, Type_D (*CellId*=2,4)
- Type_C, Type_D (*CellId*=3,4)

Thus, suppression of the “Total” cell is important to protect the total, but also to protect all other cells in the same circuit because they are also sensitive. It is important to keep this in mind for the rest of the example.

Job 2: Country variable

Let's now move on to identifying sensitive cells for the country variable. We begin by running PROC SENSITIVITY:

```
proc sensitivity
   data=Data_ex2_pays
   outcell=outcell_ex2_pays
   outconstraint=outconstraint_ex2_pays
   srule="pq 0.2"
   hierarchy="Total Pays_USA
         Pays_OTH Pays_CAN;"
   acceptnegative
   additivenoise
   ;
   id Entid;
   var Value;
   dimension Var_Pays;
run;
```

Outcell_ex2_pays

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Var_Pays
1	1	0	0	20	8278	-3657	V	C	8278	Pays_USA
2	2	0	0	20	15261	2545	S	C	15261	Pays_OTH
3	3	0	0	20	-1139188	-256322.8	V	C	3292474	Pays_CAN
4	4	0	0	20	-1115649	-276261.6	V	C	3316013	Total

First, note that, in this example, the total is not considered sensitive. It is sensitive in the table in Job 1, but not in the table in Job 2. As mentioned earlier, suppression of the total in the table in Job 1 protected the total but also protected other cells in the table that were in the same circuit. It is therefore very important to suppress the total in the second table, but also to ensure that an appropriate amount of protection is provided to protect the total in the second table. In other words, we cannot simply suppress the total in the second table: we must also find complementary cells to suppress in our second table to protect the total.

In order for G-Confid to understand that the total must absolutely be suppressed and to specify to it the amount of protection required, we must change two values in the table *Outcell_ex2_country* before sending it to the %SUPPRESS macro:

- The status of the cell must be marked as "X". This ensures that the cell is suppressed.
- The sensitivity of the cell must be changed to represent the amount of protection required by the cell. The sensitivity is replaced by two times the *NetVariation* of the corresponding cell in the first table. In this case, the total had a *NetVariation*=467748.5 in the Job 1 *Outpattern* file. This value is multiplied by 2 to get the sensitivity of the total in Job 2.

This gives:

```
data outcell_ex2_pays_v2;
set outcell_ex2_pays;
if CellId=4 then do;
   sensitivity=467748.5*2;
   status="X";
end;
run;
```

Outcell_ex2_pays_v2

	CellId	NbAnonym	AnonymTotal	NbRespondents	Total	Sensitivity	Status	Type	TotalNoise	Var_Pays
1	1	0	0	20	8278	-3657	V	C	8278	Pays_USA
2	2	0	0	20	15261	2545	S	C	15261	Pays_OTH
3	3	0	0	20	-1139188	-256322.8	V	C	3292474	Pays_CAN
4	4	0	0	20	-1115649	935497	X	C	3316013	Total

Let's take a moment to look at what this has changed in the fourth row of the table. The cell status is now "X" but the sensitivity has changed from -276261.6 to 935497. Two comments about sensitivity:

- Having positive sensitivity means that G-Confid will have to select cells for complementary suppression in order to protect the total. If the sensitivity was left at -276261.6, G-Confid would suppress the total cell because of the "X" under Status, but would not suppress complementary cells because negative sensitivity indicates that no

protection is required. In this case, this could result in an exact or partial disclosure of the cell. In short, we cannot leave the sensitivity at -276261.6!

- Suppression of the total protects not only the total but also other sensitive cells in the table in the same circuit in Job 1. The value of the sensitivity in the second table must therefore reflect this reality. Without going into details on how the *NetVariation* variable works, it is important to know that using this variable allows us to account for the protection required by suppressed cell circuits. That is why we have a sensitivity of 935497 instead of the 752358 that the total had in Job 1. In fact, 935497 corresponds to the sensitivity of aggregate 7 (the most sensitive cell in the table in Job 1), which was in the same circuit of suppressed cells as the total.

Now that our *Outcell* file has been modified to reflect Job 1, we can proceed to the secondary suppression step:

```
%suppress(
   InCell        = outcell_ex2_pays_v2,
   Constraint    = outconstraint_ex2_pays,
   OutCell       = outpattern_ex2_pays,
   Cfunction1    = digits,
   Cfunction2    = information
);
```

Outpattern_ex2_pays

	OutStatus	NetVariation	CellId	NbAnonym	AnonymTotal	NbRespondents	total_old	Sensitivity	Status	Type	total	Var_Pays
1	P	0	1	0	0	20	8278	-3657	V	C	8278	Pays_USA
2	X	7630.5	2	0	0	20	15261	2545	S	C	15261	Pays_OTH
3	X	460118	3	0	0	20	-1139188	-256322.8	V	C	3292474	Pays_CAN
4	X	467748.5	4	0	0	20	-1115649	935497	X	C	3316013	Total

Looking at *OutStatus*, we can see that all cells in this table have been suppressed except “Country_USA”. This means that the total has been suppressed and it is not possible to deduct it accurately from this table. Therefore, both of our tables (the one for type and the one for country) are well protected.

To conclude this example, if the jobs had been run in reverse order (country first, then type), it would have been necessary to run the country job again to protect the new sensitive cell found. In practice, when we have several jobs in G-Confid because of linked tables, they must be run in a loop until each sensitive cell is properly protected in all tables in which it appears.

2.11 Additive controlled rounding

G-Confid offers two modules that round tables of cells so that all segments (rows, columns) are additive; control is maintained by avoiding rounding more than one multiple of the base beyond the original value. The module %GCONFIDROUND uses an iterative approach that remained consistent with Statistics Canada methodology (Boudreau, Filep and Liu, 2004). The module %GCONFIDOPTROUND leverages the functionality of PROC OPTMODEL in SAS to find an optimal solution. For this reason, only an example using the latter module is provided in this section. Note that the macro %GCONFIDOPTROUND is available as of the 1.07.002 G-Confid version.

Creation of a fictitious data file

Suppose we have a table containing frequencies by industry (1,2) and region (A, B, C).

```
data Ex_data;
ID='01'; Industry="1"; Region="A"; Freq=35; output;
ID='02'; Industry="1"; Region="B"; Freq=3; output;
ID='03'; Industry="1"; Region="C"; Freq=5; output;
ID='04'; Industry="2"; Region="A"; Freq=6; output;
ID='05'; Industry="2"; Region="B"; Freq=3; output;
ID='06'; Industry="2"; Region="C"; Freq=9; output;
run;
```

Ex_data

	ID	Industry	Region	Freq
1	01	1	A	35
2	02	1	B	3
3	03	1	C	5
4	04	2	A	6
5	05	2	B	3
6	06	2	C	9

We hence have for example a frequency of 35 enterprises in industry 1 and region A.

PROC SENSITIVITY call

Because these are frequencies, we can use rounding to protect the data. To do this, we will first need to create some input data files that will be used in the macro %GCONFIDOPTROUND. It is possible to create these files by hand, but that can become tedious in the presence of complex or big data files. We will therefore show here how to use PROC SENSITIVITY to create the wanted files more easily.

```
proc sensitivity
   data=Ex_data
   outcell=outcell
   outconstraint=outconstraint
   srule="arb -1"
   hierarchy="TOT_INDUSTRY 1 2;
              TOT_REGION A B C;"
   ;
   id ID;
   var Freq;
   dimension Industry Region;
run;
```

Outcell

	CellId	NbAnonym	AnonymTotalVar	NbRespondents	TotalVar	Sensitivity	Status	Type	TotalNoise	Industry	Region
1	1	0	0	1	35	-35	V	C	35	1	A
2	2	0	0	1	3	-3	V	C	3	1	B
3	3	0	0	1	5	-5	V	C	5	1	C
4	4	0	0	1	6	-6	V	C	6	2	A
5	5	0	0	1	3	-3	V	C	3	2	B
6	6	0	0	1	9	-9	V	C	9	2	C
7	7	0	0	6	61	-61	V	C	61	TOT_INDUSTRY	TOT_REGION
8	8	0	0	3	43	-43	V	C	43	1	TOT_REGION
9	9	0	0	3	18	-18	V	C	18	2	TOT_REGION
10	10	0	0	2	41	-41	V	C	41	TOT_INDUSTRY	A
11	11	0	0	2	6	-6	V	C	6	TOT_INDUSTRY	B
12	12	0	0	2	14	-14	V	C	14	TOT_INDUSTRY	C

Outconstraint

	ConstraintId	CellId	Coefficient
1	1	8	1
2	1	9	1
3	1	7	-1
4	2	10	1
5	2	11	1
6	2	12	1
7	2	7	-1
8	3	1	1
9	3	4	1
10	3	10	-1
11	4	2	1
12	4	5	1
13	4	11	-1
14	5	3	1
15	5	6	1
16	5	12	-1
17	6	1	1
18	6	2	1
19	6	3	1
20	6	8	-1
21	7	4	1
22	7	5	1
23	7	6	1
24	7	9	-1

Before going further, note that any sensitivity rule could have been used here, since we don't use PROC SENSITIVITY to evaluate the sensitivity. It's only the structure of the frequency table that we are interested in (the linear constraints and the total of each cell, including the marginal cells).

Creation of the input files

We will now use the files provided by PROC SENSITIVITY to produce the input files required by %GCONFIDOPTROUND.

First, we need to identify the constraints from the *Outconstraint* file. In this case, we only have 7 constraints:

```
data outconstraint2;
  set outconstraint;
  where (coefficient eq -1);
  keep constraintid;
run;
```

Outconstraint2

	ConstraintId
1	1
2	2
3	3
4	4
5	5
6	6
7	7

Next, we update the *Outcell* file. We include a *weight* variable with a default of 1, and we force the result of *CellId*=1 to be rounded to at least 40 in the variable *CellLB*. We must also specify the variable *Total*, which is equivalent to *TotalVar*. Only the relevant variables for the rounding are kept here in the data file (we see that the sensitivity is not retained, as mentioned earlier).

```
data outcell2;
set outcell;
weight=1;
if (cellid eq 1) then CellLB=40;
rename TotalVar=Total;
keep cellId TotalVar weight cellLB;
run
```

Outcell2

	CellId	Total	weight	CellLB
1	1	35	1	40
2	2	3	1	.
3	3	5	1	.
4	4	6	1	.
5	5	3	1	.
6	6	9	1	.
7	7	61	1	.
8	8	43	1	.
9	9	18	1	.
10	10	41	1	.
11	11	6	1	.
12	12	14	1	.

%GCONFIDOPTROUND results

Finally, we use the %GCONFIDOPTROUND macro. We need to specify in the macro the names of the *Outconstraint* file and the two updated files (*Outcell2*, *Outconstraint2*). We also need to specify the rounding base (here it is 5) and the name of the file where we want the macro to put the results (named *Outround*). Other parameters could have been included in the macro, but they are not shown here to keep the example simple. See 5.7 The %GCONFIDOPTROUND macro section for more information on the available parameters.

```
%GCONFIDOPTROUND(
  InCells=outcell2,
  InConstraints=outconstraint2,
  InCellConstraints=outconstraint,
  OutCells=outround,
  base=5
  );
```

Outround

	Cell Identification	Upper Residual	Positive Shift Indicator	Positive Base Shift	Positive Shift	Lower Residual	Negative Shift Indicator	Negative Base Shift	Negative Shift	Residual Indicator	Shift	Absolute Shift	Cell Lower Bound	Cell Upper Bound	Weight	Total	Rounded Total
1	1	0	1	5	5	0	0	0	0	0	5	5	40	.	1	35	40
2	2	2	0	0	0	3	1	0	3	1	-3	3	.	.	1	3	0
3	3	0	1	0	0	0	0	0	0	0	0	0	.	.	1	5	5
4	4	4	0	0	0	1	1	5	6	1	-6	6	.	.	1	6	0
5	5	2	1	0	2	3	0	0	0	1	2	2	.	.	1	3	5
6	6	1	1	0	1	4	0	0	0	1	1	1	.	.	1	9	10
7	7	4	0	0	0	1	1	0	1	1	-1	1	.	.	1	61	60
8	8	2	1	0	2	3	0	0	0	1	2	2	.	.	1	43	45
9	9	2	0	0	0	3	1	0	3	1	-3	3	.	.	1	18	15
10	10	4	0	0	0	1	1	0	1	1	-1	1	.	.	1	41	40
11	11	4	0	0	0	1	1	0	1	1	-1	1	.	.	1	6	5
12	12	1	1	0	1	4	0	0	0	1	1	1	.	.	1	14	15

The *Outround* table contains several variables. We can see the original total and the rounded one in the two last columns of the table. The variable *Shift* simply shows by how much the original value was changed due to rounding.

The variable *CellId* in the *Outcell* file tells us where each value should go in the tables. We therefore obtain the following "before" and "after" tables. We can see in the rounded table that the cell in *Region*='A', *Industry*=1 has a value of 40 and that every cell in the table has a value that is a multiple of 5.

CellId

	Industry		Total
Region	1	2	
A	"1"	"4"	"10"
B	"2"	"5"	"11"
C	"3"	"6"	"12"
Total	"8"	"9"	"7"

Original table

	Industry		Total
Region	1	2	
A	35	6	41
B	3	3	6
C	5	9	14
Total	43	18	61

Final table after rounding

	Industry		Total
Region	1	2	
A	40	0	40
B	0	5	5
C	5	10	15
Total	45	15	60

3. Details

The purpose of this section is to provide users with additional information that might be useful in understanding certain concepts associated with G-Confid and the protection of tabular data. We recommend that users refer to this section as needed, but it is not necessary to read the entire section.

3.1 Common error and warning messages

G-Confid users often come across certain error and warning messages. We will try to list a few of them and provide a possible solution for each one. This is a partial list only, and we welcome any suggestions regarding messages that should be added.

Omission of a statement or a mandatory option

Certain statements and options are mandatory in PROC SENSITIVITY. Forgetting to specify them can result in the following error messages:

Omitted statement/option	Error message
VAR	`ERROR: VAR is mandatory.`
ID	`ERROR: ID is mandatory.`
DIMENSION	`ERROR: The number of dimensions (2) in HIERARCHY statement does not match the number of variables (0) in the DIMENSION statement.` `ERROR: DIMENSION is mandatory.`
HIERARCHY	`ERROR: HIERARCHY is mandatory.`
SRULE	`ERROR: SRULE is mandatory.`

DATA	`301  proc sensitivity`
	`302     outcell=outcell`
	`303     outconstraint=outconstraint`
	`304     srule="pq 0.15"`
	`305     hierarchy="TOT_REGION A B; Industry 1 2;"`
	`306     ;`
	`307     id Entid;`
	`ERROR: Variable ENTID not found.`
	`308     var Profit;`
	`ERROR: Variable PROFIT not found.`
	`309     dimension Region Industry;`
	`ERROR: Variable REGION not found.`
	`ERROR: Variable INDUSTRY not found.`
	`310  run;`

The VAR, ID, DIMENSION, HIERARCHY, SRULE and DATA options and statements must be specified in order for PROC SENSITIVITY to work.

Incorrect data filename or variable

If the user makes a mistake in the name of the data file or the variable, the following error messages may appear. The fix is to correct the name of the data file or the variable.

`ERROR: File WORK.MYDATA2.DATA does not exist.`
`ERROR: Variable PROVINCE not found.`

Using the wrong type of variable

Some variables must be either numeric or character-based in order for PROC SENSITIVITY to accept them. If the wrong type of variable is entered, the result is a message such as the following:

```
ERROR: Variable Profit in list does not match type prescribed for this list.
```

To determine which type of variable to use, see the 5. Syntax section.

Using an invalid sensitivity rule

In the SRULE option, the user must specify a valid sensitivity rule. If the rule is deemed invalid by PROC SENSITIVITY, an error message beginning with "ERROR: Sensitivity rule parser:" is displayed in the SAS log. The user will also see a warning message when attempting to use a Duffett rule since it is no longer recommended (we normally recommend using the C2 rule at Statistics Canada). Using the NK rule with a parameter of $k < 50$ also generates a warning message, because it does not make sense to use such a parameter for an NK rule and it could lead to unexpected results.

```
ERROR: Sensitivity rule parser: Invalid PQ parameter (15.000000). It must be strictly between 0 and 1.
ERROR: Sensitivity rule parser: Invalid NK parameter (80). N must be an integer between 1 and 5 inclusively.
ERROR: Sensitivity rule parser: NK needs at least 2 parameters.
ERROR: Sensitivity rule parser: NK needs a maximum of 3 pairs of parameters.
ERROR: Sensitivity rule parser: NK needs an even number of parameters.
ERROR: When using the NK rule with the WEIGHT statement and a WEIGHTPROTLEVEL other than EXACT, a maximum of
2 nk rules can be specified.
ERROR: Sensitivity rule parser: C2 does not need parameters.
ERROR: Sensitivity rule parser: ARB needs 1 to 4 parameters.
ERROR: Sensitivity rule parser: Invalid sensitivity rule (CAKE).
WARNING: Sensitivity rule parser: Duffett is not recommended. You should use another sensitivity rule.
WARNING: Sensitivity rule parser: NK k (40) parameter is lower than 50.000000.
```

For more information on the rules that can be used, it may be helpful to refer to the sensitivity rule syntax in 5.1 PROC SENSITIVITY or examples of 2.2 Sensitivity rules.

Procedure/macro not found

The user may come across a message like this when running PROC SENSITIVITY or one of the G-Confid macros:

```
ERROR: Procedure SENSITIVITY not found.
WARNING: Apparent invocation of macro SUPPRESS not resolved.
WARNING: Apparent invocation of macro AUDIT not resolved.
```

These messages mean that SAS cannot find the SENSITIVITY procedure or the macro that the user is trying to use. In this case, we suggest checking whether the correct version of SAS was used. For example, it is possible to accidentally use "SAS 9.3" rather than "SAS 9.3 with G-Confid 1.07.002 64-bit SAS93". Another possibility is that there was a simple mistake in the name of the procedure or macro. For example, G-Confid should recognize the %SUPPRESS macro, but would not recognize the %SUPRESS macro (since it is missing a 'P').

Incorrect hierarchy

Specifying the hierarchy is one of the most complex steps in using PROC SENSITIVITY. We recommend referring to examples on 2.3 Hierarchies to fully understand hierarchies. Below are a few common errors that occur when specifying the hierarchy:

Error type	Error message
Omission of semi-colon at the end	ERROR: Hierarchy parser: Looking for 'Colon' but found 'Done' instead. Error at or before: TOT_REGION A B; Industry 1 2 ^
Hierarchy and dimension do not have the same number of variables	ERROR: The number of dimensions (2) in HIERARCHY statement does not match the number of variables (1) in the DIMENSION statement.
Multiple roots	ERROR: Hierarchy parser: The hierarchy contains multiple roots while only one is permitted.
The observation is missing a dimension	WARNING: There were 1 observations dropped from DATA data set because one of the DIMENSION variables is missing. WARNING: These observations will not be used for sensitivity calculation. WARNING: Reading microdata: A code is missing for dimension dimvar. The observation is dropped.
Values are suppressed because they do not exist in the hierarchy	WARNING: Reading microdata: Microdata file contains a code that is either a parent in the hierarchy or a code that is not in the hierarchy/range for the dimension Region, code 'B'. Only children codes will be processed WARNING: There were 7 observations dropped from DATA data set because one of the DIMENSION variables is not in the hierarchy. WARNING: These observations will not be used for sensitivity calculation.
The hierarchy and dimensions do not specify the variables in the same order	WARNING: Reading microdata: Microdata file contains a code that is either a parent in the hierarchy or a code that is not in the hierarchy/range for the dimension Industry, code '1'. Only children codes will be processed WARNING: Reading microdata: Microdata file contains a code that is either a parent in the hierarchy or a code that is not in the hierarchy/range for the dimension Industry, code '2'. Only children codes will be processed WARNING: There were 14 observations dropped from DATA data set because one of the DIMENSION variables is not in the hierarchy. WARNING: These observations will not be used for sensitivity calculation. WARNING: No valid observations in DATA data set.

It is important for the hierarchy and dimensions to match, which means that they need to have the same number of variables and the variables must be in the same order.

It should also be noted that any values not specified in the hierarchy or range are excluded from the sensitivity calculations. It is therefore important that the user indicate all the values to include in the hierarchy (or range, where applicable). It is not a problem if the hierarchy or range includes values that are not in the data file (e.g., specifying the province of Ontario even if it is not a value in the data file).

The error involving multiple roots occurs when there are somehow two (or more) grand totals in the hierarchy. G-Confid assumes that the hierarchy adds up to one grand total. This grand total may be produced in different ways, as in the case of multiple decompositions, but there can be only one grand total per dimension. Most of the time, the user's intent was not to put multiple totals, but it was interpreted that way by G-Confid because the specified hierarchy is incomplete. In complex hierarchies, it is useful to prepare a diagram that visually represents the hierarchy, in case a multiple roots error message is encountered. Most often, by having a diagram, the user becomes aware that there are missing links between certain cells, or that certain cells are completely orphaned, i.e., they are not connected at all to others in the hierarchy. In that case, the user needs to specify how these cells are connected to others, making sure not to miss any link, to solve the problem.

Missing or negative values

If there are negative or missing values, PROC SENSITIVITY displays the following warnings by default:

```
WARNING: There were 4 observations dropped from the DATA data set because the variable Profit is negative.
WARNING: These observations will not be used for sensitivity calculation.
WARNING: There were 1 observations dropped from the DATA data set because the variable value is missing.
WARNING: These observations will not be used for sensitivity calculation.
```

If a user wants G-Confid to consider negative values in the sensitivity calculations, the option ACCEPTNEGATIVE can be specified.

Using the WAIVER statement instead of the PWAIVER statement

A situation may arise where an enterprise provides us with a partial waiver, i.e., it authorizes us to disclose its information for some cells, but for not all. In such a case, the PWAIVER statement rather than the WAIVER statement must be used. The WAIVER statement displays a message such as the one below if one or more enterprises are found with a partial waiver:

```
ERROR: Contributor 'SO1' has a waiver flag for some but not all contributions. Please verify the
       consistency of the waiver flag and rerun.
ERROR: Unable to update waiver flags lists
```

However, the WAIVER statement can be used to ensure that all waivers are complete in our data set. If we know that all waivers should be complete in the data set, an error message can help to identify which enterprises contain an incorrect value for their waiver variable.

3.2 Sensitivity rules

General formula

The general formula for linear sensitivity (Cox and Sande, 1979) is described as follows:

$$S = \sum_{i=1}^{N} \alpha_i x_i,$$

Where

- S is a cell sensitivity
- N is the number of contributors in a cell
- $x_i, i = 1,2, \dots, N$ is the value of the i^{th} contributor to a cell, with contributions in decreasing order ($x_1 \geq x_2 \geq \cdots \geq x_N \geq 0$)
- $\alpha_i, i = 1,2, \dots, N$ are the standard coefficients for the sensitivity rule chosen.

Note: The PTN framework does not make use of this form of the linear sensitivity measure.

PQ rule

According to the PQ rule, a cell is considered sensitive if the second largest respondent can estimate within ±(pq)% the contribution of the first largest respondent to the cell. In fact, according to this definition, it can be shown that a cell is sensitive if and only if, after the publication of the table, it is possible for some respondent to estimate the contribution of another respondent to within ±(pq)% of its original value. The PQ rule can be expressed as follows:

$$S = (pq) \cdot x_1 - \sum_{i \geq 3} x_i \quad \text{where } 0 \leq pq \leq 1$$

NK rule

According to the NK rule, a cell is considered to be sensitive if the sum of the largest *n* contributions account for more than *k* percent of the total cell value. Therefore, we define sensitivity as:

$$S = \frac{100}{k} \sum_{i=1}^{n} x_i - \sum_{i=1}^{N} x_i$$

Where $n \geq 1, N \geq 1$ and $50 < k \leq 100$.

Usually, a small value is taken for the parameter *n*, at most up to 5, while *k* is taken to be large, such as 80. The main idea behind this rule is the following. One tries to avoid that *n-1* respondents can make a good estimate of the contribution of the n^{th} respondent by pooling their own values and by comparing this sum with the total cell value. The larger *n* is, the less likely that *n-1* respondents will cooperate. Therefore, the parameter *n* should be chosen to be larger than the maximum size of an (imagined) coalition of respondents.

Duffett rule
This rule is a mixture of NK rules, with different values of *n* and *k* being used for different numbers of cell respondents. This rule was created by and for Statistics Canada, and so only the internal version of G-Confid offers this functionality. Sensitivity is defined as follows:

$S = \max(S_1, S_2)$, where

- S_1 is calculated using a rule (n=1, k=k_1)
- S_2 is calculated using a rule (n=2, k=k_2)

where parameters k_1 and k_2 are confidential.

It should be noted that the Duffett rule is no longer regularly used, as since 2009 Statistics Canada's C2 rule is instead recommended for use.

C2 rule
The C2 rule, also created by and for Statistics Canada, can only be used internally. This is the recommended rule for economic data at Statistics Canada. Sensitivity is defined as follows:

$S = \max(S_D, S_P)$, where

- S_D is calculated using a rule similar to the Duffett rule
- S_P is calculated using a p-percent rule

The details of the parameters used in the C2 rule are confidential.

Arbitrary rule
This rule is a special case of the linear sensitivity rule in which the first four coefficients are specified by the user and other coefficients are set to -1:

$$S = \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \alpha_4 x_4 - \sum_{i=5}^{N} x_i.$$

3.3 Suppression methodology

The main objective of the macro %SUPPRESS is to provide the desired protection level to the sensitive cells while minimizing the loss of information. Using the macro %SUPPRESS, G-Confid identifies the set of cells to suppress that provide the desired protection level.

Underlying the macro %SUPPRESS is the SAS procedure PROC OPTMODEL which solves a linear programming problem for each sensitive cell. The solution to each linear programming problem is a circuit of suppressed cells within the table. To define the linear programming problem, there is an optimization function and a set of constraints. The set of constraints is represented by the structure of the table, and how the cells are related to one another in segments (such as rows and columns).

The optimization function minimizes the amount of protection demanded from other cells in order to protect the sensitive cell.

The cost variable represents the importance of publishing or suppressing each cell. A sensitive cell must be suppressed and therefore G-Confid assigns it a cost of zero. For all other cells, we may specify a cost variable of our choosing.

The cost variable has three components: (i) a variable on the *Outcell* file that represents the informational value of the cell, (ii) one of four pre-defined functions that adjusts this variable, and (iii) one of three pre-defined scaling factors that adjusts this variable.

By default, G-Confid applies the values of *TotalNoise* to represent the informational value of each cell. *TotalNoise* is a reflection of the size and importance of the cell. We can always add a variable to *Outcell* after running PROC SENSITIVITY and prior to running the macro %SUPPRESS, for example, a variable that represents the absolute value of *TotalVar*. To override the default, we specify this added variable using the parameters CVAR1= and CVAR2= when calling the macro %SUPPRESS.

We can apply one of four functions: SIZE, DIGITS, CONSTANT and INFORMATION. The SIZE function is an identity function and does not change the values. The DIGITS function applies a logarithmic transformation to the values. The CONSTANT function yields values of one for all cells. The INFORMATION function applies a reciprocal logarithmic transformation, so the cells with the smallest informational value obtain the largest values and vice versa.

We can apply one of three rescaling factors: NONE, MEAN or SCALE. The NONE factor is an identity function and does not change the values. The MEAN factor rescales relative to a mean value. The SCALE factor is more complex and is defined in the syntax of 5.2 The %SUPPRESS macro. Rescaling may lead to oversuppression in terms of the informational value that is suppressed, although rescaling tends to reduce substantially the runtime of the macro %SUPPRESS.

Usually, there is more than one sensitive cell to be protected in a table. G-Confid processes sensitive cells sequentially, one at a time, starting with the sensitive cell that requires the largest amount of ambiguity. In the process of protecting a particular sensitive cell, G-Confid may partially or fully protect other sensitive cells. The cells that are suppressed to protect a particular sensitive cell form an optimally selected circuit of suppressed cells. Across all sensitive cells that require protecting, we cannot be sure to obtain an optimal solution. To reduce the potential oversuppression that may have occurred, a second residual suppression may be run with the intent of liberating redundant suppressions.

3.4 Dimension variables, the hierarchy and the range

The dimension variables, the hierarchy and the range are three closely related concepts. Dimensions are the variable names presented in our tables (e.g., region); hierarchy indicates the structure and makeup of each dimension in the table (e.g., the table may include the following region categories: Pacific, Prairies, Ontario, Quebec and Atlantic). If the original data do not specifically contain the categories described in the hierarchy, a range can be used (e.g., it could indicate which provinces are included in each region if the province variable is available on the input microdata file).

Dimension variables

Each dimension variable is categorical and consists of levels or classes. The combinations of the levels of the dimension variables define the table of results. Typically for business surveys, dimension variables include:

- The geographic location of the contributor
- The (primary) industry in which the contributor operates
- The category of firm size to which the contributor belongs
- The (primary) activity or type of product the contributor produces
- The geographic location in which the economic activity takes place or to which a good or service is sold

Using the DIMENSION statement of PROC SENSITIVITY we specify the names of the dimension variables that appear in the input microdata file.

Hierarchy

Using the HIERARCHY option we define the hierarchical groupings of cells in the table of results. For each dimension variable that we specify in the DIMENSION statement, we must specify the hierarchy pertaining to that dimension variable. For information pertaining to the structure and the syntax of the hierarchy, please see the 5.1 PROC SENSITIVITY section and also the examples on 2.3 Hierarchies.

Range

Using the RANGE= option we relate the values of the dimension variables on the input microdata file to the most detailed levels of the dimension variables that appear in the table of results.

Although the range is optional, we need to specify the range if the input microdata file provides information at a more detailed level then we intend to produce in the table of results, and for which no confidentiality vetting is needed. For each dimension variable that we specify in the DIMENSION statement, we must specify the range pertaining to that dimension variable. For information pertaining to the structure and the syntax of the range, please see the 5.1 PROC SENSITIVITY section and also the examples on 2.3 Hierarchies.

3.5 The ID variable

With respect to Statistics Canada's classification of business entities, we consider an individual respondent to be an enterprise (as opposed to an establishment or location). Consequently, each row of microdata represents a single record within an enterprise, and the respondent ID variable identifies records belonging to the same enterprise. Although it is quite typical for a vast majority of enterprises to be represented by a single record in the microdata table, it is possible to have multiple records per enterprise that carry the same respondent ID.

We strongly believe that an enterprise with several establishments is better protected when we calculate sensitivity at the enterprise level rather than at the establishment (or location) level. Using contributions at the establishment level underestimates the sensitivity of our cells and increases the risk of publishing sensitive cells. Basically, it is best to base contributions at the enterprise level for the ID variable!

Now, let's briefly consider anonymous respondents. G-Confid recognizes two types of respondents: identifiable respondents and anonymous respondents. Rows of microdata corresponding to identifiable respondents have an identification character in the respondent ID field, while rows corresponding to anonymous respondents have a space character in the respondent ID field. In particular, anonymous respondents in a cell could represent values contributed by a large number of respondents. It is assumed that an intruder cannot estimate these small values overly accurately; in other words, an anonymous respondent will never cause a cell to be sensitive. Note that cells containing anonymous respondents can still be sensitive. In fact, for a particular cell, sensitivity is positive when, for at least one non-anonymous respondent in a cell, the estimate of the respondent's value based on the total cell value is too accurate.

In conclusion, the respondent ID variable is required to be of an alphanumeric character SAS data type.

3.6 The Shadow variable

The shadow variable is an optional variable for which G-Confid calculates the totals at the cell level. These totals appear in the *TotalShadow* variable of the *Outcell* file. If we specify a WEIGHT statement, then *TotalShadow* includes weighted totals, otherwise the totals are unweighted.

Using versions of G-Confid prior to v1.07, we could only specify non-negative, unweighted contributions when assessing the sensitivity. In order to obtain the weighted totals, net of positive and negative contributions, we used to create a pre-weighted variable to serve as the shadow variable. As of version 1.07 we can specify the ACCEPTNEGATIVE option and use a WEIGHT statement so that *TotalVar* calculates the weighted net totals per cell of the variable that we specify in the VAR statement.

3.7 Aggregates

An aggregate is a set of two or more cells within the same segment (row, column), and includes at least one sensitive cell. A sensitive aggregate has a positive sensitivity and meets one of two criteria:

1. The same contributor contributes to two or more of the cells that comprise the aggregate
2. The contributor that is the adversary in the aggregate is also the target in one of the cells that comprise the aggregate

As part of the functionality of PROC SENSITIVITY, G-Confid automatically detects aggregates and creates cells that represent sensitive aggregates. These cells are included on the *Outcell* file with *Type*="A". Please see the examples on 2.4 Aggregates for more details.

An aggregate may include non-sensitive cells. In order to limit the set of non-sensitive cells that G-Confid may add to an aggregate, we set the parameter options M, X, Y and Z (see 5.1 PROC SENSITIVITY). We can adjust these parameters to reduce the number of aggregates and also the runtime of the macro %SUPPRESS, although we also reduce the confidentiality protection.

3.8 The ACCEPTNEGATIVE parameter option

As of version 1.07 of G-Confid, the input microdata going to PROC SENSITIVITY may include negative values of the analysis variable. If this parameter is specified then G-Confid does not exclude any observation on the input microdata file that has a negative value. If ACCEPTNEGATIVE is not specified then the default parameter REJECTNEGATIVE is enforced, where negative values are excluded from the analysis.

3.9 Using a Proxy Variable

Purpose

The purpose of using a proxy variable is to represent the magnitude of a contributor which has a negative contributed value. We choose to rely on a function of this measure of magnitude instead of relying on a smaller and/or negative value that was actually observed.

Suppose for a given industry, typically the profit is about 3.5% of gross revenue, so a business enterprise with one billion dollars of revenue is expected to make about 35 million dollars. Suppose instead the business enterprise loses $200. We might conclude that this value does not reasonably reflect the magnitude of this business enterprise. Instead, we might prefer to rely on 0.035 times the gross revenue as a better representation of the magnitude.

Method

In order to use a proxy variable, the ACCEPTNEGATIVE parameter option must be specified.

We identify a non-negative numeric variable that serves as an indicator of magnitude of the business enterprises. We include this variable on the input microdata file in skinny format going to PROC SENSITIVITY. In the call to PROC SENSITIVITY we specify the name of this variable using the PROXYSIZE statement. In a survey of business enterprises, we might choose a variable such as total revenue, total assets or the number of employees as being a suitable indicator of magnitude.

Next, we specify the proxy ratio, which is the proportion of the PROXYSIZE variable that reflects the comparable magnitude of the analysis variable. Continuing with the example, we set the parameter option PROXYRATIO=0.035 to indicate the proportion of total revenue to use as the minimum value in place of the absolute value of profit.

As part of PROC SENSITIVITY G-Confid then generates the proxy variable, defined as the maximum of (i) the absolute value of the original variable and (ii) the proxy ratio times the PROXYSIZE variable. If the PROXYSIZE variable and the PROXYRATIO parameter are specified, but for certain observations in the microdata file the PROXYSIZE has a missing value, the absolute value of the original value is used as the proxy. We can observe the microdata values of the proxy variable in the *Outlargest* file for the five largest contributions.

Instead of specifying the value of PROXYRATIO, we can let G-Confid choose the proxy ratio based on a specified percentile of the ratios of the absolute values of the analysis variable to the PROXYSIZE variable. G-Confid calculates the proxy ratio of each observation, ranks the ratios from smallest to largest, and selects the proxy ratio corresponding to the PROXYPERCENTILE value that we specified.

If we specify a PROXYSIZE variable without specifying either PROXYRATIO or PROXYPERCENTILE, by default G-Confid applies PROXYPERCENTILE=0.10.

In practice (from Wight, 2017)

To make use of a proxy variable, let X denote the original variable and let Y denote a non-negatively valued variable that is representative of the relative magnitude of each business. The variable Y should be judiciously chosen from among the variables that correlate strongly with the amount of protection that X would ordinarily require. For a given business enterprise r, let $Z_r = \max\{|X_r|, \delta Y_r\}$ denote the variable that will be analyzed for sensitivity, with parameter $0 \leq \delta \leq 1$. For example, if X represented the profit (or loss) of a business enterprise, the total gross revenue might serve as the auxiliary variable Y. The definition of Z was proposed by Tambay and Fillion (2013) who recommended using a small value of δ suitable for data scenarios in which $|x_r|$ was very much smaller than $|y_r|$. Applying the PQ rule to proxy variable Z would lead to protecting the ratio $|X/Y|$ to within $\delta p \times 100\%$.

A suitable choice of the value of δ may not be obvious. With that in mind, a data-driven approach is proposed to determine its value: among cells at the most detailed level of the table, the ratio $|x_r|/y_r$ for the r^{th} contributor to a cell is calculated. The resulting values are ranked in ascending order. The percentile of the ranked distribution (π) at which δ is to be selected may then be specified. As a consequence, the term δy_r is used in place of contributions with the smallest π% of ratio values, and $|x_r|$ otherwise.

The utility of the proxy variable becomes evident over the course of multiple cycles. The use of a proxy variable removes some of the fluctuation associated with variables such as profit that can be positive or negative, and vary in magnitude, across cycles. This approach stabilizes the protection demanded or offered by a contributor, in terms of the magnitude of the contribution. Across cycles, this approach helps to stabilize the suppression pattern, which serves to meet the expectation of the users who seek to obtain results for the domains across cycles.

3.10 The MINRESPW parameter option

If a weight variable is specified, MINRESPW permits us to identify the minimum weighted count of contributors to a cell, below which we want the cell to be considered sensitive and require protection. If the weighted count of contributors is below the value that we specified using MINRESPW, G-Confid sets the sensitivity to the maximum of 1 and the sensitivity as calculated using the sensitivity rule.

Just as with the MINRESP parameter option, the use of MINRESPW is recommended particularly for use with frequency count data instead of magnitude data.

3.11 The *&_GCONFIDRC_* macro variable

After running the %SUPPRESS or the %AUDIT macro, it is possible to know if the execution of the macro succeeded or failed with the macro variable *&_GCONFIDRC_*. With PROC SENSITIVITY, a similar macro variable that can be used is *&syserr*. These variables contain the value 0 if the execution succeeded and a positive non-zero integer if it failed.

These macro variables can therefore be useful if we have a complex program to make sure that everything ran as expected, and also to stop the execution of the code while it is running if an error is found along the way.

4. Strategies

The purpose of this section is to provide users with tools and information to deal with real problems that they might encounter. The section is divided into two categories:

- Publishing more and better
- Specific cases

In Publishing more and better, we discuss different methods to promote the publication of a larger number of cells, or specific cells chosen by the user. In Specific cases, we present a few ideas on what to do in less obvious situations, e.g., how to protect frequency counts.

Note that the Strategies section is under development and could be updated based on user needs. Suggestions are welcome!

4.1 Publishing more and better

4.1.1 Deciding which data not to protect

Customarily we protect the data of enterprises, consistent with our legal and moral obligations to our data providers. G-Confid implements data-driven, evidence-based methodology that was developed over many years and is applied at statistical agencies worldwide. Even so, it may be argued that some data do not need protecting.

Data not requiring protection by law: The *Statistics Act*, a law of Canada, specifies certain industries that may not need protecting, such as statistics available to the public by law (including some Government sector statistics), and information relating to a carrier of persons or of commodities.

Data that may not need protecting due to the inaccuracy of the data value: A value that does not necessarily reflect the contribution of an enterprise may not need protecting. This decision rests with the senior manager who is responsible for ensuring the implementation of the confidentiality policy of the organization (the director, at Statistics Canada). The decision to protect or not to protect imputed data depends in part on the method of imputation. Deterministic imputation, historical imputation, and imputation using administrative data generally lead to a value that reflects exactly or closely the value of the contributor. Allocation by proportions, imputation by parameterized models and donor imputation generally lead to a value that may differ non-negligibly from the value of the contributor. This assessment requires care to make an informed decision. Please consult your confidentiality advisor or the G-Confid team for assistance.

Data that may not need protecting to the non-sensitive nature of the variable: Surveys of enterprises typically include data of a sensitive nature, such as financial variables (most notably revenue and expense variables). Other data may be of a less sensitive nature, such that in the opinion of the director they do not need protecting, all the while mindful of the legal and moral obligations to the data providers.

The value zero customarily does not need protecting. Although for a non-negative variable the value zero in itself is disclosing, i.e., every business enterprise contributed the value zero, there is no measure of economic activity that demands or offers protection. For this reason, National Statistical Institutes (NSIs) generally exclude domains with the value zero from the set of domains to be protected by reason of confidentiality.

Even so, the value zero discloses information that may be to the detriment of the business enterprise, and may be inappropriate for the NSI to reveal this information, such as the expenditure spent on legal requirements (e.g., compliance with waste management laws) or ethical operations (e.g., funding employee health and safety initiatives), or indicators of business viability (e.g., undertaking research and development).

4.1.2 Using waivers

To apply waivers when using G-Confid, create a {0,1} binary numeric variable on the input microdata where 1 indicates that a waiver has been obtained, 0 otherwise.

Full waivers

Suppose a full waivers data file exists and includes the identifiers of the business enterprises. Suppose also the input microdata file for use by G-Confid also includes these identifiers. Merge (e.g., using a PROC SQL join) these two files on the identifiers, and set the binary numeric variable to 1 if the business enterprise is on the waiver data file.

Partial waivers

A partial waiver applies to an internal or marginal cell that is generated by PROC SENSITIVITY, but not necessarily the parent cells related to that cell. Partial waivers that apply to internal cells, and every parent cell to which these internal cells contribute, can be identified by merging a waiver data file by both the identifiers of the business enterprises and the dimension variables.

Partial waivers that apply in any other way cannot be specified prior to running PROC SENSITIVITY. Instead, the *Sensitivity* and *Status* of the specific cell to which a partial waiver applies may be altered so long as the G-Confid user is confident that the cell does not need protecting due to partial waivers.

The waiver functionality may also be used to identify enterprises that do not need protecting due to another reason, such as according to law or decided by the responsible senior manager.

As an alternative to using the waiver functionality, if an entire set of domains are known not to require protection, these domains may be excluded from the hierarchy and therefore from the confidentiality process. These domains will be used neither in the process of checking for sensitivity (as they are deemed not to be sensitive) nor serve to protect other cells. In Canada, certain provisions of the *Statistics Act* enable the publication of results related to the government sector. As these domains are publicly available by statute, these data neither demand protection nor offer protection. For this reason, these domains may be excluded altogether from the confidentiality process.

For more information on using waivers in G-Confid, see the 2.5 Waivers section in the examples.

4.1.3 Favouring specific cells for publication

We can favour the publication of non-sensitive cells by creating a cost variable on the *Outcell* file. To favour publishing a specific cell we specify an elevated cost, for example the value of the grand total of the table of results. For the other cells we set the value of the cost variable equal to *TotalNoise*.

Example: suppose we want to favour the publication of Cell 123, and the grand total of the table of results is 654000. After running PROC SENSITIVITY, we create the variable *CostVar* on the *Outcell* file using a DATA step in SAS.

```
/* cell 123 is not sensitive and we want to favour its publication */
Data outcell;
  Set outcell;
  If (Cellid eq 123) then CostVar=654000;
  Else costvar=TotalNoise;
Run;
```

The choice of cost value affects the suppression pattern. If we do not create a cost variable, by default G-Confid uses *TotalNoise* as the cost. That way, cells with larger values are more likely to be published, while cells with smaller values are more likely to be selected for complementary suppression. This has for effect to maximize the total informational value of the published results. By creating a customized cost variable, we may favour the publication of cells that we want to publish, at the risk of publishing less informational value.

The choice of value affects how G-Confid selects a circuit of cells to suppress. For example, if a segment of cells sums to a marginal cell, and we set the value of the cost of one of these cells higher than the *TotalNoise* of the marginal cell, then the marginal cell is more likely to be suppressed. NSIs do not generally favour publishing a particular cell more than the marginal cell to which it belongs. To favour a particular cell within a segment, a better option may be to set its cost slightly less than the *TotalNoise* of its marginal cell. This approach takes additional manual work.

4.1.4 Reducing the Protection

Using G-Confid we can reduce the protection in several ways. However, not all of these approaches are recommend and may conflict with legal or ethical requirements regarding the protection that we offer to our data providers.

Including take-none data

Data representing the contributions of sampling units in take-none strata, should be included in the input microdata file. The preferred approach is to include observations at the level of the microdata for each take-none contributor including its identifier within the variable representing the ID and its contribution within the variable representing the VAR. These observations are treated just like any other contributions in the microdata. Given that take-none contributions tend to pertain to enterprises of smaller size, these units usually provide protection to larger enterprises and thereby serve to reduce the sensitivity of the cell. If a weight variable is used, typically the weight of a take-none contribution is set to one.

The alternative approach is to create a single observation in the input microdata file that represents the collective net contribution at the cell level of the set of take-none enterprises that contribute to that cell. Using this approach, we anonymize the collective contribution by setting the value of the ID variable to missing (blank).

Caution: this alternative approach may underestimate the sensitivity of a cell in which a take-none contributor serves as either the target or the adversary.

Changing the sensitivity rule

We can release more data by changing the sensitivity rule that we apply. For example, setting SRULE from "PQ 0.2" to "PQ 0.15", or from "NK 1 80" to "NK 1 70", serves to reduce the value of the Sensitivity of the cells, so that the cells demand less protection.

Caution: this approach may contravene the legal or ethical requirements regarding the protection offered to our data providers.

When choosing the sensitivity rule, we may consider the inherent sensitivity of the variable. Some NSIs may prefer to adhere to a single sensitivity rule to apply to all of its confidentiality assessments. Other NSIs may offer the latitude to choose a sensitivity rule that depends on the variable being assessed. Financial variables and measures of economic activity are generally held to a more stringent sensitivity rule. Non-financial variables (e.g., surface area, number of company vehicles) may be held to a less stringent sensitivity rule if the NSI permits.

Seeking waivers

As discussed previously, waivers can help reduce the amount of protection needed.

Using weights

If our survey includes estimation weights, we should include the weight variable on the input microdata file. Applying the PTN framework (Gray, 2016), the weights provide protection against disclosure. See 4.2.2 Using a weight variable for more information.

Using only a minimum count rule

Caution: this approach leads to the disclosure of sensitive data.

This approach involves using a minimum counts rule in place of a sensitivity assessment. See "A note on using MINRESP with magnitude data" within 4.2.3 Protecting frequency counts. This approach ignores the methodology of assessing the sensitivity. Instead, we assume that external users of our results are unable to identify the largest contributors to a cell. Effectively we assume that industry analysts, financial investment experts and even business rivals do not know the identity of the largest enterprises that contribute to a cell.

Assuming a waiver

Caution: this approach leads to the disclosure of sensitive data.

Caution: this approach may contravene the legal and ethical requirements of the NSI to protect its data providers.

This approach involves treating a contribution as if a waiver had been supplied for it, and therefore G-Confid does not protect it. A possible application of this approach is when the contributed value was derived from a probabilistic method such as a parametric model or donor imputation, such that the derived value has a reasonable probability of not reflecting the uncollected or unused value of the contributor.

See 4.1.2 Using waivers for more information.

4.2 Specific cases

4.2.1 Working with negative values

With the introduction of version 1.07 of G-Confid, it is now possible to work with negative values. In the past, G-Confid rejected negative values and they were not taken into account in sensitivity calculations. In order to have negative values included in calculations, the user simply needs to specify the ACCEPTNEGATIVE option in the SENSITIVITY procedure.

By default, when ACCEPTNEGATIVE is specified, G-Confid uses the ADDITIVENOISE option to handle marginal cells. It is, however, possible to specify NOADDITIVENOISE, where sensitivity is calculated independently at the marginal level. To better understand the difference between these two options, see the examples on 2.7 Negative values.

In addition to specifying ACCEPTNEGATIVE and choosing how to handle marginal cells (ADDITIVENOISE or NOADDITIVENOISE), it is possible to provide G-Confid with a proxy. This may prove useful if the user has an auxiliary variable that accurately reflects the size of our contributors. For more information on the subject, see the 3.8 The ACCEPTNEGATIVE option section and the examples on 2.7 Negative values.

4.2.2 Using a weight variable

To use a weight variable, (1) specify the name of the variable using the WEIGHT statement of PROC SENSITIVITY, and (2) specify the WEIGHTPROTLEVEL= parameter option of PROC SENSITIVITY. By default, the weight protection level is set to LINEAR.

The weight variable is a numeric variable with values of at least -1 and no upper limit other than the limit imposed by SAS, which for SAS 9.3 is in excess of 10^{50}.

For all sensitivity rules supported by PTN and using G-Confid v1.07 or later, the weight protection level specifies the protective function that is applied to the microdata. Recommended levels include LINEAR and STEP.

By specifying LINEAR, G-Confid reduces the amount of protection demanded by a target as its weight differs from one, and no protection is required if its weight is -1 or less. The LINEAR approach is consistent with the view that weighted data is inherently protective, so long as the weights are not known to the users of the results.

By specifying STEP, G-Confid maintains the amount of protection demanded by a target if its weight lies between 1 and 3-*p*, where *p* is the value of the PQ parameter (or $p = \frac{(100-k)}{k}$ for an NK rule). Some protection is demanded between *3-p* and *p* and no protection is demanded for a weight greater than 3. The STEP approach is consistent with the view that subsampling is inherently protective, so long as the units selected into sample are not known to the users of the results.

By specifying EXACT, the weights are considered to be known to the users of the results and provide no protection. For example, EXACT would be appropriate if external users were told that all units in a particular domain received a weight of 1.25 because 80% of the units were selected into the sample. Ordinarily a NSI would not reveal the weights.

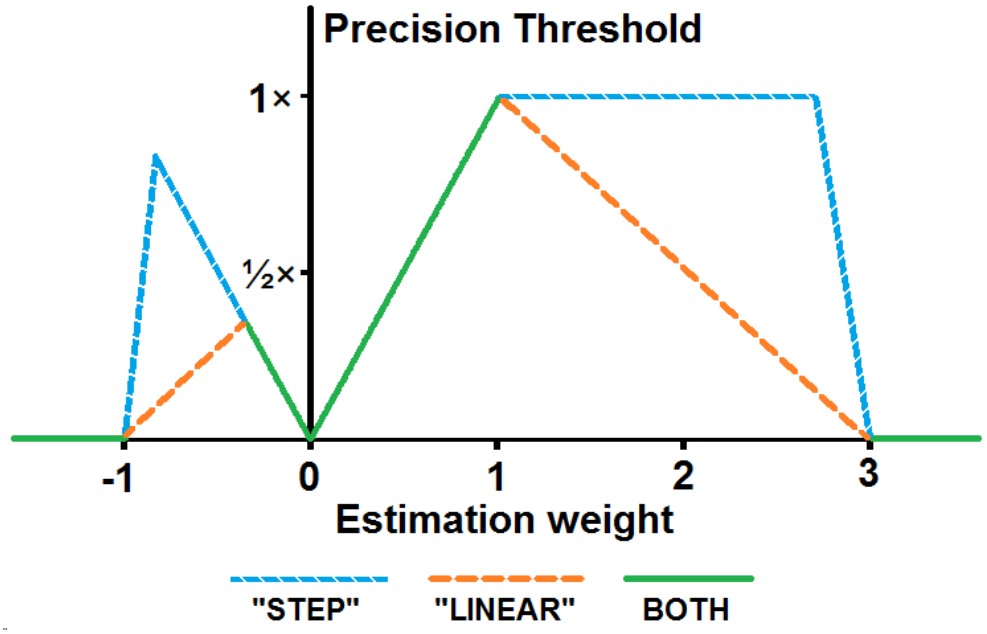


Chart: Factor by which the precision threshold (PT) is reduced for a target contributor
(STEP function in blue, LINEAR function in orange, both in green)

Although LINEAR leads to more publishable cells than STEP, it provides less protection to the data providers. It also increases the risk of perceived disclosure, particularly as doubleton cells become publishable in certain data scenarios. Where a doubleton is known to exist, i.e., there are only two business enterprises in the population that operate in that domain and both were included in sample, but due to weight adjustments their weights are well above one, by using LINEAR the cell is deemed not to be sensitive. The two business rivals in the domain are perceived to be able to determine each other's contribution, given that the NSI published the total value of the domain. In tables of results with substantial numbers of cells representing the contributions of only two business enterprises, STEP is the preferred weight protection level to guard against perceived disclosure.

Reweighting due to nonresponse or calibration

Even if we obtained our input microdata from a census or from administrative sources, data may be unavailable for certain units in the population of study. Consult your survey methodologist about creating estimation weights as a function of nonresponse. Macro-level results may also be calibrated to known totals, and we may create estimation weights that account for the adjustment due to calibration.

4.2.3 Protecting frequency counts

The protection of frequency counts involves a different disclosure control methodology from the assessment of the confidentiality of magnitude data. Each contributor represents a single unit and contributes the value 1, instead of contributing a value that represents the level of economic activity (e.g., revenue measured in dollars). G-Confid offers a choice of two approaches to protecting the confidentiality of frequency counts: rounding or suppression.

Creating frequency counts from microdata using PROC SENSITIVITY

To begin, we set up and run PROC SENSITIVITY and we specify a minimum count rule. Suppose for example we want to protect any frequency count less than five. Using PROC SENSITIVITY we set the parameter option MINRESP=5. This setting indicates to G-Confid to set the Sensitivity equal to 1 if the cell has fewer than five distinct contributors. In order to prevent G-Confid from assessing the dominance or precision of the contributions, we set the SRULE to "ARB -1". The arbitrary rule with all coefficients equal to -1 ensures that G-Confid never calculates a positive value of Sensitivity using the analysis variable that we specify in the VAR statement. PROC SENSITIVITY produces the O*utcell* and *Outconstraint* files that we use as inputs to the rounding module.

Supplying a cell-level data file prior to rounding

We may also supply a data file at the cell level, similar to the *Outcell* file that PROC SENSITIVITY would have produced. We must also supply a file of constraints that describe the linear relationships between the cells, making reference to a *CellId* variable.

Rounding

To round the cell-level data, we can choose to use the macro %GCONFIDROUND or the macro %GCONFIDOPTROUND. We specify a rounding base (such as 5), and G-Confid will round the cell totals to multiples of that base (e.g., 0, 5, 10, 15,…).

Suppression

To suppress the cell-level data, we use the macro %SUPPRESS. Because any cell that is sensitive due to low counts has *Sensitivity*=1, any non-zero cell in the same segment can serve as a complementary suppression.

A note on using MINRESP with magnitude data

The use of MINRESP is not a recommended approach for protecting magnitude data. For example, MINRESP=3 ensures that a count of 1 or 2 results in *Sensitivity*=1, which means the cell demands protection of the value 1, so any other cell in the segment with a frequency count of at least 1 can offer protection. For example, suppose a segment includes cells A, B, C and D where the total of D equals the sum of totals of A, B and C. In cell A,

Enterprise 1 contributes $1,000,000

Enterprise 2 contributes $100,000

Suppose we do not assess dominance or precision and just rely on MINRESP=3. Given that only two enterprises contribute to the cell, the Sensitivity is set to 1, which equates to demanding protection of $1 (because we are protecting magnitude data measured in dollars, and not count data measured in units). Suppose cell B has a total value of $3,000. Because 3,000 is at least as large as 1, G-Confid chooses cell B as a complementary suppression for cell A.

A	B	C	D
X	X	80,000	1,183,000

Enterprise 2 calculated 1,183,000 minus 80,000 minus its own contribution 100,000, and so its best guess of the contributed value of Enterprise 1 is $1,003,000. Using MINRESP we only offer protection to Enterprise 1 as if we had used a p=0.003 rule.

We can use MINRESP in conjunction with a sensitivity rule to protect magnitude data. However, for any NK rule where N ≥ 2 or any PQ rule, the *Sensitivity* will already be positive if fewer than three enterprises contribute to a cell, because the value of *Sensitivity* is the maximum of 1 (due to MINRESP) and the result calculated using the sensitivity rule.

For example, suppose three enterprises contribute to a cell:

Enterprise 1: $20,000 and weight=1.2

Enterprise 2: $2,200 and weight=1.5

Enterprise 3: $1,000 and weight=2

Suppose MINRESP=5 and the SRULE="PQ 0.2". The value of *Sensitivity* is at least 1 because fewer than five enterprises contribute to the cell. However, the *Sensitivity* takes the value $S = \left(\frac{3-1.2}{2}\right) \cdot 0.2 \cdot 20000 - 0.5 \cdot 2200 - 2 \cdot 1000 = 500$ according to the SRULE, applying WEIGHTPROTLEVEL="LINEAR" by default.

4.2.4 Producing revised results after the microdata have changed (including historical revisions)

After releasing preliminary results, some surveys update their microdata and prepare a set of revised tables of results. Confidentiality protection must then be offered to the enterprises that contribute to the revised set of data. Providing external users with two sets of results, preliminary and revised, may facilitate the disclosure of revised results that we intended to protect. To prevent this occurrence, we should favour the suppression pattern of the preliminary results when the macro %SUPPRESS seeks a suppression pattern for the revised results.

To illustrate the peril of generating the suppression pattern of the revised results independently from that of the preliminary results, consider an example in which the suppression patterns of the preliminary table and the revised table were generated independently. In the preliminary table the cell in the first row and first column was sensitive and required protecting. In the revised table, the cell in the second row and third column requires protecting.

Preliminary table

X	X	80	170
X	X	10	100
40	50	90	180
90	180	180	450

Revised table (unsafe)

15	70	80	165
X	60	X	115
X	50	X	180
85	180	195	460

Users with both tables will likely guess (correctly) that in the third row the values remained 40, 50 and 90 because the sum remained 180. Therefore the newly sensitive cell has the value 15, up from 10 in the preliminary release.

Revised table (safe)

X	70	X	165
X	60	X	115
40	50	90	180
85	180	195	460

Users are unsure exactly how many cells saw their values change between the preliminary and revised results, and by how much. Users might assume that the cell in the first row and third column remained 80, but maybe it is 80 no longer.

Notice that we re-used some of the same suppressions in the revised table, but only to the extent that we needed to protect the revised results, and we published the rest.

To favour a suppression pattern, first we include the *Outstatus* variable from the O*utpattern* of the preliminary results (called in the example below *outpattern_old*) on the *Outcell* of the revised results. We only include from the old pattern the cells where the Type=”C”, not the aggregates. Then, we specify a cost variable for the macro %SUPPRESS that encourages but does not force the same suppression pattern to be used with the revised results. That variable is called here *costvar*.

Example:

```
proc sql;
create table outcell2 as
select unique
        new.*,
        old.outstatus as outstatus_old,
        case when
                old.outstatus eq 'X' then 1 /*favour suppression*/
                else new.TotalNoise /*favour release*/
        end as costvar
from outcell as new left join outpattern_old as old
on old.Type='C' and new.DIM_1=old.DIM_1 and new.DIM_2=old.DIM_2 /* etc., include all dimension variables here*/;
quit;
```

Finally, using the macro %SUPPRESS we specify the cost variable.

```
%Suppress(incell=outcell2,
Constraint=outconstraint,
Cfunction1=size,
Cfunction2=information,
Cvar1=costvar,
Cvar2=costvar,
Outcell=outpattern,
Outcomplement=outcomp);
```

4.2.5 Using macro-level adjustments

We can represent a macro adjustment by including an anonymous observation in the microdata input file that represents the value of the adjustment. If the macro adjustment has a negative value, we must specify the ACCEPTNEGATIVE parameter option of PROC SENSITIVITY.

A macro adjustment should never represent an adjustment to the contribution of an enterprise. We can adjust the contribution of an enterprise in one of two ways:

1. Adjust the contributed value of an existing microdata observation that represents the contribution of that enterprise
2. Create another observation in the microdata file that represents the adjustment

Using either approach, the variable that indicates the enterprise ID must identify the enterprise.

5. Syntax

In this section, we present the details of the options and statements for each of the 7 components in G-Confid. This section is taken from the user manual. Therefore, credit for this section goes to the Systems Engineering Division (SED) team.

5.1 PROC SENSITIVITY

Overview

This procedure processes microdata files by aggregating the data into table cells, according to user specifications. It then calculates the sensitivity of the table cells (to be published) and of combinations of cells. Please note that all mandatory options or statements are in red.

Procedure Syntax

PROC SENSITIVITY *<option(s)>*;
  ID *variable*;
  VAR *variable*;
  DIMENSION *variable(s)*;
  SHADOW *variable*;
  WAIVER *variable*;
  PWAIVER *variable*;
  PROXYSIZE *variable*;
  WEIGHT *variable*;
  BY *variable(s)*;

Statement*	Use
ID	Identify the name of the variable which is the key of the input data set
VAR	Identify the name of the variable containing the respondent data
DIMENSION	Identify the name(s) of the variable(s) that represent the hierarchy dimensions
SHADOW	Identify the name of the shadow variable
WAIVER	Identify the name of the variable containing the full waiver flag
PWAIVER	Identify the name of the variable containing the partial waiver flag
PROXYSIZE	Identify the name of the auxiliary size variable used in the calculation of a sensitivity proxy variable under certain conditions.
WEIGHT	Identify the variable that represents the estimation weight
BY	Calculate sensitivity for each BY group

*The statements in red are mandatory.

PROC SENSITIVITY Statement - Options

Option*	Use
DATA=	Specify the input microdata file
HIERARCHY=	Specify the hierarchy used for each dimension
SRULE=	Specify the sensitivity rule to use
OUTCELL=	Specify the SAS output data set that contains the sensitivity of the cells
OUTCONSTRAINT=	Specify the SAS output data set that contains the constraints
OUTLARGEST=	Specify the SAS output data set that contains information about the largest contributors to a cell
OUTPAIRS=	Specify the SAS output data set that contains information about the most sensitive observation pairs in each cell.
OUTTARGETS=	Specify the SAS output data set that contains information the most sensitive combination of observations in each cell.
RANGE=	Specify the interval of codes associated with the lowest level of the hierarchy
MINRESP=	Specify the minimum number of respondents with a non-zero value in a cell
M=	Specify the M parameter
X=	Specify the X parameter to adjust the number of cell combinations
Y=	Specify the Y parameter to adjust the number of cell combinations
Z=	Specify the Z parameter to adjust the number of cell combinations
TOLERANCE=	Specify the upper bound value below which the sensitivity is deemed too small (below this value, the sensitivity is set to 0)
PRINTCODES | NOPRINTCODES	Specify that hierarchies and ranges be printed to the log or not
LIMITWARNINGS | NOLIMITWARNINGS	Specify that the printing of the warnings generated while reading the microdata be limited or not
ACCEPTNEGATIVE	Include negative values when calculating sensitivity. This option is required when using the PROXYSIZE statement.
REJECTNEGATIVE	Exclude negative values from the sensitivity calculations, these observations will be considered as invalid and will be removed from the data set before sensitivity calculations
PROXYRATIO	Specify the threshold below which observations are overridden by the proxy variable.
PROXYPERCENTILE	Specify the percentage of observations overridden by the proxy variable.
PROXYDIAG | NOPROXYDIAG	Determine whether or not to produce a diagnostic report related to the use of a proxy variable
WEIGHTPROTLEVEL	Set the desired level of protection in the presence of weights, which will affect the sensitivity calculations.
WEIGHTDIAG | NOWEIGHTDIAG	Determine whether or not to produce a diagnostic report related to the use of a weight variable.
MINRESPW	Specify the weighted minimum number of respondents with a non-zero value in a cell
ADDITIVENOISE | NOADDITIVENOISE	Determine how the system calculates sensitivity variables at the marginal (or aggregate) cell level

*The options in red are mandatory.

DATA = *SAS-data-set*

Specifies the input SAS data set containing the respondent microdata. The variables associated with the DIMENSION and ID statements must be character variables. All other variables must be numeric, except for the BY variables which can be of either type.

HIERARCHY = *quoted string of alphanumeric codes*

Specifies the hierarchy for each dimension. The dimensions are separated with a semi-colon (;). Each level is separated with a colon (:). A level is a parent code and the remaining values help to identify the codes for its children. There are two ways to identify children codes. Children codes can be listed individually, e.g., parent code 11 can be followed by children codes 111 112 113 114. Alternatively, children codes can be specified by a starting code value, an increment, and an ending code value. To avoid ambiguities the increment is given as a negative value (for example 112 -1 114). The increment can only take the value -1. If using the -1 notation, the value of the end code must be larger than the value of the start code.

It is possible to insert comments anywhere in the string. Comments are enclosed within /* */ characters.

Example:
HIERARCHY="/*DIM1*/1 11 12 13: 11 111 112: 111 1111 -1 1119;/*DIM2*/2 21 22: 21 211 -1 227: 22 23 -1 33;"

For more detailed examples on this topic, please see examples on 2.3 Hierarchies.

SRULE = *quoted string*

Specifies the sensitivity rule to apply. For more detailed examples on this, see examples on 2.2 Sensitivity rules.

Name	Rule parameters	Example
ARB	1 to 4 numbers where each number is: $-1 \leq$ number ≤ 1. The numbers must be specified in decreasing (non-ascending) order.	"arb .75 .5 0 -.5"
DUFFETT	None expected. For Statistics Canada users only.	"duffett"
NK	1 to 3 pairs of numbers where the first number is the number of respondents (n) and the second number represents the percentage of the total (k). *n* is an integer that can take on the values 1, 2, 3, 4 or 5 *k* is a real number, $0 < k < 100$ and $k > 50$ is recommended Note: When using the NK rule with the WEIGHT statement and a WEIGHTPROTLEVEL other than EXACT, a maximum of 2 pairs can be specified.	"nk 1 76 2 80"
PQ	1 number, where $0 <$ number < 1 $0.03 \leq$ number ≤ 0.25 is recommended	"pq .03"
C2	None expected. For Statistics Canada users only.	"c2"

Note: SRULE must be PQ, NK, C2 when the WEIGHT statement is specified along with a WEIGHTPROTLEVEL other than EXACT. As such, the Duffett rule is not supported when using the WEIGHT statement.

OUTCELL = *SAS-data-set*

Specifies the output SAS data set containing the sensitivity of table cells. The variables in this data set are *CellID*, *NbAnonym*, *AnonymTotal*, *NbRespondents*, *Total*, *Sensitivity*, *Status*, *Type*, *ShadowTotal* and the dimension variables specified with the DIMENSION statement. When the WAIVER or PWAIVER statement is specified, variables *SensitivityBeforeWaivers* and *FavCost* will also be added to the data set. If a BY statement is specified, then this data set also contains the specified BY variables. When the PROXYSIZE statement is specified, variable *TotalProxy* will be added to the data set. If you want the *Outcell* data set to be permanent, specify a two-level name (*libref.member-name*). If this option is not specified, a data set with a SAS default name will be generated (DATA*n*). Please refer to the SAS documentation for more details.

Description of variables:

Variable	Content
CellID	Identification of the cell. This identification is generated by the procedure and used in conjunction with the *Outconstraint* file.
NbAnonym	Number of respondents that contributed to the total of the cell but did not have a value in the ID variable.
*AnonymTotal**	The total value of the anonymous respondents for the cell. G-Confid considers as anonymous respondents any observations on the input microdata file with a missing value for the ID variable.
NbRespondents	Number of respondents that contributed to the total of the cell (excluding the anonymous). This number corresponds to the number of distinct respondents.
*Total**	Total cell value
TotalNoise	The total amount of protection that can be provided by a cell against residual disclosure. This variable is required by %SUPPRESS. In the absence of negative values or weights, this value is equal to *Total*.
Sensitivity	The sensitivity of the cell. When the WAIVER or PWAIVER statement is specified, it will contain the cell sensitivity adjusted for waivers.
Status	The status of the cell: If the cell is sensitive: S. If the cell is not sensitive: V.
Type	The type of the cell. If the cell is a table cell: C. If the cell is an aggregate: A.
Sensitivity BeforeWaivers	When the WAIVER or PWAIVER statement is specified, this variable will contain the cell sensitivity before the adjustment for waivers.
FavCost	When the WAIVER or PWAIVER statement is specified, this variable will be added and will contain the cost calculation (*FavCost*) related to the use of waivers.
*ShadowTotal**	If a shadow variable is specified with the SHADOW statement, it will contain the total value of this variable.
*TotalProxy**	Total cell value for the auxiliary variable specified by PROXYSIZE. This variable is only calculated when the PROXYSIZE statement is specified.

*The variables *AnonymTotal, Total, ShadowTotal* and *TotalProxy* in the *Outcell* file were renamed in G-Confid version 1.07.002. They are now respectively named: *AnonymTotalVar, TotalVar, TotalShadow* and *TotalProxyVar*.

OUTCONSTRAINT = *SAS-data-set*

Specifies the output data set containing the information about the constraints. The variables in this SAS data set are *ConstraintId, CellId* and *Coefficient*. They are used to define linear equations of the form cell*1*+cell*2*+…+cell*n* = total. If a BY statement is specified, then this data set also contains the BY variables. If you want the *Outconstraint* data set to be permanent, specify a two-level name (*libref.member-name*). If this option is not specified, a data set with a SAS default name will be generated (DATA*n*). Please refer to the SAS documentation for more details. Description of variables:

Variable	Content
ConstraintId	Identification of a constraint (linear equation). Integer from 1 to N, where N is the number of constraints that G-Confid identifies.
CellId	Identification of a cell that is part of the constraint.
Coefficient	Coefficient of a cell within a constraint. If the value is = 1, the cell is on the left side of the equation. If the value is = -1, the cell is a total.

OUTLARGEST = *SAS-data-set*

Specifies the output data set containing the information about the largest contributors of a cell. This SAS data set is optional. If this option is not specified, no data set will be generated. The variables in this SAS data set are *CellId*, "*key-variable"*, *NbRespondents, Total, TotalPercent*, and optionally, if a shadow variable is specified, *ShadowTotal* and *ShadowPercent*. When the WAIVER or PWAIVER statement is specified, variable *WaiverFlag* will also be added to the data set. If a BY statement is specified, then this data set also contains the BY variables. If the input microdata file contains negative values and ACCEPTNEGATIVE is specified, then those negative values appear in the *Outlargest* file while proxy values do not appear in the *Outlargest* file. If you want the *Outlargest* data set to be permanent, specify a two-level name (*libref.member-name*). Please refer to the SAS documentation for more details. Description of variables:

Variable	Content
CellId	Identification of the cell
<key-variable>	The key value of the contributing respondent. When the value="Anon", the statistics are those of the anonymous observations. When the value="Pool", the statistics pertain to observations that are not among the top five contributors. The name of this variable is the name of the variable specified using the ID statement.
NbRespondents	Number of respondents that contributed to the total of the cell (excluding the anonymous respondents, but including respondents providing contributions of zero). This number corresponds to the number of distinct respondents.
*Total**	The total value of the respondent data, anonymous (*key-variable*="Anon") or the rest of the respondents (*key-variable*="Pool").
TotalPercent	The proportion of the total cell value this respondent represents.
WaiverFlag	When the WAIVER or PWAIVER statement is specified, the *WaiverFlag* from the DATA= data set will be added.
*ShadowTotal**	The total value of the shadow variable for the respondents, anonymous contributors (Anon) or the rest of the respondents (Pool).
ShadowPercent	The proportion of the total shadow value this respondent represents.

*The variables *Total* and *ShadowTotal* in the *Outlargest* file were renamed in G-Confid version 1.07.002. They are now respectively named *TotalVar* and *TotalShadow*.

OUTPAIRS = *SAS-data-set*

Specifies the output data set containing information about the most sensitive observation pairs in each cell. This data set can only be generated when the WEIGHT statement is specified with SRULE="pq" and a WEIGHTPROTLEVEL value other than EXACT. A warning message will show up in the log if these conditions are not met. This SAS data set is optional. If this option is not specified, no data set will be generated. If you want the OUTPAIRS= data set to be permanent, specify a two-level name (*libref.member-name*). This data set contains the following variables:

Variable	Content
CellId	Identification of the cell.
TargetId	Identification of the target ("FT1" or "FT2"), depending on which pair produced higher sensitivity. The observation identified as the target in the cell's most sensitive target / attacker pair.
TargetPT	Precision Threshold (PT) value for the identified target.
AttackerID	The observation identified as the attacker in the cell's most sensitive target / attacker pair. If there is only one respondent in a cell, *AttackerId* will be blank.
AttackerSN	Self-Noise (SN) value for the identified attacker. If there is only one respondent in a cell, *AttackerSN* will be blank.
RemainderCount	Number of remaining units (other than target/attacker). If there are two or fewer respondents, *RemainderCount*=0.
RemainderN	The total noise (N) for remaining units. If there are two or fewer respondents, *RemainderN*=0.

OUTTARGETS = *SAS-data-set*

Specifies the output data set containing information about the most sensitive combination of observations in each cell. This option can only be generated when the WEIGHT statement is specified with SRULE="nk" and a WEIGHTPROTLEVEL value other than EXACT. A warning message will show up in the log if these conditions are not met. This SAS data set is optional. If this option is not specified, no data set will be generated. If you want the OUTTARGETS= data set to be permanent, specify a two-level name (*libref.member-name*). This data set contains the following variables:

Variable	Content
CellId	Identification of the cell.
<*variable-clé*>	Includes the ID of the first *n* observations and the value "Remainder" for the last observation. Includes the ID for the *n* (as specified by the NK rule parameter) observations contained in the most sensitive combination of targets in the cell, and the value "Remainder" for all remaining observations. If the cell contains *n* or fewer observations, no "Remainder" is included.
PTNvariable	Specifies which PTN variable is included in the row: Precision Threshold (PT) for targets, and cumulative Noise (N) for the remaining units.
Value	The value of the variable specified in the corresponding *PTNvariable* column.

RANGE = *quoted string of alphanumeric codes*

Specifies the range of codes associated with the lower level of a hierarchy. The dimensions are separated with a semi-colon (;). Each range is separated with a colon (:). Range codes can be listed individually, e.g., parent code 1112 can be followed by children codes 11121 and 11128. Alternatively, children codes can be specified by a starting code value, an increment, and an ending code value. The increment can only take the value -1. If using the -1 notation, the value of the end code must be larger than the value of the start code.

It is possible to insert comments anywhere in the string. Comments are enclosed within /* */ characters.

Example:
RANGE="/*RANGE1*/1111 11111 -1 11119:/*RANGE2*/1112 11121 11128;"

Range is also discussed in examples on 2.3 Hierarchies.

MINRESP = *positive integer*

MINRESP is used to specify the minimum number of respondents with positive values required for each cell. If a cell has less than the number of respondents specified, then the cell will be sensitive. The sensitivity of the cell is set to 1 (due to the use of MINRESP) and then the sensitivity is calculated according to the SRULE and G-Confid retains the higher of the two values. If that value exceeds the cell total, then the sensitivity equals the total.

M = *integer*

M represents the maximum number of non-sensitive cells that the user permits G-Confid to include in a set of sensitive cells that G-Confid combined to form an aggregate.

This number is optional and has a default of 5.

$0 \leq M \leq 10$

X, Y, Z = *real number*

These numbers are used to adjust the number of cell combinations.

X represents the minimum relative difference between the total of a non-sensitive cell and the net change in the sensitivity of the aggregate (if that cell were included in the aggregate). If the relative difference is below X then the non-sensitive cell is not included in the aggregate.

Y represents the minimum ratio of the total of a non-sensitive cell to the sensitivity of the aggregate without this cell (for this cell to be included in the aggregate). If the ratio is below Y then the non-sensitive cell is not included in the aggregate.

Z represents the minimum ratio of the value of the largest contributor to an aggregate to the total of a non-sensitive cell (before including that cell in the aggregate). If the ratio is below Z then the non-sensitive cell is not included in the aggregate.

The numbers are optional and their default value is 0.

$0 \leq X, Y, Z \leq 100$

TOLERANCE = *real number*

The tolerance identifies the upper bound value below which the sensitivity is deemed too small (below this value, the sensitivity is set to zero). For example, if TOLERANCE=0.05, any value of the sensitivity between 0 and 0.05 will be set to zero.

This number is optional and has a default of 0.01.

0 ≤ TOLERANCE ≤ 1000

PRINTCODES | NOPRINTCODES

Specifies if the HIERARCHY= and RANGE= codes will be printed in the log or not. If PRINTCODES is specified, the codes will be printed in the log. If NOPRINTCODES is specified, no codes will be printed in the log. Optional. PRINTCODES is the default.

LIMITWARNINGS | NOLIMITWARNINGS

Specifies if warning messages printed in the log when reading the input microdata is limited or not. If LIMITWARNINGS is specified, a maximum of 15 warning messages will be printed. If NOLIMITWARNINGS is specified, all warning messages will be printed. Optional. LIMITWARNINGS is the default.

ACCEPTNEGATIVE

This option is required to treat negative values. When enabled, the system will use the absolute value of the VAR variable when calculating sensitivity. This option is required when using any of the Proxy related options, which are otherwise not permitted when all the data is strictly positive. This option is disabled by default.

Note: If both ACCEPTNEGATIVE and REJECTNEGATIVE are specified, REJECTNEGATIVE will be used by default.

REJECTNEGATIVE

This option will exclude negative values when calculating sensitivity. When enabled, observations with negative values will be considered invalid and will be removed from the data set before sensitivity calculations. . REJECTNEGATIVE is the default option, which makes it consistent with G-Confid 1.06 and earlier versions. Information about the number of invalid observations will be displayed in the log.

Note: If both ACCEPTNEGATIVE and REJECTNEGATIVE are specified, REJECTNEGATIVE will be used by default.

PROXYRATIO = *real number*

This option specifies when and how to incorporate the PROXYSIZE variable when calculating sensitivity. When specified, it will be used to determine the threshold at which observations are overridden by the proxy variable. It is optional and its values must be between 0.0 and 1.0. ACCEPTNEGATIVE and PROXYSIZE must both be specified when using this option, otherwise the system will issue a warning in the log and ignore all Proxy related options.

When PROXYRATIO is not specified, the system will calculate PROXYRATIO based on the value provided in PROXYPERCENTILE.

Note: If PROXYSIZE is specified but neither PROXYRATIO nor PROXYPERCENTILE are specified, the system will use a default value for PROXYPERCENTILE of 0.1. When PROXYSIZE is not specified, PROXYRATIO will be ignored.

PROXYPERCENTILE = *real number*

This option specifies when and how to incorporate the PROXYSIZE variable when calculating sensitivity. When specified, it will be applied on the input data set to calculate a corresponding PROXYRATIO value. The default value is 0.1. It is optional and its values must be between 0.0 and 1.0. ACCEPTNEGATIVE and PROXYSIZE must both be specified when using this option, otherwise the system will issue a warning in the log and ignore all Proxy related options.

Note: If PROXYSIZE is specified but neither PROXYRATIO nor PROXYPERCENTILE are specified, the system will use a default value for PROXYPERCENTILE of 0.1. When PROXYSIZE is not specified, PROXYPERCENTILE will be ignored.

PROXYDIAG | NOPROXYDIAG

Specifies whether diagnostic information related to use of the PROXYSIZE statement is added to the log when using the PROXYSIZE statement. When PROXYDIAG is specified along with the PROXYSIZE statement, Table P1 (Effect on sensitivity of using a proxy variable) and P2 (Effect on cell status counts of using a proxy variable) will be added to the log. Default value is NOPROXYDIAG.

Note: If PROXYDIAG is specified while the PROXYSIZE statement is omitted, the system will issue a warning and ignore the stated PROXY options.

WEIGHTPROTLEVEL = *quoted string*

Specifies the level of desired protection in the presence of weights (only applies when the WEIGHT statement is specified). Optional. Valid values are LINEAR, STEP and EXACT. Default value is LINEAR. For more details on these options, see the 4.2.2 Using a weight variable section and the 2.6 Survey weights examples.

If WEIGHTPROTLEVEL is specified without the WEIGHT statement, a warning message will be displayed in the log indicating that all weight related options will be ignored. SRULE must be PQ, NK, C2 when the WEIGHT statement is specified along with a WEIGHTPROTLEVEL other than EXACT.

WEIGHTDIAG | NOWEIGHTDIAG

Specifies whether diagnostic information related to use of the WEIGHT statement is added to the log when using the WEIGHT statement. When WEIGHTDIAG is specified along with the WEIGHT statement, table W1 (Effect on sensitivity of using a weight variable) and table W2 (Effect on cell status counts of using a weight variable) will be added to the log. Default value is NOWEIGHTDIAG, which will not output any weight related diagnostics in the log.

Note: If WEIGHTDIAG is specified while the WEIGHT statement is omitted, the system will issue a warning and ignore the stated WEIGHT options.

MINRESPW = *real number*

Specifies the weighted minimum number of respondents with a non-zero value in a cell. Optional. Similar to MINRESP, this option sets a cell to sensitive if the total weight within the cell is less than a specified threshold. For example, a single respondent with a weight of 3.2 would count as 3.2 respondents. This treatment is consistent with current sensitivity calculations, in that sensitivity is unchanged if all weights are equal to one. MINRESPW must be > 1.0. When MINRESPW is not specified while the WEIGHT statement is used, the system will use the value specified in MINRESP (if provided by the user).

Note: If MINRESPW is specified while the WEIGHT statement is omitted, the system will issue a warning and ignore the stated WEIGHT options.

ADDITIVENOISE | NOADDITIVENOISE

This option determines how the system calculates sensitivity variables at the marginal (or aggregate) cell level. It has no effect on sensitivity at the internal cell level. ADDITIVENOISE is the default value and when used, the system will calculate sensitivity of marginal cells by adding up the values from all the internal cells. When NOADDITIVENOISE is enabled (this is also known as independent noise), the system will calculate sensitivity at the marginal level independently. The default value is ADDITIVENOISE.

Other Statements

ID Statement

ID *variable*;

Specifies the variable that the procedure uses as the key variable of the input data set. It is mandatory. The variable must be a character variable and only one is allowed (no composite key). A missing value represents an anonymous respondent.

VAR Statement

VAR *variable*;

Specifies the variable for which the procedure calculates the sensitivity. It is mandatory. The variable must be a numeric variable.

DIMENSION Statement

DIMENSION *variable-1 <... variable-n>*;

Specifies the variables that the procedure uses as the dimension variables. It is mandatory. The variables must be character variables.

Note: The variables in the DIMENSION statement must be in the same order as those described in the HIERARCHY option. For more information, see examples on 2.3 Hierarchies.

SHADOW Statement

SHADOW *variable*;

Specifies an auxiliary variable that is processed alongside the main variable mostly for diagnostic purposes. It is optional. The variable must be a numeric variable.

This variable is not used by G-Confid in the calculation of the sensitivity.

WAIVER Statement

WAIVER *variable*;

Specifies the variable that the procedure uses as the full waiver flag. It is optional. The variable must be numeric. The WAIVER and PWAIVER statements are mutually exclusive and cannot both be specified. If the WAIVER statement is omitted, the system will calculate the cell sensitivity without waivers as usual. If you specify a full waiver, then the value of *WaiverFlag* in the input dataset must obey the constraint that each contributor has the same value for the *WaiverFlag* in all their contributions, otherwise an error message will be printed in the log regarding the consistency of the waiver flag. When using the WAIVER statement, valid values for the waiver flag are 1 (waivers apply) or 0 (no waivers). If other values are found in the waiver flag, a warning will appear in the log stating that value 0 will be used (i.e., no waiver).

Note: A waiver flag is typically provided by the client to publish their data even though they are confidential. In the case of data tables, waivers are used to publish cells that would otherwise be suppressed in order to respect the confidentiality of the respondent. The system will protect the confidentiality for all other enterprises that contribute to the cell.

PWAIVER Statement

PWAIVER *variable*;

Specifies the variable that the procedure uses as the partial waiver flag. It is optional. The variable must be numeric. The WAIVER and PWAIVER statements are mutually exclusive and cannot both be specified. If PWAIVER is omitted, the system will calculate the cell sensitivity without waivers as usual. When using the PWAIVER statement, valid values for the waiver flag are 1 (waivers apply) or 0 (no waivers). If other values are found in the waiver flag, a warning will appear in the log stating that value 0 will be used (i.e., no waiver).

PROXYSIZE Statement

PROXYSIZE *variable*;

Specifies the name of an auxiliary variable to be used under certain conditions to better represent the protection required (noise provided) by a respondent's contribution. When ACCEPTNEGATIVE is specified, G-Confid uses the magnitude (i.e., the absolute value) of VAR for this purpose. The PROXYSIZE statement can be used to specify an auxiliary size variable that more accurately reflects the protection required / noise provided by a contribution than the absolute value of VAR under certain conditions. This treatment is consistent with current sensitivity calculations, in that sensitivity is unchanged if all values are positive and no proxy is specified.

This statement is optional and the variable must be on the INCELL= input data set. The ACCEPTNEGATIVE option must be specified when the PROXYSIZE statement is used. The variable must be numeric, all of its values should be > 0.0 and not have any missing values. Before calculating sensitivity with the proxy variable, the system will verify the contents of the PROXYSIZE variable and will generate a warning in the log if negative or missing values are found. In each of these cases, it will replace missing or negative values with 0.0. The cases will also be counted and listed in the Microdata Statistics report in the log.

Note: PROXYSIZE can be used along with the following options: PROXYRATIO, PROXYPERCENTILE, PROXYDIAG, and NOPROXYDIAG.

WEIGHT Statement

WEIGHT *variable*;

Specifies the variable used by the procedure to apply the estimation weight. It is optional. The variable must be numeric. The system will verify the contents of the WEIGHT variable and will generate a warning in the log if observations with missing weights are found. When missing values are found in the WEIGHT variable, the system will replace these missing with 1 for the given observations. Cases with negative values of the WEIGHT variable will also be counted and listed in the Microdata Statistics report in the log. For more details on the use of weights, see the 4.2.2 Using a weight variable section and the examples on 2.6 Survey weights.

BY Statement

BY *variable-1 <... variable-n>*;

Specifies the variables that the procedure uses to form BY groups. The BY variables will appear in the three output data sets. You can specify more than one variable. This statement is optional. The variables can be numeric or character.

Details

- If BY variables are specified, the observations of the DATA= input data set must be sorted by the values of those variables. The sorting of variables must be done in ascending order of the values.
- If BY variables are specified, the BY variables specified will appear in the OUTCELL=, OUTCONSTRAINT= and OUTLARGEST= data sets.
- When the REJECTNEGATIVE option is specified (default value), observations with a negative or missing value for the VAR variable will be skipped. The number of skipped observations will be printed in the log.
- The following characters are valid values for the **first** character of a hierarchy or range code:
  0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZçéèêëáàâäîíïóòôöúùûüýÿ
  ÇÉÈÊËÁÀÂÄÎÍÏÓÒÔÖÚÙÛÜÝŸ
- Along with the values specified in the previous bullet, the characters listed below are valid values for the characters following the first character of a hierarchy or range code:
  -!@#$%?&(){}[]_+<>,.´=«»\"
- The values of the DIMENSION, HIERARCHY and RANGE options must be aligned.
  Example:
  DIMENSION *prov naics*;
  Two dimension variables
  HIERARCHY = *"0 1 2; 0 1 2 3;"*
  0 1 2 will be used with the variable *prov*
  0 1 2 3 will be used with the variable *naics*.
  RANGE = *";1 101 201 301: 2 102 202 302: 3 103 203 303;"*
  The string starts with a semi-colon (;). This means that no range values have been specified for the hierarchy 0 1 2 associated with the *prov* variable.
  The values after the semi-colon (;) will be associated with the hierarchy 0 1 2 3 of the *naics* variable.
- When changing the status value in the OUTCELL= output dataset (for further processing in the %SUPPRESS module), the following rules apply:

Status	Sensitivity	Conclusion
"P"	>0	Unacceptable, an error message will be generated
"V"	>0	Consider cell as sensitive, a warning message will be generated
"S"	>0	Acceptable
"X"	>0	Consider cell as sensitive, a warning message will be generated
"P"	≤0	Acceptable
"V"	≤0	Acceptable
"S"	≤0	Consider cell as non-sensitive, a warning message will be generated
"X"	≤0	Acceptable

5.2 The %SUPPRESS macro

Overview

This macro identifies cells to be suppressed in a table, in addition to the sensitive cells, in order to prevent confidential data disclosure. The macro processes aggregated data through a linear programming solver (SAS/OR) using the constraints generated by the PROC SENSITIVITY procedure. Please note that all mandatory parameters are in red.

Macro Syntax

```
%SUPPRESS(
        INCELL=,
        CONSTRAINT=,
        CFUNCTION1=,
        CFUNCTION2=,
        CVAR1=,
        CVAR2=,
        OUTCELL=,
        OUTCOMPLEMENT=,
        BY=,
        SCALECOST=,
        DEBUGINFO=
);
```

Parameters

INCELL = *SAS-data-set*

Name (in the form *libref.member*) of the input data file. This data set contains the actual table cells, and the sensitive aggregates. Starting with G-Confid 1.07, variable *TotalNoise* will appear on the output from the PROC SENSITIVITY procedure. In this case, the system will use the value from *TotalNoise* instead of *Total*. The following variables are required in this data set:

Variable name	Type	Description
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Total	real number	The value of the cell. Is renamed *TotalVar* as of G-Confid 1.07.002.
TotalNoise	real number	The value of the cell when the PROXYSIZE or WEIGHT statement is used to calculate sensitivity in PROC SENSITIVITY.
Sensitivity	real number	Is the sensitivity of the cell as calculated by PROC SENSITIVITY
Status	character	Indicates the status of the cell. Valid values are “P” for published cells, “S” for sensitive cells, “X” for suppressed cells, and “V” for other cells.
Type	character	The type of the cell. If the cell is a table cell: “C”. If the cell is an aggregate: “A”.
<BY-*variables*>	character or numeric	Variables used to form BY groups to which the suppression will be applied.

This data set is mandatory.

CONSTRAINT = *SAS-data-set*

Name (in the form *libref.member*) of the input data file containing the linear constraints coefficients. The following fields are required in this data set:

Variable name	Type	Description
ConstraintId	integer	Is the constraint identification, unique for each linear constraint
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Coefficient	1 or -1	For example, if constraint 2 is as follows (x1, x2 and x3 being cells in a table): x1 + x2 = x3 it can be written: x1 + x2 - x3 = 0. The corresponding 3 observations in the constraint data file will be: *ConstraintId* = 2 *CellId* = 1 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 2 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 3 *Coefficient* = -1
<*BY-variables*>	character or numeric	Variables used to form BY groups to which the suppression will be applied.

This data set is mandatory. The observations of this data set must be sorted in ascending order of the values of the *ConstraintId* variable. If BY variables are specified, the observations of this data set must be sorted by the values of those variables, and by the values of the *ConstraintId* variable; when sorting, the BY variables must be listed before the *ConstraintId* variable. If this data set was generated by PROC SENSITIVITY, it is already sorted with the same BY variables.

CFUNCTION1 = *character string*

Specifies the name of the cost function to be used in phase 1 of %SUPPRESS. Possible values are SIZE (or SIZ), DIGITS (or DIG), CONSTANT (or CON) and INFORMATION (or INF).

This parameter is optional and the default value is DIGITS.

CFUNCTION2 = *character string*

Specifies the name of the cost function to be used in phase 2 of %SUPPRESS. Possible values are SIZE (or SIZ), DIGITS (or DIG), CONSTANT (or CON), INFORMATION (or INF) and NONE.

This parameter is optional. If not specified, phase 2 of the cell suppression process will not be executed.

CVAR1 = *character string*

Specifies the name of the cost variable to be used in phase 1 of %SUPPRESS. It must be a numeric variable present in the INCELL= data set. This variable must have only positive values, and missing values are not allowed. When this parameter is provided, the variable specified is used to calculate the cost function coefficients, instead of *Total* (or *TotalNoise* as of G-Confid 1.07.002).

This parameter is optional and the default value is TOTAL. As of G-Confid 1.07.002 the default value is TOTALNOISE.

CVAR2 = *character string*

Specifies the name of the cost variable to be used in phase 2 of %SUPPRESS. It must be a numeric variable present in the INCELL= data set. This variable must have only positive values, and missing values are not allowed. When this parameter is provided, the supplied variable is used to calculate the cost function coefficients, instead of *Total*.

This parameter is optional and the default value is TOTAL if CFUNCTION2 is not equal to NONE.

OUTCELL = *SAS-data-set*

Name (in the form libref.member) of the output data file.

In addition to the fields contained in the INCELL= data set, this file contains the following two new fields:

Variable name	Type	Description
OutStatus	character	Indicates the status of the cell, as determined by the macro. Valid values are "P" for published cells or "X" for suppressed cells.
NetVariation	numeric	Is the net variation in absolute value of the cell, required to protect sensitive cells, as calculated by the macro. A sensitive cell always has a non-zero net variation. A non-zero net variation for a non-sensitive cell indicates that this cell has been selected as a complement to a sensitive cell. A published cell has a zero net variation.

This parameter is optional and the default value is *work.Suppressoutcell*.

OUTCOMPLEMENT=*SAS-data-set*

Name (in the form *libref.member*) of the output complement data file. A complement or complementary cell is a cell that must be suppressed in order to protect sensitive cells or sensitive aggregates. Complements may be sensitive or not, but the algorithm implemented in %SUPPRESS favors the selection of sensitive cells as complements. This file contains the identification of all the complements identified, by sensitive cell. Note that complements are listed for a cell only if the LP solver has been called to protect that cell; complements are not listed for sensitive cells that have been protected as a result of the protection of other cells. This file contains the following fields:

Variable name	Type	Description
CellId	integer	Unique cell identification of the sensitive cell for which the complements are listed.
Phase	integer	Indicates the phase of the suppression process, 1 or 2.
ProtectionNumber	integer	Order in which the sensitive cells were protected.
ComplementId	integer	Lists the values of *CellId* of all the complements of the sensitive cell, for a given phase.

Note: Generate this file with care as its size can be very large. This parameter is optional.

BY = *list of variable names separated by blanks*

Specifies one or more variable names used to create BY groups. The %SUPPRESS process will be applied to each BY group separately. If no BY variables are specified, the process will be applied to the entire input data set. If BY variables are specified then the observations of this data set must be sorted by the values of the BY variables.

This parameter is optional.

SCALECOST = *character string*

Specifies the method used to reduce the cost function coefficients. Possible values are NONE, MEAN and SCALE.

- When SCALECOST=NONE, the cost function coefficients of the LP problem will be used without modifications.
- When SCALECOST=SCALE, the cost function coefficients of the LP problem will be scaled according to the following algorithm:

  $$y = \frac{(b-a)(x-min)}{(max-min)}$$

  where *b* and *a* are the upper and lower bounds of the scaling interval, *max* and *min* are the maximum and minimum values of the cost function coefficients, *x* is the value of the current coefficient being scaled and *y* is the value of the scaled coefficient.

- When SCALECOST=MEAN, the cost function coefficients of the LP problem will be reduced according to the following algorithm:

  $$y = \frac{\sum_1^n x_i}{n}$$

  where *x* is the value of the coefficient, *y* is the value of the reduced coefficient and *n* is the number of coefficients of the cost function.

This parameter is optional and the default value is NONE.

DEBUGINFO = *character string*

Specifies to print or not additional information in the log for debugging purposes. Possible values are YES and NO. If the value is YES, the last linear programming problem submitted to the solver is also saved in a data set, in MPS format, in the same location as the OUTCELL= data set.

This parameter is optional and the default value is NO.

5.3 The %AUDIT macro

Overview

This macro verifies the validity of a suppression pattern. Please note that all mandatory parameters are in red.

Macro Syntax

```
%AUDIT(
        INCELL=,
        CONSTRAINT=,
        OUTCELL=,
        LBFACTOR=,
        UBFACTOR=,
        BY=,
        DEBUGINFO=,
        REPORTLEVEL=,
        SASCONNECT=
);
```

Parameters

INCELL = *SAS-data-set*

Name (in the form *libref.member*) of the input data file. This data set contains the actual table cells and the sensitive aggregates. The following fields need to be present in this data set:

Variable name	Type	Description
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Total or *TotalNoise*	real number	The value of the cell. This variable is called *Total* up to G-Confid version 1.07.001, and *TotalNoise* as of v1.07.002.
Sensitivity	real number	Is the sensitivity of the cell as calculated by PROC SENSITIVITY
Status	character	Indicates the status of the cell. Valid values are “P” for published cells, “S” for sensitive cells, “X” for suppressed cells, and “V” for the other cells.
OutStatus	character	Indicates the status of the cell. Valid values are “P” for published cells, and “X” for suppressed cells
Type	character	“C” for table cells, and “A” for sensitive aggregates
NetVariation	real number	Is the net variation in absolute value of the cell, required to protect sensitive cells, as calculated by %SUPPRESS. A sensitive cell always has a non-zero net variation. A non-zero net variation for a non-sensitive cell indicates that this cell has been selected as a complement to a sensitive cell. A published cell has a zero net variation.
<*BY-variables*>	character or numeric	Variables used to form BY groups to which the audit process will be applied.

This data set is mandatory.

CONSTRAINT = *SAS-data-set*

Name (in the form *libref.member*) of the input data file containing the coefficients of the linear constraints. The following fields are required in this data set:

Variable name	Type	Description
ConstraintId	integer	Is the constraint identification, unique for each linear constraint
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Coefficient	1 or -1	For example, if constraint 2 is as follows (x1, x2 and x3 being cells in a table): x1 + x2 = x3 it can be written: x1 + x2 - x3 = 0. The corresponding 3 observations in the constraint data file will be: *ConstraintId* = 2 *CellId* = 1 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 2 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 3 *Coefficient* = -1
<*BY-variables*>	character or numeric	Variables used to form BY groups to which the audit process will be applied.

This parameter is mandatory. The observations of this data set must be sorted in ascending order of the values of the *ConstraintId* variable. If BY variables are specified, the observations of this data set must be sorted by the values of those variables, and by the values of the *ConstraintId* variable; when sorting, the BY variables must be listed before the *ConstraintId* variable. If this data set was generated by PROC SENSITIVITY, it is already sorted.

OUTCELL = *SAS-data-set*

Name (in the form *libref.member*) of the output data file. This data set contains only cells identified as suppressed, (i.e., having an 'X' for OUTSTATUS in the INCELL= dataset), as well as the sensitive aggregates. The following fields are present in this data set:

Variable name	Type	Description
CellId	integer	Is the cell ID, unique for each cell (including sensitive aggregates)
Total or *TotalNoise*	real number	The value of the cell. This variable is called *Total* up to G-Confid version 1.07.001, and *TotalNoise* as of v1.07.002.
Sensitivity	real number	The sensitivity of the cell as calculated by PROC SENSITIVITY
OutStatus	character	Indicates the status of the cell. The only possible value in this output data set is 'X' because this data set contains only suppressed cells.
LBound	real number	Lower bound value of the cell (=LBFACTOR * *Total*)
MinValue	real number	Minimum value of the cell, as calculated by the LP solver (should be between *LBound* and *Total*)
MaxValue	real number	Maximum value of the cell, as calculated by the LP solver (should be between *Total* and *UBound*)
UBound	real number	Upper bound value of the cell (= UBFactor * *Total*)
MidPoint	real number	Midpoint value of the cell (= (*MinValue* + *MaxValue*)/2)
ProblemIndicator	integer	Problem indicator for the cell. For sensitive cells or sensitive aggregates, its value is 0 for a good protection, 1 for an unachieved protection, and 2 for an exact disclosure. For complements, its value is 0 for a good protection, and 2 for an exact disclosure.
Type	character	'C' for table cells, and 'A' for sensitive aggregates
IntervalWidth	real number	Interval width of the cell, in percentage (= 100*(*MaxValue* - *MinValue*)/ *Total*)
LTolerance	real number	Lower tolerance of the cell (=*Total* – *Sensitivity*/2)
UTolerance	real number	Upper tolerance of the cell (=*Total* + *Sensitivity*/2)
The coordinates of cells, and other fields present in the INCELL data set.		

This parameter is optional and the default value is *work.Auditoutcell*.

LBFACTOR = *real number*

Numeric value for the lower bound factor, which is between 0 and 1 inclusive.

This parameter is optional; the default value is 0.5.

UBFACTOR = *real number*

Numeric value for the upper bound factor, which is between 1 and 10 inclusive.

This parameter is optional and the default value is 1.5.

BY = *list of variable names separated by blanks*

Specifies one or more variable names used to create BY groups. The %AUDIT process will be applied to each BY group separately. If no BY variables are specified, the process will be applied to the entire input data set. If BY variables are specified then the observations of this data set must be sorted by the values of the BY variables. This parameter is optional.

DEBUGINFO = *character string*

Specifies to print or not additional information in the log for debugging purposes. Possible values are YES and NO. If the value is YES, the last linear programming problem submitted to the solver is also saved in a data set, in MPS format, in the same location as the OUTCELL= data set.

This parameter is optional and the default value is NO.

REPORTLEVEL = *integer*

Specifies the level of reports to be generated when %AUDIT is completed. Possible values are 1 and 2. When the value is 1, only the report giving the statistics for *Problem Indicator* is generated. This report gives the number of "Good protection", "Unachieved protection", and "Exact disclosure" for the suppressed cells. When the value is 2, two supplementary reports are generated, giving the statistics of the "Midpoint Problem" for suppressed cells and sensitive aggregates (Tambay and Fillion, 2011).

This parameter is optional and the default value is 1.

SASCONNECT = *character string*

Specifies to use SAS/Connect to perform parallel processing across multiple processors in order to decrease execution time. Valid values are YES and NO. If the value is YES, the cells to be processed in %AUDIT are distributed to the available processors of the machine where distinct SAS sessions are created to process them. The output from all these SAS sessions are then collected to create the final %AUDIT output. All this process is transparent to the user.

This parameter is optional and the default value is NO. It should be set to YES when running in a SAS Grid environment to take advantage of parallel processing across multiple nodes.

Details

Parameters not available in version 1.07 of G-Confid:

- AUDITSENSITIVECELLSONLY = *character string*
- SASCONNECTTHRESHOLD = *integer*
- PARALLELMODE = *integer*
- NUMBEROFNODES = *integer*
- SASAPPLICATIONSERVER = *character string*

5.4 The %AGGREGATE macro

Overview

This macro describes sensitive aggregates using the constraints file and cells file produced by PROC SENSITIVITY. Aggregates are not the marginal cells in a table. Aggregates (cells with *type* = "A" in the OUTFILE= file) are combinations of cells that are sensitive. G-Confid performs this check to identify cases, for example, where the same respondent is dominant in more than one cell. Please note that all mandatory parameters are in red.

Macro Syntax

```
%AGGREGATE (
        DIMENSION=,
        CONSTRAINT=,
        INCELL=,
        OUTFILE=,
        BY=
);
```

Parameters

DIMENSION = *list of variable names separated by blanks*

Specifies one or more variable names used as the dimension variables. This list of variables must be the same as the one defined as the dimension variable in the procedure PROC SENSITIVITY. The variables must be character variables and must be separated by blanks. This parameter is mandatory.

CONSTRAINT = *SAS-data-set*

Name (in the form *libref.member*) of the input data file containing the linear constraints coefficients. The following variables need to be present in this data set:

Variable name	Type	Description
ConstraintId	integer	The constraint identification, unique for each linear constraint.
CellId	integer	The cell identification, unique for each cell (including sensitive aggregates).
<BY-variables>	character or numeric	Variables used to form BY groups.

This file is mandatory.

INCELL = *SAS-data-set*

Name (in the form *libref.member*) of the input data file containing the actual table cells and the sensitive aggregates. Starting with G-Confid 1.07, variable *TotalNoise* will appear on the output from the PROC SENSITIVITY procedure. In this case, the system will use the value from *TotalNoise* instead of *Total*. The following fields are required in this data set:

Variable name	Type	Description
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Total	real number	The value of the cell
TotalNoise	real number	The value of the cell when the PROXYSIZE or WEIGHT statement is used to calculate sensitivity in PROC SENSITIVITY.
Sensitivity	real number	Is the sensitivity of the cell as calculated by PROC SENSITIVITY
Status	character	Indicates the status of the cell.
Type	character	The type of the cell. If the cell is a table cell, the type is “C”. If the cell is an aggregate, the type is “A”.
<DIMENSION-Variables>	character	Variables representing the dimensions.
<BY-variables>	character or numeric	Variables used to form BY groups.

This file is mandatory.

OUTFILE = *SAS-data-set*

Name (of the form of *libref.member*) of the output file. Each constraint (*ConstraintId*) corresponds to a different aggregate. The following variables will be present in this data set:

Variable name	Type	Description
<BY-variables>	character or numeric	Variables used to form BY groups.
<DIMENSION-variables>	character	Variables representing the dimensions.
ConstraintId	integer	Is the constraint identification, unique for each linear constraint
CellId	integer	Is the cell identification, unique for each cell (including sensitive aggregates)
Type	character	The type of the cell. If the cell is a table cell, the type is “C”. If the cell is an aggregate, the type is “A”.
Total or *TotalNoise*	real number	The value of the cell. This variable is called *Total* up to G-Confid version 1.07.001, and *TotalNoise* as of v1.07.002.
Sensitivity	real number	Is the sensitivity of the cell as calculated by PROC SENSITIVITY

This parameter is optional and the default value is *work.Aggregateoutcell*.

BY = *list of variable names separated by blanks*

Specifies one or more variable names used to create BY groups. This list of variables must be the same as the one defined as the dimension variable in the procedure PROC SENSITIVITY. The list of variable names must be separated by blanks. This parameter is optional.

5.5 The %REPORTCELLS macro

Overview

This macro produces a report where sensitive cells are highlighted in red and complements are highlighted in yellow. The report produces 2 dimensional tables for all combinations of variables specified in the parameter BY (defined below). Please note that all mandatory parameters are in red.

Macro Syntax

```
%REPORTCELLS(
        COLNAME=,
        ROWNAME=,
        INCELL=,
        HIERCOL=,
        HIERROW=,
        BY=
);
```

Parameters

COLNAME = *variable*

Specifies one variable used to identify the columns of the report. This parameter is mandatory.

ROWNAME = *variable*

Specifies one variable used to identify the rows of the report. This parameter is mandatory.

INCELL = SAS-data-set

Name (in the form *libref.member*) of the input data file. The input file corresponds to the output of the macro %SUPPRESS. This file content cells to be treated as well as the status. The following variables need to be present in this data set:

Variable name	Type	Description
Total or *TotalNoise*	real number	The value of the cell. This variable is called *Total* up to G-Confid version 1.07.001, and *TotalNoise* as of v1.07.002.
Sensitivity	real number	Is the sensitivity of the cell as calculated by PROC SENSITIVITY
OutStatus	character	Indicates the status of the cell, as determined by the macro %SUPPRESS.
Type	character	The type of the cell. If the cell is a table cell: “C”. If the cell is an aggregate: “A”.
<COLNAME-variable>	character or numeric	Name of the variable used in the parameter COLNAME
<ROWNAME-variable>	character or numeric	Name of the variable used in the parameter ROWNAME
<BY-variables>	character or numeric	Name of the variable(s) used in the parameter BY

This file is mandatory.

HIERCOL = *quoted string of alphanumeric codes*

Specifies the hierarchy used to order the columns of the report. This hierarchy must correspond to the variable defined in the parameter COLNAME. The hierarchy can be either a list containing the desired order or a list defined by level (please refer to PROC SENSITIVITY documentation for the level hierarchy). This parameter is optional. By default, the columns of the report are in alphanumeric order.

HIERROW = *quoted string of alphanumeric codes*

Specifies the hierarchy used to order the rows of the report. The hierarchy must correspond to the variable defined in the parameter ROWNAME. The hierarchy can be either a list containing the desired order or a list defined by level (Please refer to PROC SENSITIVITY documentation for the level hierarchy). This parameter is optional. By default, the rows of the report are in alphanumeric order.

BY = *list of variable names separated by blanks*

Specifies one or more variable used to produce the tables of the report. If there is more than one variable, the tables will be produced for all combinations of these variables. This parameter must contain all variables defined in the parameters DIMENSION and BY in PROC SENSITIVITY that are not use to define the columns (COLNAME) and rows (ROWNAME) of this report. This parameter is optional.

Details

- All variables defined for the parameters DIMENSION and BY in PROC SENSITIVITY must be contained in the parameters BY, COLNAME and ROWNAME in this macro.
- Cells suppressed by users are considered sensitive if the sensitivity is greater than 0, otherwise the cells are considered residual.
- The report can be produced in Pdf, Excel or Word format.

```
Pdf:
ods pdf="Location where we want the output file\Name of the output file.pdf";
Excel:
ods html="Location where we want the output file\Name of the output file.xls";
Word:
ods html="Location where we want the output file\Name of the output file.doc";
```

5.6 The %GCONFIDROUND macro

Overview

This macro is used to round a data table and create uncertainty about the exact value of the data counts contained in the original table. Please note that all mandatory parameters are in red.

Macro Syntax

```
%GCONFIDROUND(
        INCELL=,
        INCONSTRAINT=,
        CLASS=,
        VAR=,
        OUTCELL=,
        OUTMETADATA=,
        OUTDISTANCE=,
        BASE=,
        MULTISTART=,
        MAXTIME=,
        SEED=,
        CTRL=,
        SOLVE=,
        INTEGERRESULTS=
);
```

Note: It is not necessary to have both the CLASS and INCONSTRAINT parameters specified. Only one of those two is necessary.

Parameters

INCELL = *SAS-data-set*

Name (in the form *libref.member*) of the input data file. This data set contains the initial table of cells to be rounded. The specific variables to include on this dataset depend on the rounding method that is used. The system can perform the rounding in two ways, either method 1 (no constraint file) or method 2 (with constraint file).

Variable name	Type	Description
<VAR-variable>	real number	Target variable to be rounded. This variable must be numeric. This variable is required for both methods.
<CLASS-variable(s)>	character or numeric	Classification variable(s). Can contain one or more classification variables. Required for method 1 (no constraint file).
CellId	character or numeric	Unique identifier of each cell. Required for method 2 (with constraint file).
Type	character	Type of cell. If the cell is a table cell: “C”. If the cell is an aggregate: “A”. Optional. This variable will be used to identify the aggregate records to exclude when the INCELL file is created by PROC SENSITIVITY.

This data set is mandatory and the input file must be rectangular (complete).

INCONSTRAINT = *SAS-data-set*

Name (in the form *libref.member*) of the input data file containing the linear constraints to apply. This constraint file can be obtained by running PROC SENSITIVITY. The following fields are required in this data set:

Variable name	Type	Description
ConstraintId	numeric (integer)	Unique identifier of each constraint.
CellId	character or numeric	Unique identifier of each cell.
Coefficient	1 or -1	Cell coefficient in the constraint. If the value is 1, the cell will be on the left side of the equation. In the opposite case, if the value is -1, the cell will be on the right side of the equation. For example, if constraint 2 is as follows (x1, x2 and x3 being cells in a table): x1 + x2 = x3 it can be written: x1 + x2 - x3 = 0. The corresponding 3 observations in the constraint data file will be: *ConstraintId* = 2 *CellId* = 1 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 2 *Coefficient* = 1 *ConstraintId* = 2 *CellId* = 3 *Coefficient* = -1

This data set is mandatory for method 2 (with constraint file) but is not used for method 1 (no constraint file).

CLASS = *character string*

Specifies the variable(s) whose values define subgroups for the analysis. These variables need to be part of the INCELL= dataset.

This parameter is mandatory for method 1 (no constraint file) but is not used for method 2 (with constraint file).

VAR = *character string*

Specifies the name of the variable that contains the data to be rounded. This variable must be numeric and be contained in the INCELL= data set.

Note: The presence of missing values in the VAR variable is not recommended and can lead to unpredictable results.

OUTCELL = *SAS-data-set*

Name (in the form *libref.member*) of the output data file. This file contains the following fields:

Variable name	Type	Description
CellId	numeric	Unique identifier of each cell.
<CLASS-variable(s)>	character or numeric	Classification variable(s). Variable(s) will appear on the file when method 1 is used (no constraint file).
<VAR-variable>	real number	Target variable to be rounded.
RoundCell	numeric	Rounded result of the variable specified by the VAR= parameter.

This parameter is mandatory.

OUTMETADATA = *SAS-data-set*

Name (in the form *libref.member*) of the output metadata file, providing key statistics on the rounding that was performed. This file contains the following fields:

Variable name	Type	Description
JobDate	date	Date and start time for the execution of the rounding macro.
ElapsedTime	character	Elapsed time required to solve the rounding problem, presented in the format hh:mm:ss.
TotalCells	numeric	Total number of cells.
Internal Cells	numeric	Total number of interior cells.
MarginalCells	numeric	Number of marginal cells.
FinalDistance	numeric	Distance of the final solution (sum of the absolute distances exceeding the upper or lower limit as defined by the rounding base).
FinalInfoLoss	numeric	The loss of information measures the extent of the change relative to the rounding base.
ControlStatus	numeric	Specifies whether the solution is controlled or not. If *ControlStatus*=1 (yes), solution is controlled. If *ControlStatus* =0 (no), solution is not controlled.
AdditivityStatus	numeric	Specifies whether the solution is additive or not. If *AdditivityStatus* =1 (yes), solution is additive. If *AdditivityStatus* =0 (no), solution is not additive.
AdditivityRate	numeric	Additivity rate expressed as a percentage. If *ControlStatus* =1, equal to the additivity rate of marginal cells. If *ControlStatus* =0, *AdditivityRate* will be missing(.) When *FinalDistance* =0, *AdditivityRate* is 1.

OUTDISTANCE = *SAS-data-set*

Name (in the form *libref.member*) of the output distance file, providing information on the distance calculated by the rounding macro. This file contains the following fields:

Variable name	Type	Description
DistanceType	character	Type of distance calculated (LP, SEQ, ITE) Where LP: Linear Programming SEQ: Sequential ITE: Iterative
StartPoint	numeric	MULTISTART key. When *DistanceType*=LP or SEQ, this variable will be missing.
Iteration	numeric	Number of iterations required by the system to resolve the table in iterative mode or the number of cells adjusted when using SOLVE=LP or SOLVE=MILP.
Distance	numeric	Distance of the final solution.

BASE = *real number*

Specifies the rounding base used by the rounding algorithm. The value must be greater than or equal to 1.

This parameter is optional and the default value is 1.

MULTISTART = *integer*

Specifies the number of starting points used by the iterative rounding algorithm. The value must be greater than or equal to 0.

This parameter is optional and the default value is 1.

MAXTIME = *integer*

Specifies the upper time limit used to resolve the rounding problem, specified in seconds. By default, there is no time limit set by the system.

This parameter is optional.

SEED = *integer*

Specifies the initial value used to perturb the solution. A perturbation is applied before each iteration of MULTISTART. The value must be greater than or equal to 1.

This parameter is optional and the default value is 1.

CTRL = *integer*

Specifies whether the solution must satisfy or not the margin control conditions. The value must be either 1 (the solution must satisfy the conditions) or 0 (the solution does not need to satisfy the conditions).

This parameter is optional and the default value is 0.

SOLVE = *character string*

Specifies the suggested method used to resolve the rounding problem. Possible values are LP (use linear programming), MILP (Mixed Integer Linear Programming) or NOLP (do not use linear programming).

This parameter is optional and the default value is NOLP.

INTEGERRESULTS = *character string*

When INTEGERRESULTS=YES, values will be rounded to the nearest integer.

This parameter is optional and the default value is NO.

5.7 The %GCONFIDOPTROUND macro

Overview

This macro is used to round a data table and create uncertainty about the exact value of the data counts contained in the original table. The %GCONFIDOPTROUND macro produces an output table with the same structure as the input table, and is as similar as possible. Please note that all mandatory parameters are in red.

Macro Syntax

```
%GCONFIDOPTROUND(
        INCELLS=,
        INCONSTRAINTS=,
        INCELLCONSTRAINTS=,
        OUTCELLS=
        OUTCONSTRAINTS=,
        OUTSUMMARY=,
        BASE=,
        OBJECTIVE=,
        SOLVER=,
        OPTMODELLOG=,
        OPTMODELSTATUS=
);
```

Parameters

INCELLS = *SAS-data-set*

Name (in the form *libref.member*) of the input data file with the cell level input data set. The following fields are required in this data set:

Variable name	Type	Description
CellId	numeric	Unique identifier of each cell. This key column must be unique for each row. It is mandatory and must contain real numbers.
Total	numeric	The value of the cell to be rounded. It is mandatory and must contain real numbers.
Weight	numeric	The weight that is applied by the rounding algorithm. It is mandatory and must contain positive real numbers.
CellLB	numeric	The cell lower bound value. For each row, the value of *CellLB* must be less than or equal to the value of *CellUB*. If the column *CellLB* is missing, a missing value will be used for the lower bound of each cell. It is optional and can contain real numbers or missing values.
CellUB	numeric	The cell upper bound value. For each row, the value of *CellLB* must be less than or equal to the value of *CellUB*. If the column *CellUB* is missing, a missing value will be used for the lower bound of each cell. It is optional and can contain real numbers or missing values.

This data set is mandatory.

INCONSTRAINTS = *SAS-data-set*

Name (in the form *libref.member*) of the constraint level input data set. The following fields are required in this data set:

Variable name	Type	Description
ConstraintId	numeric	Unique identifier for each constraint. Mandatory.
ConstraintLB	numeric	For each row, the value of *ConstraintLB* must be less than or equal to the value of *ConstraintUB*. If the column *ConstraintLB* is missing, a value of zero is used for the lower bound of each cell. It is optional and can contain real numbers or missing values.
ConstraintUB	numeric	For each row, the value of *ConstraintLB* must be less than or equal to the value of *ConstraintUB*. If the column *ConstraintUB* is missing, a value of zero is used for the lower bound of each cell. It is optional and can contain real numbers or missing values.

This data set is mandatory.

INCELLCONSTRAINTS = *SAS-data-set*

Name (in the form *libref.member*) of the cell and constraint level input data set. This constraint file can be obtained by running PROC SENSITIVITY. The following fields are required in this data set:

Variable name	Type	Description
CellId	numeric	See description of *ConstraintId*.
ConstraintId	numeric	The column *CellId* and *ConstraintId* are a composite key column that must have a unique pair of values for each row. Both are mandatory.
Coefficient	1 or -1	Coefficient of a cell within a constraint. If the value is = 1, the cell is on the left side of the equation. If the value is = -1, the cell is a total.

Note : Each value of *CellId* must occur in the data set specified by parameter INCELL= and each value of *ConstraintId* must occur in the data set specified by parameter INCONSTRAINT=. This data set is mandatory.

OUTCELLS = *SAS-data-set*

Name (in the form *libref.member*) of the output data set. The values in this data set are determined by solving the table rounding problem. They are calculated from the input data sets using the OPTMODEL procedure. This data set contains the following fields:

Variable name	Type	Description
CellId	numeric	Unique identifier for each row.
UpperRes	numeric	Upper_residual. Contains real numbers between 0 and 1.
PosShiftIndi	numeric	Positive_shift_indicator. Contains 0 or 1.
PosBaseShift	numeric	Positive_base_shift. Contains positive integers.
PosShift	numeric	Positive_shift. Contains positive real numbers.
LowerRes	numeric	Lower_residual. Contains real numbers between 0 and 1.
NegaShiftIndi	numeric	Negative_shift_indicator. Contains 0 or 1.
NegaBaseShift	numeric	Negative_base_shift. Contains positive integers.
NegaShif	numeric	Negative_shift. Contains positive real numbers.
Shift	numeric	Contains real numbers.
AbsShift	numeric	Absolute_shift. Contains positive real numbers.
ResIndi	numeric	Residual_indicator. Contains 0 or 1.
CellLB	numeric	Cell_LowerBound. Contains real numbers or missing values.
CellUB	numeric	Cell_UpperBound. Contains real numbers or missing values.
Weight	numeric	Contains positive real numbers.
Total	numeric	Rounded (adjusted) value.
RoundedTotal	numeric	Contains integer multiples of the base.

This parameter is optional. The OUTCELLS= data set will only be created when specified.

OUTCONSTRAINTS = *SAS-data-set*

Name (in the form *libref.member*) of the output data set containing information about the constraints. The values in this data set are determined by solving the table rounding problem. They are calculated from the input data sets using the OPTMODEL procedure. This data set contains the following fields:

Variable name	Type	Description
ConstraintId	numeric	Unique identifier for each constraint. Mandatory.
ConstraintLB	numeric	Constraint Lower Bound. Contains real numbers or missing values.
ConstraintUB	numeric	Constraint Upper Bound. Contains real numbers or missing values.
ConstraintTotal	numeric	The total value of the given constraint. Contains real numbers.

This parameter is optional. The OUTCONSTRAINTS= data set will only be created when specified.

OUTSUMMARY = *SAS-data-set*

Name (in the form *libref.member*) of the summary output data set. The values are summary statistics calculated from the OUTCELLS= data set. The values of these columns are calculated over all the cells using the weight multiplied by the absolute difference. This data set contains the following fields:

Variable name	Type	Description
TotWeightDiff	numeric	The total weighted difference. Contains real numbers.
AveWeightDiff	numeric	The average weighted difference. Contains real numbers.
MaxWeightDiff	numeric	The maximum weighted difference. Contains real numbers.

This parameter is optional. The OUTSUMMARY= data set will only be created when specified.

BASE = *real number*

Specifies the rounding base used by the rounding algorithm. The value must be a positive real number. This parameter is optional and the default value is 1.

OBJECTIVE = *character string*

The parameter OBJECTIVE determines the objective function used in the optimization problem. Valid values are FULL or SIMPLE. When the value of the parameter OBJECTIVE is FULL the optimization problem is solved using the full objective function. When the value of the parameter OBJECTIVE is SIMPLE, the optimization problem is solved using the simple objective function.
This parameter is optional and the default value is FULL.

SOLVER = *character string*

Allows to user to specify specific solver options for the SAS OPTMODEL procedure. This parameter is optional and the default value is blank.

OPTMODELLOG = *character string*

Specifies whether or not the OPTMODEL notes are written to the log. Valid values are YES and NO. This parameter is optional and the default value is NO.

OPTMODELSTATUS = *character string*

Specifies whether or not the OPTMODEL solution status parameters are written to the log. Valid values are YES and NO. This parameter is optional and the default value is NO.

6. Frequently Asked Questions

Q: I got an error or warning message. What does it mean?

A: Please consult the list 3.1 Common error and warning messages.

Q: How does G-Confid produce results for more than one survey variable at a time?

A: G-Confid can produce results for survey variables that are related in a linear combination, for example, the components of revenue that sum to total revenue. The input microdata file must be in skinny format. See the example 2.1 Transposing a file into skinny format.

Q: Does G-Confid calculate the *estimated totals* by domain? If so, does that mean I do not have to calculate the estimates using G-Est or other tabulation tool?

A: By default G-Confid calculates unweighted totals. By using a WEIGHT statement and specifying the variable with the estimation weights, G-Confid calculates weighted totals. In order to rely on these totals, we must include in the input microdata file all of the data that we use to calculate the estimates including macro-level adjustments if applicable. G-Confid does not calculate the estimates of the variance of the totals. The weighted totals appear as *TotalVar* in the *Outcell* file that PROC SENSITIVITY produces.

Q: My survey collects *waivers* from responding enterprises. How do I use that information?

A: Using G-Confid v1.06 or later, include a {0,1} binary numeric variable in the input microdata file. Assign the value 1 to an observation for which the enterprise supplied a waiver. Specify the name of the variable using either the WAIVER or PWAIVER statement of PROC SENSITIVITY. For more information, please see the 4.1.2 Using Waivers section of the Strategies module and the examples on 2.5 Waivers.

Q: In the opinion of my director, my survey includes *a domain that we do not have to protect*. How do we ensure that the sensitivity analysis does not determine that this domain requires protecting?

A: If the domain is at the level of an internal cell, then we use the same approach as a waiver. Using G-Confid v1.06 or later, include a {0,1} binary numeric variable in the input microdata file. If the observations contribute to the domain that we do not need to protect, then set this variable to 1. For more information, please see the 4.1.2 Using Waivers section of the Strategies module and the examples on 2.5 Waivers.

If the domain is at the level of a marginal cell, then we run PROC SENSITIVITY and then adjust the sensitivity of the cell that represents that domain. For more information, please see the 4.1.4 Reducing the Protection section of the Strategies module.

Q: My survey includes observations, the values of which were obtained using *imputation*. How do we decide if the domains that include these observations are safe to release?

A: First we have to decide whether the imputation results in a value that does not need protecting. Please see the 4.1.1 Deciding which data not to protect section of the Strategies module. If a particular observation is considered not to require protection, then we use the same approach as a waiver. Using G-Confid v1.06 or later, include a {0,1} binary numeric variable in the input microdata file. If the observations contribute to the domain that we do not need to protect, then set this variable to 1. For more information, please see the 4.1.2 Using Waivers section of the Strategies module and the examples on 2.5 Waivers.

Q: My microdata file includes *data with negative values*. How do I make use of negatively valued data?

A: Using G-Confid v1.07 or later, specify the ACCEPTNEGATIVE procedure option of PROC SENSITIVITY, and G-Confid will calculate the absolute values of the variables. Note that v1.06 and prior versions dropped any observation with a negative value. For more information, please see the examples on 2.7 Negative values.

Q: My microdata file includes data with negative values. I also have an auxiliary variable that I want to use as a *proxy variable*. How do I use an auxiliary variable in this way?

A: Include the auxiliary variable on the microdata file. For every observation for which we want the auxiliary variable to be considered for us, include the value of the auxiliary variable. By default, the proxy variable is defined as the absolute value of the analysis variable for 90% of the observations, and a proportion of the auxiliary variable for the other 10% of observations. For more information, please see the 3.9 Using a proxy variable section of the Details module. Please see also the examples on 2.7 Negative values.

Q: My survey represents a census of a population. Does that mean I do not use a variable that represents the estimation *weight*?

A: Even if no subsampling occurred, a variable representing the estimation weight may be used to represent adjustments due to nonresponse or calibration. Please consult your survey methodologist to identify and calculate a weight variable that suits your survey.

Q: My survey includes a variable that represents the estimation *weight*. How do I make use of this variable?

A: Using G-Confid v1.07 or later, include this variable on the input microdata file. Identify the name of this variable using a WEIGHT statement of PROC SENSITIVITY. By default, a LINEAR function will adjust the protection demanded by the target unit. For more information, please see the 4.2.2 Using a weight variable section of the Strategies module. Please see also the examples on 2.6 Survey weights.

Q: My survey includes a variable that represents the estimation *weight*. Which setting of the weight protection level parameter should I choose?

A: In general it is recommended to use the LINEAR weight protection level. This level permits the target enterprise to demand less protection as its weight differs from 1. LINEAR leads to publishing more data than STEP. Using STEP, the weight of the target becomes protective only as the weight nears 3 or as the weight decreases from 1. STEP leads to publishing more data than EXACT. We use EXACT only if we inform the external users of the weights that were used. For example, if we report that all units in a certain domain received a weight of 1.25 because 80% of units were selected into the sample, then we select EXACT. For more information, please see the 4.2.2 Using a weight variable section of the Strategies module and the examples on 2.6 Survey weights.

Q: Do I have to protect a domain with the value *zero*?

A: Publishing the value zero for a domain attributes to all contributors the value zero, if the variable is defined on non-negative values. Often in practice, the value zero is considered not to require protection. By default G-Confid does not use observations with the value zero. See the 4.1.1 Deciding which data not to protect section of the Strategies module.

Suppose instead we determine that a zero reveals information about the contributors that should not be revealed. We adjust the microdata to include one or more non-zero contributions. This approach requires care to select suitable values. Please consult your confidentiality advisor or the G-Confid team for assistance.

Q: I have a file of contributions of enterprises in *take-none strata*. The contributions were modelled using data from administrative sources as explanatory variables. Should I make use of these contributions in the sensitivity analysis, and if so then how?

A: We should make use of contributions in take-none strata, as these contributions enable us to publish more results safely. Include the take-none observations in the input microdata file. If the contributions are at the level of the microdata, then we include the enterprise identifier of each contributor in the identifier variable. If a set of take-none contributions comprise a single observation at the macro level, one observation per domain (i.e., per cell), then the identifier variable must be left blank (missing) and not contain a value, otherwise G-Confid will mistake the observation to refer to a single contributor.

Q: How can I *publish more results* without breaking the promise of confidentiality that we guarantee to our data providers?

A: Obtaining waivers, using weights, reorganizing output tables, including take-none data and macro-level adjustments all help us to publish more results. As a last resort, we may consider compromising our promise by reducing the protection that we offer to the business enterprises. For more information please see the 3.7 Aggregates section in Details and the 4.1.4 Reducing the protection in Strategies.

Q: I need to produce *results based on revised microdata*. How do I use G-Confid given that results using preliminary data were already released?

A: Run PROC SENSITIVITY on the set of revised data. Next, we create a cost variable on the *Outcell* file. Obtain the G-Confid suppression output file of the preliminary results. If for a particular cell, the result was suppressed in the preliminary results then set the cost equal to 1, and if it was published then set the cost equal to *TotalNoise*. When running the macro %SUPPRESS set the variables CVAR1 and CVAR2 to refer to the cost variable. If the revised results are similar to the preliminary results, then G-Confid builds a suppression pattern for the revised results that is similar to the suppression pattern of the preliminary results. This approach mitigates the risk of disclosure by relying on the published preliminary results to deduce the suppressed revised results. For more information please see 4.2.4 Producing revised results after the microdata have changed in the Strategies section.

Q: How do I *favour publishing* certain cells?

A: After running PROC SENSITIVITY, we create a cost variable on the *Outcell* file. For cells that we want to publish, we set an elevated value of the cost variable, for example, the value of the grand total of the table of results. For the other cells, we set the cost of the cell equal to its *TotalNoise*. When running the macro %SUPPRESS set the variables CVAR1 and CVAR2 to refer to the cost variable. For more information please see the 4.1.3 Favouring specific cells for publication section of the Strategies module.

Q: The values of the *TotalVar* variable do not agree with the point estimates of the results I plan to publish. Did I make a mistake?

A: Differences might exist between the microdata that were used for calculating the point estimates and the microdata used for confidentiality. The hierarchy might not exactly agree with the domain definitions used with estimation. A PROXYSIZE variable might have been specified when using PROC SENSITIVITY. A weight variable might not have been specified, or the WEIGHT statement was omitted.

Q: Some *domains are missing* that I expected to see in the *Outcell* file. Why?

A: Certain domains might not be specified in the hierarchy. One or more variables that define the domains might not be specified in the DIMENSION or BY statements.

Q: I used the same inputs with two different versions of G-Confid (or of SAS). Why do the *suppression patterns differ*? And why does it take longer to run using one version compared to the other?

A: Different versions of G-Confid do not generally lead to difference in runtime. Different suppression patterns may arise because the other of two equally optimal suppressions was selected during a separate run. Different production versions of SAS, or different maintenance versions of the same production version of SAS, rely on different SAS/OR solvers. These solvers can lead to very different runtimes.

Q: I ran the macro %SUPPRESS and the *Log window has not changed* for five minutes. Has G-Confid stopped working?

A: Not necessarily. First, ensure the Log window is active (a PROC PRINTTO statement was not used to redirect the information away from the Log window). The macro %SUPPRESS is allotted 10 minutes during which to identify an optimal circuit of suppressed cells. A new line in the Log should appear at least every 10 minutes possibly plus a few seconds for the latest iteration to be finalized. If the Log window is active and no new line appears in the Log after about 10½ minutes, it is possible that %SUPPRESS is stuck in an infinite loop and G-Confid is not able to run to completion.

In this circumstance, we recommend to interrupt the program using the (!) icon of the SAS session, setting SCALECOST=SCALE and resubmitting the macro %SUPPRESS. If the problem persists, please contact the G-Confid team for assistance.

Q: How long does G-Confid take to run? *Can G-Confid run any faster*?

A: Very large jobs, on the order of 100,000+ cells and 100,000+ linear constraints, and with 10+% sensitive cells, may require more than 24 hours to run. Ensure that the SAS memory allocation is appropriate for your computer or server. (Note that SAS for use at StatCan permits 2 GB RAM by default. If your computer or server has 8 GB RAM then consider requesting a higher allocated RAM for SAS such as 6 GB).

The SAS procedure PROC OPTMODEL, on which the macro %SUPPRESS is based, is not a high performance procedure. The sequential optimization of the macro %SUPPRESS cannot benefit from parallel processing on an HP-configured server grid.

When calling the macro %SUPPRESS we can include the parameter setting SCALECOST=SCALE which rescales the informational value across a narrower interval prior to being used as the cost variable. This tends to yield a less efficiently built suppression pattern, although typically in a considerably shorter runtime.

Q: One variable needs to be greater than or equal to another variable. How do I *respect an inequality* using G-Confid?

A: On the microdata file, derive a third variable as the difference. Include this variable in the hierarchy such that the largest variable is the sum of the other two.

Example :
If X≥Y, create a variable Z=X-Y.
In other words, we have X=Z+Y.
We can therefore indicate in the hierarchy that X is the sum of Z and Y.

Glossary

This glossary contains definitions for some terms that are important for confidentiality purposes and for G-Confid. The French equivalent of the English term is indicated in parentheses for each definition. Many of the definitions are based on Elliot et al. (2009).

A

Adversary (Adversaire)
A data user who attempts to link a respondent to a microdata record or make attributions about particular population units from aggregate data. Adversaries may be motivated by a wish to discredit or otherwise harm the National Statistical Institute, the survey or the government in general, to gain notoriety or publicity, or to gain profitable knowledge about particular respondents.

Aggregate (Agrégat)
An aggregate is a union of two or more cells sharing common values for all classification variables except one.

Approximate disclosure (Divulgation approximative)
Approximate disclosure happens if a user is able to determine an estimate of a respondent value that is close to the real value. If the estimator is exactly the real value the disclosure is exact.

B

Bounds (Bornes)
The range of possible values of a cell in a table of frequency counts where the cell value has been perturbed or suppressed. Where only margins of tables are released it is possible to infer bounds for the unreleased joint distribution.

C

Calculated interval (Intervalle calculé)
The interval containing possible values for a suppressed cell in a table, given the table structure and the values published.

Cell suppression (Suppression de cellules)
In tabular data the cell suppression SDC method consists of primary and complementary (secondary) suppression. Primary suppression can be characterised as withholding the values of all risky cells from publication, which means that their value is not shown in the table but replaced by a symbol such as '×' to indicate the suppression. According to the definition of risky cells, in frequency count tables all cells containing small counts and in tables of magnitudes all cells containing small counts or presenting a case of dominance have to be primary suppressed. To reach the desired protection for risky cells, it is necessary to suppress additional non- risky cells, which is called complementary (secondary) suppression. The pattern of complementary suppressed cells has to be carefully chosen to provide the desired level of ambiguity for the risky cells with the least amount of suppressed information.

Complementary suppression (Suppression complémentaire)
Synonym of secondary suppression.

Complete disclosure (Divulgation complète)
Synonym of exact disclosure.

D

Data protection (Protection des données)
Data protection refers to the set of privacy-motivated laws, policies and procedures that aim to minimise intrusion into respondents' privacy caused by the collection, storage and dissemination of personal data.

Data utility (Utilité des données)
A summary term describing the value of a given data release as an analytical resource. This comprises the data's analytical completeness and its analytical validity. Disclosure control methods usually have an adverse effect on data utility. Ideally, the goal of any disclosure control regime should be to maximise data utility whilst minimising disclosure risk. In practice disclosure control decisions are a trade-off between utility and disclosure risk.

Disclosive cells (Cellules divulgatrices)
Synonym of risky cells.

Disclosure (Divulgation)
Disclosure relates to the inappropriate attribution of information to a data subject, whether an individual or an organisation.

Disclosure control methods (Méthodes de contrôle de la divulgation)
There are two main approaches to control the disclosure of confidential data. The first is to reduce the information content of the data provided to the external user. For the release of tabular data this type of technique is called restriction based disclosure control method and for the release of microdata the expression disclosure control by data reduction is used. The second is to change the data before the dissemination in such a way that the disclosure risk for the confidential data is decreased, but the information content is retained as much as possible. These are called perturbation based disclosure control methods.

Disclosure limitation methods (Méthodes de limitation de la divulgation)
Synonym of disclosure control methods.

Disclosure risk (Risque de divulgation)
A disclosure risk occurs if an unacceptably narrow estimation of a respondent's confidential information is possible or if exact disclosure is possible with a high level of confidence.

Dissemination (Diffusion)
Supply of data in any form whatever: publications, access to databases, microfiches, telephone communications, etc.

Dominance rule (Règle de dominance)
Synonym of NK rule.

E

Exact disclosure (Divulgation exacte)
Exact disclosure occurs if a user is able to determine the exact attribute for an individual entity from released information.

F

G

H

I

Intruder (Intrus)
1) Synonym of adversary. 2) An adversary who contributes to the data under attack, such as a contributor to a cell.

J

K

L

Linear constraint (Contrainte linéaire)
Constraint expressed in linear terms. These constraints may be equations or inequalities. For example: $X+Y=Z$ or $X+2Y-Z \leq 0$.

Linear programming (Programmation linéaire)
The procedure used in maximising or minimising a linear function of several variables when these variables, or some of them, are subject to constraints expressed in linear terms: these may be equations or inequalities. The term 'programming' in this context indicates a schedule of actions.

Lower bound (Borne inférieure)
The lowest possible value of a cell in a table of frequency counts where the cell value has been perturbed or suppressed.

LP solver (Solveur PL)
Linear programming strategy. Each version of SAS has its own solver.

M

Microdata (Microdonnées)
A microdata set consists of a set of records containing information on individual respondents or on economic entities.

N

NK rule (Règle NK)
A cell is regarded as confidential, if the *n* largest units contribute more than *k* % to the cell total, e.g., *n*=2 and *k*=85 means that a cell is defined as risky if the two largest units contribute more than 85 % to the cell total. The *n* and *k* are given by the statistical authority. In some National Statistical Institutes the values of *n* and *k* are confidential.

NSI (INS)
Abbreviation for National Statistical Institute.

O

Oversuppression (Suppression excessive)
A situation that may occur during the application of the technique of cell suppression. This denotes the fact that more information has been suppressed than strictly necessary to maintain confidentiality.

P

Partial disclosure (Divulgation partielle)
Synonym of approximate disclosure.

P-percent rule (Règle p-pourcent)
A PQ rule where *q* is 100 %, meaning that from general knowledge any respondent can estimate the contribution of another respondent to within 100 % (i.e., knows the value to be nonnegative and less than a certain value which can be up to twice the actual value).

PQ rule (Règle PQ)
It is assumed that prior to publication of tabular data the contribution of one individual to a cell total can be estimated to within *q* per cent (a priori relative error in estimating the individual contribution). If after publication of the statistic the value can be estimated to within *p* per cent (a posteriori relative error in estimating the individual contribution), the cell is declared as confidential. The parameters *p* and *q* are determined by the statistical authority. In some National Statistical Institutes the values of *p* and *q* are confidential.

Perturbing the data (Perturbation des données)
This process involves changing the data in some systematic fashion, with the result that the figures are insufficiently precise to disclose information about individual cases.

Primary suppression (Suppression primaire)
This technique can be characterized as withholding all disclosive cells from publication, which means that their value is not shown in the table, but replaced by a symbol such as '×' to indicate the suppression. According to the definition of disclosive cells, in frequency count tables all cells containing small counts and in tables of magnitudes all cells containing small counts or representing cases of dominance have to be primary suppressed.

Protection interval (Intervalle de protection)
Interval around the true value of the target that indicates the limits of what is considered a disclosure. If an intruder can estimate the value of the target with more precision than the Protection interval, then the cell is considered sensitive.

Q

R

Risky cells (Cellules à risque)
Cells of a table which are non-publishable due to the risk of statistical disclosure. By definition there are three types of risky cells: small counts, dominance and complementary suppression cells.

S
SDC (CDS) Abbreviation for Statistical Disclosure Control.
Secondary suppression (Suppression secondaire) To reach the desired protection for risky cells, it is necessary to suppress additional non- risky cells, which is called secondary suppression or complementary suppression. The pattern of complementary suppressed cells has to be carefully chosen to provide the desired level of ambiguity for the disclosive cells at the highest level of information contained in the released statistics.
Sensitive cells (Cellules sensibles) A sensitive cell is a cell for which knowledge of the value would permit an unduly accurate estimate of the contribution of an individual respondent. Sensitive cells are identified by the application of a sensitivity rule such as the NK rule or the PQ rule to their microdata.
Sensitivity (Sensibilité) Sensitivity is a measure that determines whether a particular cell is sensitive and how much protection it would require. If the sensitivity is positive, then the cell is considered sensitive. Otherwise, the cell is non-sensitive.
Statistical disclosure (Divulgation statistique) Statistical disclosure is said to take place if the dissemination of a statistic enables the external user of the data to obtain a better estimate for a confidential piece of information than would be possible without it.
Statistical Disclosure Control (Contrôle de la divulgation statistique) Statistical Disclosure Control (SDC) techniques can be defined as the set of methods to reduce the risk of disclosing information on individuals, businesses or other organisations. Such methods are only related to the dissemination step and are usually based on restricting the amount of or modifying the data released.
Suppressed cells (Cellules supprimées) Cell that was suppressed from a table because it was sensitive (primary suppression) or was suppressed to protect other sensitive cells in the table (secondary suppression).
Suppression One of the most commonly used ways of protecting sensitive cells in a table is via suppression. It is obvious that in a row or column with a suppressed sensitive cell, at least one additional cell must be suppressed, or the value in the sensitive cell could be calculated exactly by subtraction from the marginal total. For this reason, certain other cells must also be suppressed. These are referred to as secondary suppressions. While it is possible to select cells for secondary suppression manually, it is difficult to guarantee that the result provides adequate protection.
Suppression pattern (Patron de suppression) The joint set of primary and secondary suppressions is called a suppression pattern.
Suspect Synonym of Intruder.
T
Tabular data (Données tabulaires) Aggregate information on entities presented in tables.
Target (Cible) Contributor to a cell that is the most at risk of disclosure based on the worst target-intruder case scenario.
U
Upper bound (Borne supérieure) The highest possible value of a cell in a table of frequency counts where the cell value has been perturbed or suppressed.

V
W
Waiver (Renonciation) Instead of suppressing tabular data, some agencies ask respondents for permission to publish cells even though doing so may cause these respondents' sensitive information to be estimated accurately. Waivers are signed records of the respondents' granting permission to publish such cells. This method is most useful with small surveys or sets of tables involving only a few cases of dominance, where only a few waivers are needed. Of course, respondents must believe that their data are not particularly sensitive before they will sign waivers.
X
Y
Z

References

Boudreau, J.-R., Filep, K., Liu, L. (2004) Iterative Rounding for Large Frequency Tables. *Proceedings of the American Statistical Association, Joint Statistical Meeting: Government Statistics Section*, Toronto.

Cox, J.L. and Sande, G. (1979). Techniques for Preserving Statistical Confidentiality. *Proceedings of the 42nd Session of the International Statistical Institute*, Manila.

Elliot, M., Hundepool, A., Schulte Nordholt, E. et al. (2009) Glossary on Statistical Disclosure Control. CASC Project, Statistics Netherlands.

Gray, D. (2016). Precision Threshold and Noise: An Alternative Framework of Sensitivity Measures. In: Domingo-Ferrer, J. and Pejić-Bach, M. (Eds.) *Privacy in Statistical Databases,* LNCS 9867*,* Dubrovnik. 15-27

INSEE (2010). Guide to Statistical Confidentiality.

Tambay, J.-L. and Fillion, J.-M. (2011) New business survey confidentiality software G-Confid. *Joint UNECE/Eurostat Work Session on Statistical Data Confidentiality, Tarragona*, Working Paper 12

Tambay, J.-L. and Fillion, J.-M. (2013) Strategies for processing tabular data using the G-Confid cell suppression software. *Proceedings of the Survey Methods Research Section, American Statistical Association Joint Statistical Meetings*, Montreal.

Willenborg, L. and De Waal, T. (2001) Elements of Statistical Disclosure Control. Springer Verlag Lecture Notes in Statistics.

Wright, P. (2017) Disclosure control that accounts for survey realities: assessing the risk using G-Confid. *Joint UNECE/Eurostat Work Session on Statistical Data Confidentiality, Skopje, Macedonia (FYROM), 20-22 September 2017*